



Centro de Investigación y de Estudios Avanzados
del Instituto Politécnico Nacional
Unidad Guadalajara

Arquitectura Autonómica, Autoensamblable y Autorreplicable para Sistemas IoT

Tesis que presenta:

David Emmanuel Ramírez Tovar

para obtener el grado de:

Maestro en Ciencias

en la especialidad de:

Ingeniería Eléctrica

Director de Tesis

Dr. Mario Angel Siller González Pico

CINVESTAV del IPN Unidad Guadalajara, Guadalajara, Jalisco, Agosto de 2020.

Arquitectura Autonómica, Autoensamblable y Autorreplicable para Sistemas IoT

**Tesis de Maestría en Ciencias
Ingeniería Eléctrica**

Por:

David Emmanuel Ramírez Tovar

Ingeniero en Sistemas Computacionales

Instituto Tecnológico Superior de Arandas 2013-2018

Becario de Conacyt, expediente No. 925351

Director de Tesis

Dr. Mario Angel Siller González Pico

CINVESTAV del IPN Unidad Guadalajara, Agosto de 2020.

Resumen

El número y diversidad de dominios de aplicación del IoT aumenta constantemente a medida que se incrementan los dispositivos conectados a la red, las tecnologías involucradas en el diseño y surgen nuevos escenarios de uso para dichos sistemas. Esto ha derivado en nuevos retos para la administración, mantenimiento y evolución de los sistemas IoT, en los que se han involucrado distintos enfoques para abordarlos tales como el cómputo autónomo. Estos retos, entre otras cosas, han originado el planteamiento de propiedades adicionales a las que originalmente fueron consideradas en la conceptualización de los sistemas autónomos. En este trabajo se propone una arquitectura autónoma para sistemas IoT en las que se definen dos nuevas propiedades: Autoensamblaje y Autorreplicación. El diseño se basa en un enfoque multi-agente, inspiración biológica y en el principio de potencial de máxima funcionalidad: “capacidad instalada, capacidad habilitada”. La unidad definida para realizar el Autoensamblaje y Autorreplicación es a nivel clase y se realiza a partir del código fuente. El principio de potencial de máxima funcionalidad se aplica a comunidades de agentes o dispositivos IoT e involucra el uso de clases activas y pasivas de código. El diseño arquitectónico y algorítmia propuestos fueron validados a través de la implementación de “Quines” en Python, pruebas estáticas de código y simulación basada en el modelado de agentes apegado al estándar de FIPA. Los escenarios experimentales de prueba consistieron en sistemas ciberfísicos de cultivo hidropónicos para uso en entornos urbano. Para la simulación se definieron comunidades de 25 a 50 agentes y de 2 a 14 clases funcionales de código. En los resultados se observa que el número de ciclos requeridos de simulación aumenta razonablemente en función del tamaño de la comunidad de agentes y número de clases funcionales involucradas en el Autoensamblaje y la Autorreplicación. En términos de implementación este incremento no resulta significativo y estaría acotado por las capacidades en hardware de los dispositivos. Por lo anterior, la arquitectura propuesta resulta escalable a escenarios reales de diversos dominios de aplicación del IoT. Finalmente, se muestra que es posible que un sistema IoT pueda replicar parte de su funcionamiento y arquitectura de aplicación en otro sistema que así lo requiera.

Abstract

The number and diversity of IoT application domains is constantly increasing as devices connected to the network increase, the technologies involved in the design and new usage scenarios for such systems emerge. This has led to new challenges for the administration, maintenance and evolution of IoT systems, in which different approaches have been involved to address them, such as autonomic computing. These challenges, among other things, have originated the proposal of additional properties to those that were originally considered in the conceptualization of autonomic systems. In this work, an autonomous architecture for IoT systems is proposed, in which two new properties are defined: Self-assembly and Self-replication. The design is based on a multi-agent approach, biologically inspired and on the principle of maximum functionality potential: “installed capacity, enabled capacity”. The unit defined to carry out Self-assembly and Self-replication is at the class level and is carried out from the source code. The principle of maximum functionality potential applies to communities of agents or IoT devices and involves the use of passive and active classes of code. The proposed architectural design and algorithm were validated through the implementation of “Quines” in Python, static code testing, and simulation based on agent modeling meeting the FIPA standard. The experimental test scenarios consisted of cyberphysical hydroponic systems for agronomic urban sites. For the simulation, communities of 25 to 50 agents and 2 to 14 functional code classes were defined. The results show that the number of simulation cycles required increases reasonably according to the size of the agent community and the number of functional classes involved in Self-assembly and Self-replication. In terms of implementation, this increase is not significant and would be limited by the hardware capabilities of the devices. Therefore, the proposed architecture is scalable to real scenarios from various IoT application domains. Finally, it is shown that an IoT system can replicate part of its functionality and application architecture in another system that requires it.

Dedicatoria

A mi madre Lorena, por su apoyo incondicional y alentarme a lograr todos mis sueños. A mi hermana Quetzally y mi hermano Hugo, por impulsarme a seguir adelante. A Dios, por permitirme llegar tan lejos.

Agradecimientos

A todos los que me apoyaron en el camino y me ayudaron a subir a hombros de gigantes.

A mi asesor de tesis, el Dr. Mario Ángel Siller González Pico, por todo el apoyo durante el desarrollo de la tesis.

A mis mejores amigos y roomies, por estar siempre ahí y escucharme.

Al grupo de redes de CINVESTAV Guadalajara, que más que grupo, es una familia.

A mis compañeros y amigos del grupo Niclab, por tratarme como parte del grupo.

A todos mis amigos del laboratorio de computación, por esas largas charlas y orientación.

A todos mis maestros, quienes me educaron en toda mi vida académica y nunca desistieron al enseñarme.

A los dos Institutos Tecnológicos que me formaron.

A todos aquellos que creyeron en mí.

Finalmente, quiero agradecer al CONACYT por el apoyo económico brindado, con el cual se logró estudiar esta maestría.

Muchas Gracias a todos.

Índice de Contenido

1	Introducción	12
1.1	Introducción	12
1.2	Definición del problema	13
1.3	Motivación	14
1.4	Hipótesis	14
1.5	Objetivo general	15
1.6	Objetivos específicos	15
1.7	Contribuciones	15
1.8	Estructura del documento	15
2	Marco Teórico	17
2.1	Sistemas Ciberfísicos	17
2.2	IoT	18
2.3	Sistemas Multi-Agente	19
2.4	Middleware	20
2.5	Computo Autónomo	21
2.6	Diseño Arquitectónico	23
2.7	Metodología de diseño	24
2.8	Arquitectura Orientada a Servicios (SOA)	24
2.9	Arquitectura Orientada a Microservicios (MSA)	25
2.10	Red definida por Software (SDN)	26
2.11	Autorreplicación	27
2.12	Quines	29
2.13	Blockchain	29
2.14	Resumen Capítulo 2	30
3	Estado del Arte	32
3.1	Arquitecturas relacionadas con sistemas IoT	32
3.2	Autoarquitectura	34
3.3	Sistemas Autorreplicables y Autoensamblables	35
4	Propuesta	36
4.1	Metodología de la investigación	36
4.1.1	Obtención de conocimiento de dominio	36
4.1.2	Revisión del estado del arte	37
4.1.3	Estudio de arquitecturas de referencia	37
4.1.4	Identificar principios de diseño	37
4.1.5	Diseño de la arquitectura	37
4.1.6	Selección del mecanismo de validación	37

4.1.7	Validación	37
4.1.8	Presentación de resultados	37
4.2	Metodología de diseño arquitectónico	38
4.2.1	Introducción	38
4.2.2	Metodología de diseño arquitectónico	38
4.2.3	Primitivas de diseño	39
4.2.4	Requerimientos funcionales	40
4.2.5	Requerimientos no funcionales	43
4.3	Propuesta de arquitectura	44
4.3.1	Componentes eliminados/derogados	46
4.3.2	Componentes modificados	47
4.3.3	Componentes agregados	48
4.3.4	Vista logica	48
4.3.4.1	Comunicaciones	48
4.3.4.2	Ensamblaje de Dispositivo	49
4.3.5	Vista de despliegue	52
4.3.5.1	Pila de Protocolos	52
4.3.5.2	Estructura de código	52
4.3.5.3	Estructura de Modulos	52
4.3.5.4	Ensamblaje de código	56
4.3.6	Vista de procesos	58
4.3.6.1	Arribo de código fuente o dispositivos	58
4.3.7	Vista física	61
4.3.8	Vista de escenarios	62
4.3.9	Comparativa	63
5	Experimentación y Análisis de Resultados	64
5.1	Descripción del Caso de Estudio	64
5.2	Experimento Estático	65
5.2.1	Primer escenario	65
5.2.2	Segundo escenario	67
5.3	Experimento Dinamico	68
5.3.1	GAMA Platform	68
5.3.2	Experimentación	69
5.3.2.1	Primer bloque de experimentos	70
5.3.2.2	Segundo bloque de experimentos	75
5.3.2.3	Análisis de resultados de simulación	81
6	Conclusiones y Trabajo Futuro	82
6.1	Discusión	82
6.1.1	Contribución	82
6.1.2	Objetivos logrados	83
6.2	Trabajo futuro	83
7	Anexo	88
7.1	Imágenes	88
7.2	Código de Simulación	94

Índice de Figuras

2.1	Diagrama de composición de Sistemas Ciber físicos	17
2.2	Diagrama Ejemplo de Constitución de Sistema Ciber Físico.	18
2.3	Diseño lógico de un sistema IoT.	19
2.4	Interacción Agente-Ambiente.	20
2.5	Taxonomía de middleware	21
2.6	Bucle de control MAPE-K	22
2.7	Modelo de vistas 4+1 de Kruchten	24
2.8	Pares complementarios de algunas bases nitrogenadas	27
2.9	Proceso de replicación del ADN	28
2.10	Representación de una cadena de bloques	29
3.1	Arquitectura IoT Autonomica	33
3.2	SCRAIoT	33
4.1	Metodologia	36
4.2	Generación y prueba de diseño arquitectónico	38
4.3	Arquitectura autonómica base	44
4.4	Propuesta	46
4.5	Obtención de datos de un sistema IoT	49
4.6	Proceso de autosanación	50
4.7	Ensamblaje de dispositivo IoT	51
4.8	Eventos desencadenadores de autoensamblaje	51
4.9	Pila de Protocolos	52
4.10	Estructura de código fuente	53
4.11	Estructura de módulo de información	53
4.12	Estructura de módulo de clases funcionales	54
4.13	Estructura de diseño por componentes	54
4.14	Estructura de módulo de clases hilos	55
4.15	Proceso de ensamblaje de código	56
4.16	Proceso de ensamblaje de clases activas y pasivas	57
4.17	Ensamblaje de código fuente nuevo de Dispositivo	58
4.18	Sustitución de código fuente de dispositivo entrante	58
4.19	Replicación directa de código	59
4.20	Ensamblaje de CF proveniente de servicios REST	59
4.21	Autoensamblaje y Autorreplicación de código fuente solicitado	60
4.22	Posible escenario físico de sistema IoT	61
4.23	Vista integradora de arquitectura propuesta.	62
5.1	Distribución física de los sistema IoT	65

5.2	Cama de cultivo hidropónico de CINVESTAV Guadalajara	66
5.3	Estructura de código fuente.	66
5.4	Ensamblaje de código foráneo	67
5.5	Fusión de clases de código foráneo	67
5.6	Estructura de código fuente	68
5.7	Fusión de clases de código de dispositivo entrante	69
5.8	Lista de capacidades	70
5.9	Lista de clases activas	70
5.10	Lista de clases pasivas.	70
5.11	Replicaciones de primer experimento	71
5.12	Comunidad compuesta por 25 agentes con 2 clases a replicar	72
5.13	Clases activas ensambladas, primer experimento.	72
5.14	Clases pasivas ensambladas, primer experimento.	72
5.15	Replicaciones de segundo experimento	73
5.16	Comunidad compuesta por veinticuatro agentes con siete clases a replicar	73
5.17	Clases activas ensambladas, segundo experimento.	74
5.18	Clases pasivas ensambladas, segundo experimento.	74
5.19	Replicaciones de tercer experimento	75
5.20	Comunidad compuesta por veinticinco agentes con catorce clases a replicar	75
5.21	Clases activas ensambladas, tercer experimento.	76
5.22	Clases pasivas ensambladas, tercer experimento.	76
5.23	Replicaciones de cuarto experimento	77
5.24	Comunidad compuesta por cincuenta agentes con dos clases a replicar	77
5.25	Clases activas ensambladas, cuarto experimento.	78
5.26	Clases pasivas ensambladas, cuarto experimento.	78
5.27	Replicaciones de quinto experimento	78
5.28	Comunidad compuesta por cincuenta agentes con siete clases a replicar	79
5.29	Clases activas ensambladas, quinto experimento.	79
5.30	Clases pasivas ensambladas, quinto experimento.	79
5.31	Replicaciones de sexto experimento	80
5.32	Comunidad compuesta por cincuenta agentes con catorce clases a replicar	80
5.33	Clases activas ensambladas, sexto experimento.	81
5.34	Clases pasivas ensambladas, sexto experimento.	81

Índice de Tablas

2.1	Bloques funcionales de un sistema IoT	19
2.2	Cinco tareas elementales de Paul Horn	21
2.3	Cuatro propiedades de J. Kephart	22
2.4	comparativa MSA-SOA	26
3.1	Comparativa de arquitecturas IoT	34
4.1	Primitiva de diseño 1	39
4.2	Primitiva de diseño 2	39
4.3	Primitiva de diseño 3	39
4.4	Requerimiento Funcional 1	40
4.5	Requerimiento Funcional 2	40
4.6	Requerimiento Funcional 3	40
4.7	Requerimiento Funcional 4	41
4.8	Requerimiento Funcional 5	41
4.9	Requerimiento Funcional 6	41
4.10	Requerimiento Funcional 7	41
4.11	Requerimiento Funcional 8	41
4.12	Requerimiento Funcional 9	42
4.13	Requerimiento Funcional 10	42
4.14	Requerimiento Funcional 11	42
4.15	Requerimiento Funcional 12	42
4.16	Requerimiento No Funcional 1	43
4.17	Requerimiento No Funcional 2	43
4.18	Requerimiento No Funcional 3	43
4.19	Requerimiento No Funcional 4	43
4.20	Requerimiento No Funcional 5	44
4.21	Requerimiento No Funcional 6	44
4.22	Comparativa de arquitecturas IoT y Propuesta	63

Capítulo 1

Introducción

1.1 Introducción

El internet es la red informática existente más grande, y ha evolucionado a lo largo de los años desde su nacimiento en 1969 como la primera red de computadoras con nodo principal en la Universidad de California en Los Ángeles, dicha evolución ha permitido su accesibilidad, según un informe de We Are Social [1] , durante el año 2019 existían 4.388 millones de usuarios conectados, la población estimada de dicho año es aproximada a 7.700 millones de personas, lo que significa que ya desde ese momento más de la mitad de la población mundial estaba conectada a Internet, se espera que para finales del año 2020 el 59% de la población mundial esté conectada a Internet.

Actualmente Internet tiene un impacto social muy profundo a distintas escalas, abarcando desde los sectores laborales y educativos hasta el entretenimiento y ocio, esto a su vez provoca que una gran variedad de campos de dominio evolucionen en una integración con el propio internet, esto se ve reflejado principalmente en los sistemas ciberfísicos.

Los sistemas ciberfísicos se caracterizan por ser un mecanismo o sistema físico controlado por algoritmos computacionales, normalmente integrados con internet o simplemente conectados a alguna red computacional, noción de la cual surgen los sistemas IoT, por su parte, el internet de las cosas (por sus siglas en inglés IoT) es un concepto propuesto en 1999, por Kevin Ashton [2], en el Auto-ID Center del MIT, hace referencia a la interconexión de objetos cotidianos con internet, y es usada ampliamente para la automatización, desde la domótica (automatización de hogares), hasta la industria 4.0 (automatización industrial), pasando por campos como la agronomía rural y urbana, sistemas médicos y casi cualquier dominio ayudando a la simplificación de procesos y tareas.

Por sus amplios usos y su gran potencial en la actualidad la conexión de dispositivos a internet, así como su interconexión, es un tema difícil de abordar dentro del contexto de estudio del IoT ya que el número de los dispositivos activos crece exponencialmente. Según datos recabados se espera que para finales del 2020 la cifra de dispositivos interconectados sea de 30,000 millones de dispositivos, según la Industrial Internet Consortium [3], información con la que se pueden dilucidar el

impacto en el área de diseño, administración y mantenimiento, así como el reto de integración con nuevos sistemas.

Por lo tanto, un aspecto a tomar en cuenta es que, con la integración de elementos informáticos en la vida diaria, surgen nuevos tipos de sistemas que pueden tener la característica de la ubicuidad, característica que aumenta la dificultad en la configuración, monitoreo y administración en general de dichos dispositivos, debido al difícil acceso a los dispositivos que esto implica; además de estar sujetos a los retos de tener que desarrollar una arquitectura nueva para cada sistema, aun cuando existen sistemas muy similares.

Para este tipo de casos no existe un mecanismo que permita automatizar la generación de arquitecturas, abordándolo dentro del estudio de la ingeniería de sistemas. En sistemas bajo el mismo campo de dominio y de funcionalidad similar, existen múltiples intersecciones en la arquitectura, es decir, muchos módulos muy similares entre las mismas, este problema, aunado al número creciente de dispositivos conectados a internet y el crecimiento de la población que demanda dichos dispositivos y sistemas.

1.2 Definición del problema

Dentro del campo del desarrollo de software y sistemas IoT existen diversos problemas, principalmente ligados a cambios de requerimientos, ajustados tiempos de entrega, falta de funcionalidad o flexibilidad en el sistema y problemas de escalado. El punto anterior mencionado es un problema especialmente importante, ya que un sistema completo puede verse obsoleto al no poderse adaptar a los cambios en su ambiente.

De la misma manera un problema emergente de la hiperconexión es la creciente demanda de nuevos sistemas IoT, lo que se traduce en la alta demanda de arquitecturas de aplicación que los definan, implicando que se realice muchas veces trabajo similar, es decir, se diseñen muchas arquitecturas similares, que bien podrían partir de aquello que tengan en común y solo desarrollar aquello que las diferencie.

Además, las arquitecturas convencionales son difíciles de modificar, al no contar con una estandarización de modularidad, por lo que el modificar alguna sección de estas puede ser muy costoso e imposibilita el Autoensamblaje entre arquitecturas. Este último término hace referencia a una evolución arquitectónica automática, al tener dos arquitecturas similares con algunos elementos que las diferencian, se podrían fusionar para dar como resultado un nuevo sistema que tenga las capacidades de los sistemas que se tomaron para su generación.

Ya que actualmente existen propuestas arquitectónicas que habilitan las propiedades del cómputo autónomo en los sistemas IoT es necesario expandir dichas capacidades a un área de generación de arquitecturas de forma automática, es decir, que el desarrollo de sistemas IoT sea más rápido y logre asegurar escalabilidad y het-

erogeneidad. La construcción de una arquitectura a partir de otras de sistemas similares en el mismo campo de dominio es referida como Autorreplicación, esta propiedad puede ayudar al nacimiento de una nueva arquitectura y sistema a través de conocimiento previo.

Algunas de las preguntas de investigación que surgieron en la definición del problemas son las siguientes:

- ¿Qué ejemplos de replicación existen en la naturaleza?
- ¿Qué necesita un sistema para replicarse?
- ¿Qué ejemplos de autoensamblaje existen en la naturaleza?
- ¿Cómo debe ser un sistema que se pueda autoensamblar?
- ¿Existe alguna relación entre el autoensamblaje y la autorreplicación?
- ¿Es necesario seguir alguna estructura para estas tareas?

1.3 Motivación

Resulta importante el proponer un diseño arquitectónico Autoensamblable y Autorreplicable ya que ayudará principalmente al desarrollo de sistemas IoT, así como encontrar garantías de buen funcionamiento al trabajar en sistemas cambiantes o diversos que podrían agotar rápidamente la elasticidad del sistema. Aprovechando las bondades de las arquitecturas más vanguardistas y adicionando las propiedades antes descritas, se busca encaminar el área de desarrollo de sistemas IoT a un área más eficiente y que aspire a la realización automática de arquitecturas, automatizando la creación de sistemas IoT; así indirectamente también ayudando a la expansión de sistemas IoT a nuevos campos de dominio y logrando una estandarización en la que se puedan basar nuevos algoritmos de optimización, seguridad, funcionamiento en general, ó inclusive de cómputo autónómico.

A su vez, gracias a la escalabilidad de la que se dotarían los sistemas con estas propiedades, y la estandarización en la definición de sus componentes, los sistemas podrían ver expandida la diversidad de aplicaciones que pueden implementar con la misma arquitectura de software y hardware. Un ejemplo a pequeña escala de esto podría ser un teléfono inteligente, que gracias a la estandarización de sus interfaces y componentes puede integrar nuevo software (aplicación móvil) que lleve a cabo tareas no contempladas en su diseño original.

1.4 Hipótesis

Basándonos en una arquitectura para sistemas IoT podemos aprovechar sus características y evolucionarla al adicionar dos nuevas capacidades: la Autorreplicación y el Autoensamblaje. Estas capacidades que nos ayudaran a poder desarrollar arquitecturas para sistemas IoT de manera automática, al auxiliarse de otras arquitecturas que cuenten con las mismas capacidades y estén bajo el mismo campo o campos de dominio. Además de ayudar a extender la elasticidad del sistema al

modularizar la arquitectura y auxiliarse del Autoensamblaje para realizar cambios en tiempo de ejecución del sistema, cambios que podrían abarcar desde actualizaciones al sistema hasta integración de nuevas funcionalidades no previstas en el diseño original. Tomando como inspiración la replicación y el ensamblaje de algunos sistemas biológicos.

1.5 Objetivo general

Proponer un diseño arquitectónico que incluya las propiedades de Autoensamblaje y Autorreplicación en sus premisas de diseño partiendo de una arquitectura autónoma para sistemas IoT, logrando ayudar a la generación de nuevos sistemas bajo el mismo o los mismos campos de dominio y ampliando la escalabilidad de estos sistemas con el ensamblaje de dispositivos en tiempo real.

1.6 Objetivos específicos

- Estudiar las propiedades de los sistemas IoT y sus patrones arquitectónicos.
- Identificar y definir mecanismos de Autoensamblaje y de Autorreplicación.
- Desarrollar una metodología de diseño arquitectónico para arquitecturas Autoensamblables y Autoreplicables.
- Identificar una arquitectura IoT autónoma para agregar las propiedades de Autoensamblaje y Autoreplicación.
- Desarrollar una arquitectura de referencia para el diseño de sistemas autónomos, Autoensamblables y Autoreplicables.
- Validar la propuesta arquitectónica mediante casos de estudio usando implementaciones físicas (desarrollo de sistema ciberfísico agrónomo) o Simulaciones en plataforma de modelo basado en agentes.

1.7 Contibuciones

- Extensión a las arquitecturas autónomas.
- Propuesta de estandarización modular para una arquitectura IoT.
- Algoritmos de autorreplicación y autoensamblaje.
- Modelado basado en agentes de sistemas IoT autónomos.

1.8 Estructura del documento

En el capítulo 2 se muestra el marco teórico relacionado a las áreas afín a este trabajo de investigación, en el capítulo 3 se muestra el estado del arte relacionado a arquitecturas IoT autónomas, basadas en agente, propuesta de Autoarquitectura, y sistemas Autorreplicables y Autoensamblables, dentro del capítulo 4 se detalla

la propuesta arquitectónica y se analizan distintas vistas de esta, en el capítulo 5 se muestra la experimentación basada en casos de estudio y en el capítulo 6 se presentan las conclusiones y trabajo futuro.

Capítulo 2

Marco Teórico

2.1 Sistemas Ciberfísicos

La evolución en la tecnología ha modificado la forma de trabajar en muchos y distintos ámbitos, primero partiendo con los sistemas embebidos que consisten en sistemas de computación diseñados para realizar una o algunas funciones dedicadas [4], después se integró la noción de sistemas ciberfísicos(CPS por sus siglas en inglés) que se caracterizan por ser la evolución de los sistemas embebidos, no solo realizando algunas funciones específicas dentro de algún otro sistema mayor, sino siendo capaz de realizar intervenciones a mayor escala por medio de la intercomunicación con otros dispositivos en la misma red, donde se pueden compartir recursos e información para la toma de decisiones, figura 2.1.



Imagen 2.1: Diagrama de composición de Sistemas Ciberfísicos.

“Un sistema ciberfísico consiste en una colección de dispositivos computacionales que se comunican unos con otros e interactúan con el mundo físico por medio de sensores y actuadores en un lazo de control” [5]. Un ejemplo de un sistema ciberfísico es la figura 2.2, donde se muestran 3 dispositivos de cómputo: dos microcontroladores(μC) y un microprocesador(μP), conectados entre sí a través de un protocolo de comunicación, lo que permite compartir datos y recursos, donde el microprocesador puede modificar su propio funcionamiento o el de los microcontroladores para realizar una colecta de datos o actuar en consecuencia de los datos recabados.

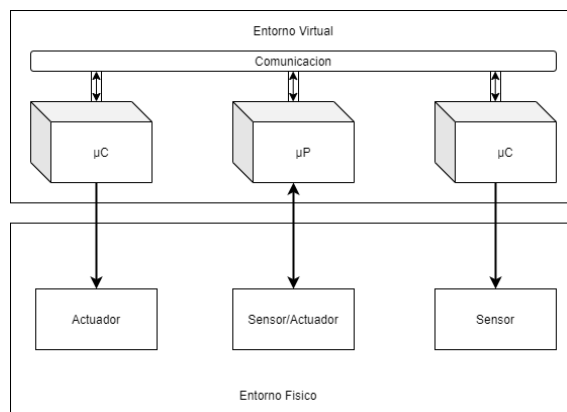


Imagen 2.2: Diagrama Ejemplo de Constitución de Sistema Ciberfísico.

La teoría de los sistemas ciberfísicos es el origen de la teoría de los sistemas IoT, por esto es de vital importancia tomar en cuenta esta información para la investigación. Se considera que todos los sistemas IoT son un sistema ciberfísicos, pero no todos los sistemas ciberfísicos son un sistema IoT.

2.2 IoT

Una evolución de los sistemas ciberfísicos son los sistemas IoT (Internet of Things o IoT por sus siglas en inglés), por tener todas las capacidades de un sistema ciberfísico además de contar con un identificador único, conexión a Internet, etc. Con la interconexión de dispositivos a internet se busca una fácil administración y control de los mismos, además de la obtención de datos con los que se pueda realizar un análisis para poder tener un entendimiento del entorno dentro del sistema y habilitar la ejecución de tareas en casos específicos.

Internet de las cosas o IoT es un término propuesto por primera vez en el año 1999 por Kevin Ashton en el MIT (Massachusetts Institute of Technology), una definición ampliamente aceptada de IoT fue propuesta por ISO/IEC en el 2014, *“IoT es una infraestructura de objetos, personas, sistemas y recursos de información interconectados entre sí que cuentan con servicios inteligentes para habilitar el procesamiento de la información del mundo físico y virtual”* [6]. Otra definición que es propuesta por Bahga y Madiseti es: *“Una infraestructura global dinámica de red que cuenta con capacidades de configuración autónoma basada en protocolos de comunicación interoperables en el que las cosas físicas y virtuales tienen identidades, atributos físicos y una personalidad virtual; y además se encuentran integrados en la red de información”* [7].

Un diseño lógico para sistemas IoT propuesto es el mostrado en la figura 2.3, donde se muestran seis bloques funcionales y sus interacciones, orientando en la parte superior los bloques relacionados con la interacción directa con el usuario y elementos de software, mientras que, en la parte inferior se orienta en la conexión de red.

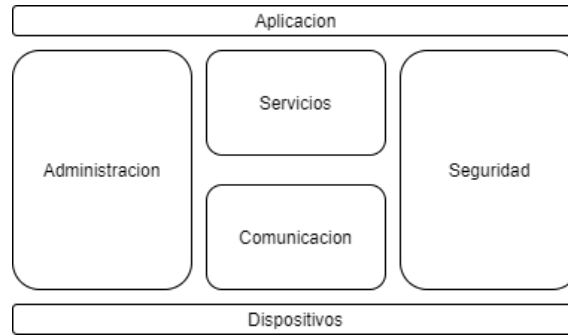


Imagen 2.3: Diseño lógico de un sistema IoT [7].

En la tabla 2.1 , se describen las funciones específicas de cada bloque funcional.

Bloque funcional	Descripción
Dispositivo	Un sistema IoT comprende dispositivos que proveen funciones sensoriales, actuadores, monitoreo y control.
Comunicación	Los bloques de comunicación se encargan de la comunicación de los sistemas IoT.
Servicios	Un sistema IoT utiliza varios tipos de servicios IoT tales como monitoreo de dispositivo, control de dispositivo, publicación de datos y descubrimiento de dispositivos.
Administración	El bloque funcional de administración provee varias funciones para gobernar el sistema IoT.
Seguridad	El bloque funcional de seguridad asegura el sistema IoT proveyendo funciones como autenticación, autorización e integridad del mensaje y del contenido.
Aplicación	Las aplicaciones IoT proveen una interfaz que los usuarios pueden usar para controlar y monitorear varios aspectos de un sistema IoT.

Tabla 2.1: Bloques funcionales de un sistema IoT [7].

2.3 Sistemas Multi-Agente

Los sistemas multiagente son propuestos como una posible solución para resolver problemas que son difíciles o imposibles de resolver para un agente individual o un sistema monolítico. Todo nace de la noción de una comunidad de agentes inteligentes, que se definen como una entidad capaz de percibir su entorno, interactuar y responder de manera racional. A través de distintos mecanismos, dichos agentes inmersos en un entorno trabajan de manera cooperativa para lograr sus metas/objetivos. En el libro “An Introduction to MultiAgent Systems” [8] se describe la interacción de un agente con el ambiente, interacción que a grandes rasgos se define como la adquisición de conocimiento del ambiente, procesamiento de la información por parte del agente y posterior intervención en el ambiente a manera de acciones o respuestas a tales estímulos (figura 2.4), el ambiente donde se desenvuelve

el agente está constituido por un entorno virtual (sistema) y puede también tener una parte física.

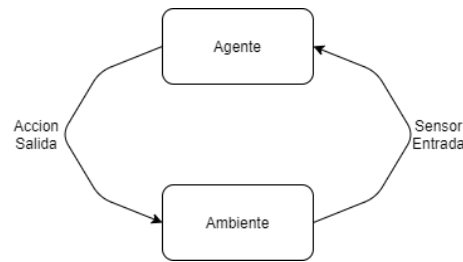


Imagen 2.4: Interacción Agente-Ambiente.

Algunas características de los agentes determinan su forma de interactuar con el ambiente o con los demás agentes inmersos en él, esto es descrito en [8]:

- Agentes reactivos.
 - Responde a cambios en el ambiente o a llamadas directas.
- Agentes proactivos
 - toma la iniciativa para lograr sus objetivos o metas.
- Agentes sociales
 - Tiene la capacidad de interactuar con otros agentes y trabajar en equipo de ser necesario para lograr sus objetivos.

El principal beneficio de los sistemas multi-agente es la descentralización de procesos y tareas. Aunque dependan de un mínimo de agentes para operar de manera correcta, en la mayoría de los casos, el fallo de un agente no desencadenará un fallo total del sistema o de un área del mismo. Estos sistemas tienen la característica de ser asíncronos, por lo tanto basan su comunicación por el paso de mensajes. Los mecanismos usados para la interacción entre agentes son descritos en “Multiagent Systems” [9]. Así mismo es importante mencionar que el procesamiento de datos de cada agente es llevado de forma independiente solo comunicándose con otros agentes para cumplir sus metas. La relevancia de este tema se ve expuesta más adelante en el punto de cómputo autónomo donde se menciona que un sistema de cómputo autónomo puede ser visto como un sistema multi-agente.

2.4 Middleware

El término middleware fue acuñado en 1968 en la conferencia de ingeniería de software de la OTAN [10], donde se definió como una idea para conectar software nuevo con sistemas anteriores o antiguos, una definición más actual describe el middleware como “*el software que ayuda a aplicaciones a interactuar o comunicarse con otras aplicaciones, redes, hardware o sistemas operativos*” [11]. En la actualidad existen diversas clasificaciones para el software middleware, aunque puede ser catalogado en dos grandes categorías (integración y aplicación) figura 2.5.

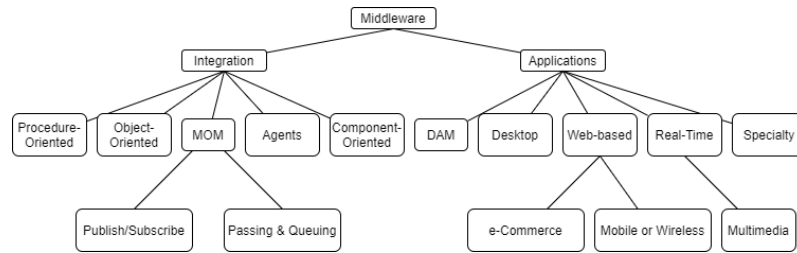


Imagen 2.5: Taxonomía de los programas middleware [11].

Dentro de la rama de integración se cuenta con los enfoques: Orientado a Objetos, Orientado a Procedimientos, Orientado a Mensajes, Orientado a Componentes y Orientado a Agentes, de los cuales el enfoque orientado a mensajes (paso de mensajes/encolamiento) será el utilizado en este trabajo, ya que es necesaria la comunicación asíncrona, entre los agentes (Elementos Autónomos) que lo conformaran.

2.5 Computo Autónomico

La importancia del cómputo autónómico fue expuesta por Paul Horn en el artículo “Autonomic Computing: IBM’s Perspective on the State of Information Technology” en 2001 [12], donde se menciona que un sistema autónómico debe “*conocer de sí mismo*” y comprender sus componentes para ser un sistema de cómputo autónómico (ACS), también poseen una identidad en el sistema, así mismo se hace mención del enfoque holístico del sistema y las cinco claves elementales o tareas del sistema para ser considerado autónómico, tabla 2.2.

Saber de sí mismo.	Necesita conocer sus componentes, estado actual, capacidades y sus conexiones.
Configurarse y reconfigurarse.	Configurarse de manera automática para ajustarse a un entorno cambiante.
Optimizarse.	Monitorear sus flujos de trabajo para lograr aprender y analizar sus funciones y proponer mejoras en el sistema.
Recuperarse de una mala funcionalidad.	Si el sistema descubre fallas o potenciales problemas entonces deberá reconfigurar el sistema o partes de este para seguir trabajando.
Auto protegerse.	Deberá detectar, identificar y protegerse a sí mismo de los ataques que lo podrían poner en riesgo.

Tabla 2.2: Cinco tareas elementales del sistema para ser considerado autónómico de Paul Horn [12].

Luego durante el 2003 Jeffrey O. Kephart en el artículo “The Vision of Autonomic Computing” [13], propone una serie de mejoras y redefiniciones a las propiedades autonómicas y presupone que el sistema “sabrá de sí mismo”. De esta forma se presenta que un sistema autonómico es aquel que se autogestiona, y para lograr dicha autogestión se necesitan cuatro propiedades básicas, tabla 2.3.

Autoconfiguración (Self-configuration).	Configuración automatizada de componentes y sistemas con políticas de alto nivel.
Autooptimización (Self-optimization).	Componentes y sistemas buscan continuamente oportunidades para mejorar el desempeño y eficiencia.
Autosanación (Self-healing).	El sistema automáticamente detecta, diagnostica y repara problemas localizados de software y hardware.
Autoprotección (Self-protection).	El sistema automáticamente se defiende de agentes maliciosos, ataques o fallas en cascada, también usa advertencias para prevenir futuras fallas.

Tabla 2.3: Cuatro propiedades básicas de Jeffrey O. Kephart [13].

Además, se propone el bucle de control MAPE-K (Monitoring, Analyzing, Planning, Execution and Knowledge), que es una representación conceptual de los componentes básicos de un administrador autonómicos, el administrador autonómico logra la gestión del componente o sistema a través de la monitorización de los componentes administrados y otros administradores autonómicos, analizando los datos obtenidos, planeando tareas y posteriormente ejecutando dichos planes, todo sustentado en el conocimiento, figura 2.6. Por otra parte Kephart establece una tentativa definición para computo autonómico “sistemas autogobernados, que se conforman de componentes autogobernados” [13].

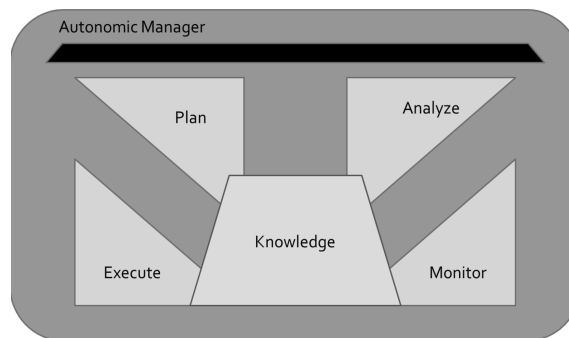


Imagen 2.6: Bucle de control MAPE-K [13].

En “A Survey of Autonomic Computing Systems” [14], se describe cómo un sistema de cómputo autonómico (ACS) está compuesto por elementos autonómicos (AEs) que en sí mismos cuentan con las propiedades autonómicas propuestas por Jeffrey O. Kephart, y pueden ser considerados agentes de software y los ACSs como sistemas multi-agente.

2.6 Diseño Arquitectónico

A grandes rasgos una arquitectura de sistemas es una representación de un sistema en la que se caracterizan sus funciones. Dentro del campo de dominio de sistemas de software podemos encontrar definiciones mas completas como la propuesta en el libro “Software architecture in practice”: *“Una arquitectura de un sistema de software es el conjunto de estructuras necesarias de un sistema, que comprenden elementos de software, la relación entre ellos y sus propiedades”* [15].

La propia arquitectura de sistemas de software es la base para la estructuración y planeación de un sistema, y permite al conjunto de desarrolladores, analistas y programadores compartir una misma línea de visión del trabajo tratando de no generar ambigüedades en sus diferentes vistas. Esto se ve reflejado en el modelo de vistas 4+1 de Kruchten [16], modelo basado en el uso de múltiples puntos de vista, figura 2.7:

- Vista lógica.
 - Esta vista muestra la funcionalidad del sistema para los usuarios finales, muestra las tareas que es capaz de ejecutar el sistema, sus funciones y los servicios que ofrece.
- Vista de procesos.
 - Esta vista se especializa en mostrar los procesos de un sistema, parte del punto de vista de un integrador de sistemas y muestra el flujo de trabajo paso a paso y operaciones de los componentes del sistema, modelando las secuencias de los procesos.
- Vista de despliegue.
 - Esta vista esta enfocada a mostrar el sistema desde la perspectiva que debe tener un programador y se ocupa de la gestión de software, muestra a detalle los componentes del sistema y sus dependencias.
- Vista física.
 - Esta vista presenta una perspectiva desde el punto de vista de un ingeniero de sistemas y muestra todos los componentes físicos necesarios para el sistema y sus conexiones también físicas.
- Vista de escenarios (+1).
 - Esta vista ilustra los casos de uso de software y tiene por objetivo unir las otras cuatro vistas mapeando componentes, clases, equipos, paquetes, etc., también sirve para validar lo descrito en las otras 4 vistas.

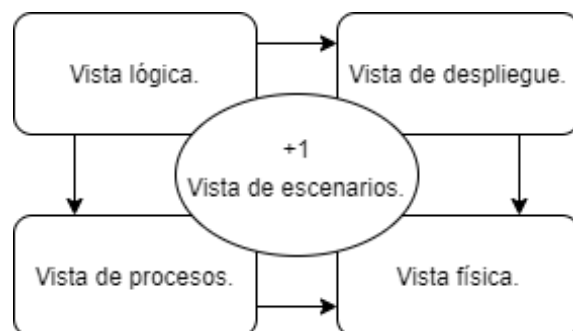


Imagen 2.7: Modelo de vistas 4+1 de Kruchten [16].

Es importante mencionar que un patrón arquitectónico comunica la estructura general de un sistema y define los componentes que debe de tener, mientras que la arquitectura de aplicación es el resultado final del diseño de un sistema que muestra y documenta las funciones del sistema, sus módulos, sus conexiones, etc.

2.7 Metodología de diseño

Las metodologías involucran principalmente procesos, modelos de procesos y métodos. Un proceso es el conjunto de actividades o tareas que toman algún tiempo y buscan un resultado [17]. Un modelo de proceso muestra una abstracción de un proceso que se usa para entender, planear y predecir el comportamiento de dicho procesos [18]. Por su parte el método es definido por varios autores como un procedimiento concreto que se usa, de acuerdo con un objetivo y fin, para proporcionar resultados coherentes.

En el libro “Introduction to the Process of Research” [19], se define la metodología de investigación como el estudio de los métodos, esto involucra la observación de los principios y procesos para alcanzar el conocimiento. También se menciona que un método es la forma en que se realiza algo una vez que la metodología ha resuelto el cómo y por qué.

Así mismo el método científico se define como una metodología para obtener nuevos conocimientos, que ha caracterizado históricamente a la ciencia, y que consiste en la observación sistemática, medición, experimentación, y la formulación, análisis y modificación de hipótesis, según el diccionario de Oxford.

2.8 Arquitectura Orientada a Servicios (SOA)

La arquitectura orientada a servicios (por sus siglas en inglés SOA) es un estilo arquitectónico de tecnologías de la información (TI) que está basada en la modularidad de servicios, parte del pensamiento de un servicio, su construcción y sus resultados. Un servicio representa una tarea específica que se debe realizar en un sistema [20]. Por esto mismo es que al contemplar todas las tareas de un sistema como servicios, se obtiene una arquitectura modular que permite la fácil actualización, acoplamiento y

desacoplamiento de elementos, así como aprovechando la transparencia del sistema y la independencia de sus protocolos [20].

En la arquitectura orientada a servicios, los servicios se asignan a las funciones comerciales que se identifican durante el análisis de procesos. Los servicios pueden ser finos o de grano grueso, dependiendo de los procesos comerciales. Cada servicio tiene una interfaz bien definida que le permite ser publicado, descubierto e invocado. Una empresa puede optar por publicar sus servicios externamente a socios comerciales o internamente dentro de la organización. Un servicio también puede estar compuesto de otros servicios. Otra definición ampliamente usada es presentada en el libro “Service-oriented architecture compass” [21], *“SOA es una framework que integra procesos de negocio y TIC de forma débilmente acoplada y segura, con componentes estandarizados como servicios que pueden ser reusados y combinados para administrar las prioridades del negocio”*. SOA es propuesta como una forma de desarrollar sistemas distribuidos donde los componentes del sistema son independientes y pueden ejecutarse en computadoras geográficamente distribuidas, en el libro “Software Architecture” [22]. Algunos puntos que caracterizan la arquitectura SOA son descritos a continuación [23]:

- Estar basado en el diseño de servicios que reflejan las actividades del negocio en el mundo real, estas actividades forman parte de los procesos de negocio de la compañía.
- Representar los servicios utilizando descripciones de negocio para asignarles un contexto de negocio.
- Tener requerimientos de infraestructura específicos y únicos para este tipo de arquitectura, en general se recomienda el uso de estándares abiertos para la interoperabilidad y transparencia en la ubicación de servicios.
- Estar implementada de acuerdo con las condiciones específicas de la arquitectura de TI en cada compañía.
- Requerir un gobierno fuerte sobre la representación e implementación de servicios.
- Requerir un conjunto de pruebas que determinen que es un buen servicio.

Estos conceptos son especialmente importantes ya que se usará la abstracción de modularidad de servicios para el diseño de las tareas que pueden realizar los elementos autónomos en la propuesta.

2.9 Arquitectura Orientada a Microservicios (MSA)

Una arquitectura de microservicios (por sus siglas en inglés MSA) consta de una colección de servicios autónomos y pequeños. Los servicios son independientes entre sí y cada uno debe implementar una funcionalidad de negocio individual [24]. Cada servicio se encarga de implementar una funcionalidad completa del negocio, cada servicio es desplegado de forma independiente y puede estar programado en distintos lenguajes y usar diferentes tecnologías de almacenamiento de datos, por

su modularidad fina un equipo puede actualizar un servicio existente sin tener que volver a generar e implementar toda la aplicación. Se suele considerar la arquitectura de microservicios como una forma específica de realizar una arquitectura orientada a servicios [25]. En la tabla 2.4 se compara la arquitectura orientada a servicios (SOA) con la arquitectura de microservicios (MSA).

Características	MSA	SOA
Compartición de componentes	Minimiza el uso de compartido de componentes a través de un contexto limitado.	Extiende todo lo posible el uso compartido de componentes lo que puede aumentar la latencia.
Granularidad del Servicio	Servicios muy especializados o de uso único.	Servicios versátiles de funcionalidad empresarial.
Coordinación	Coordinación mínima al ser diseñados de manera independiente.	Grande dependencia de coordinación en grupos de servicios para atender solicitudes.
Middleware	Trabajan con una capa de API creada entre los servicios y los consumidores de servicios.	Mediación y enrutamiento, mejora de mensajes, mensajes y transformación de protocolos.
Interoperabilidad heterogénea	Es preferible recurrir a microservicios cuando todos los servicios puedan quedar expuestos y se deba usar el mismo protocolo de acceso remoto, ya que MSA intenta simplificar el patrón de arquitectura al reducir el número de opciones de integración.	SOA promueve la propagación de múltiples protocolos heterogéneos a través de su componente middleware de mensajería, por eso, esta opción debe tenerse en cuenta en los casos en que el objetivo sea lograr la integración de varios sistemas utilizando diferentes protocolos en un entorno heterogéneo

Tabla 2.4: Comparativa MSA-SOA.

2.10 Red definida por Software (SDN)

Las redes definidas por software surgen como un desarrollo que da solución a la necesidad de automatizar, escalar y optimizar las redes [26], su objetivo es facilitar la implementación e implantación de servicios de red, evitando o minimizando las intervenciones a bajo nivel necesarias para cualquier tipo de configuración por parte de un administrador de red, haciendo uso de dispositivos genéricos que pueden cumplir con una gran cantidad funciones de los distintos dispositivos de red que existen actualmente, según la programación que tengan.

Las redes definidas por software están enfocadas en cuatro claves [27]:

- Separar el plano de control del plano de datos.
- Un control centralizado y una vista de la red.
- Abrir interfaces entre los dispositivos en el plano de control y los que están en el plano de datos.
- Programabilidad de la red por aplicaciones externas.

La separación del plano de datos y el plano de control es un punto muy importante, ya que este concepto se puede extrapolar al ámbito de los sistemas IoT que realizan intervenciones en sí mismos. Un sistema IoT Autoensamblable y Autorreplicable se puede ver beneficiado de esta separación, independizando sus procesos de gestión y otras tareas de la funcionalidad original para la que fue diseñado el sistema.

2.11 Autorreplicación

La autorreplicación se puede definir como el proceso a través del que cualquier cosa puede realizar copias de sí mismo. Un ejemplo de esto es la replicación celular, que se produce con la división de una célula. Durante este proceso el ADN se replica y puede transmitirse a la descendencia durante la reproducción, dividiendo las cadenas de ADN a través de burbujas de replicación y uniendo a la estructura los pares complementarios de las bases nitrogenadas, figura 2.8, que "de no existir errores en el proceso" implicará una reproducción fiel de la cadena original de ADN [28] figura 2.9. Por su parte, los virus informáticos son programas que se reproducen y replican usando el software o hardware del dispositivo donde se hospeda. Asimismo existe una clasificación de programa informático que cuenta con la característica de autorreplicación, que no es considerado malware, denominado Quines.

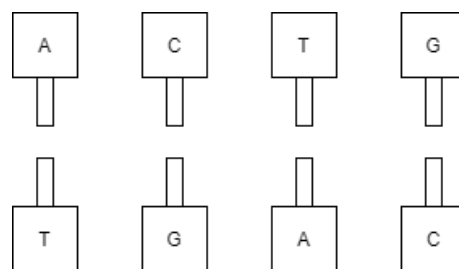


Imagen 2.8: Pares complementarios de algunas bases nitrogenadas.

A finales de la década de 1940 [29] John von Neumann establece que las formas comunes de replicador tienen 3 partes esenciales:

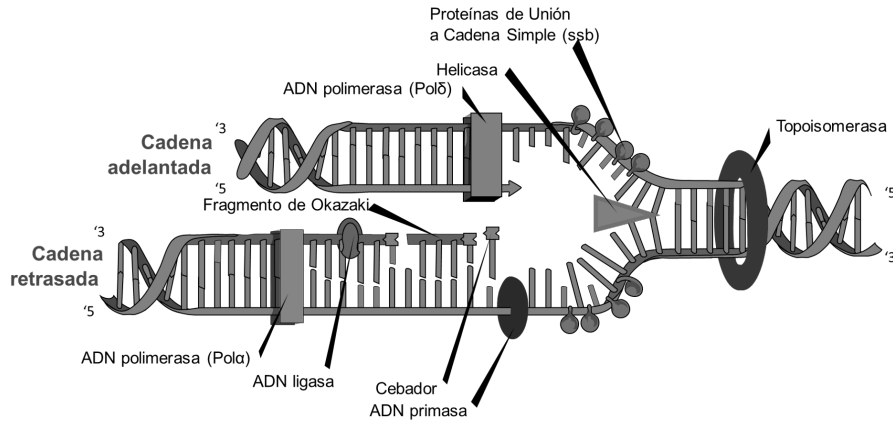


Imagen 2.9: Proceso de replicación del ADN.

- Un genoma, corresponde a una carga de información resistente a errores como el ADN.
- Un especializado conjunto de mecanismos para copiar y reparar el genoma utilizando los recursos disponibles.
- Un organismo, que reúne los recursos y energía necesarios.

Los autómatas celulares (AC) están basados en la teoría de replicación biológica ya que cumplen con un funcionamiento muy similar en su replicación al ADN, buscando materias primas, cómo los nucleótidos y se replican procesándolas, asimismo "durante la reproducción" se busca una evolución dando ciertos grados de libertad al diseño para que varíe de tal manera que los elementos que mejor consigan los recursos para reproducirse lo logren como si de selección natural se tratase.

Esto se sustenta en el libro "Theory of Self-reproducing Automata" [30], que introduce la teoría de los autómatas celulares (AC), donde se modela una máquina que sea capaz de autorreplicarse, y obteniendo por resultado un modelo matemático basado en reglas y una red rectangular, donde las células siguen las reglas estipuladas de replicación, crecimiento y muerte. Los elementos de un autómata celular son los siguientes [31]:

- Un espacio regular, ya sea una línea, un plano de 2 dimensiones o un espacio n-dimensional.
- Conjunto de Estados, es finito y cada elemento o célula del espacio toma un valor de este conjunto de estados. También se denomina alfabeto. Puede ser expresado en valores o colores.
- Configuración Inicial. Es la asignación inicial de un estado a cada una de las células del espacio.
- Vecindades. Define el conjunto de células que se consideran adyacentes a una dada, así como la posición relativa respecto a ella.
- Función de Transición Local, esta es la regla de evolución que determina el comportamiento del AC. Se calcula a partir del estado de la célula y su vecindad. Define cómo debe cambiar de estado cada célula dependiendo su estado

anterior y de los estados anteriores de su vecindad. Suele darse como una expresión algebraica o un grupo de ecuaciones.

Se toma inspiración biológica para el diseño de sistemas IoT Autorreplicables y Autoensamblables, ya que en estos sistemas se encuentran las propiedades que se buscan agregar en los sistemas IoT antes mencionados.

2.12 Quines

En el ámbito de la programación un Quine es un programa del tipo de metaprogramación que produce su propio código fuente como salida única, el termino quine fue acuñado por Douglas Hofstadter, en su libro “Gödel, Escher, Bach: un Eterno y Grácil Bucle” [32], este tipo de programas se pueden desarrollar en cualquier lenguaje que sea Turing completo y que sea capaz de generar cualquier cadena.

Por su parte la metaprogramación se basa en la tarea de generar programas que tengan la capacidad de escribir o manipular otros programas incluidos ellos mismos [33], y tratarlos como datos. Por esto mismo la línea que separa la metaprogramación del código dinámico, es muy difusa.

2.13 Blockchain

Las cadenas de bloques son estructuras de datos muy usadas actualmente, donde información se agrupa en bloques y es firmada, los bloques compiten de ser necesario con otros bloques para pasar a ser parte de la cadena mas grande (consolidarse), dejando fuera a los bloques que causaron ramificaciones que no se convirtieron en la cadena más grande, figura 2.10. Por lo tanto, cada bloque tiene la información de los bloques anteriores [34], gracias a esto la información de un bloque solo puede ser repudiada o editada modificando los bloques posteriores a este, de manera que el bloque que se desee editar forme parte de una ramificación de la cadena que sea mas grande que la original y provoque ser adoptada como la cadena principal, esto es especialmente útil en sistemas distribuidos por su naturaleza descentralizada y de autovalidación.

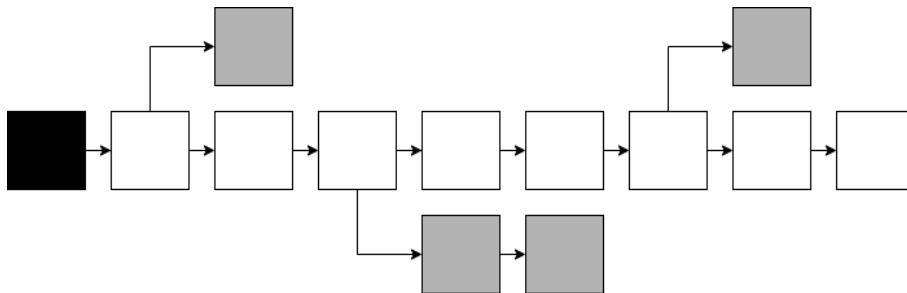


Imagen 2.10: Representación de una cadena de bloques.

Los bloques utilizados para la cadena se rigen con políticas muy específicas, que permiten la validación de los bloques, estas políticas abarcan desde la validación de las estructuras de información dentro de los bloques hasta la integridad misma de los bloques, para esto entre otras cosas se usan técnicas de firmado digital [35], de manera que cuando un bloque quiera unirse a la cadena primero será analizado y se verificará que cumpla con las políticas y seguridad requerida por parte del sistema, de esta manera el bloque será aceptado o repudiado y el sistema estará a la espera de un nuevo bloque que cumpla con lo solicitado.

El consenso a través de esta estructura de datos se da con las replicaciones de la cadena, si existen varias cadenas prevalece la cadena mas grande, en caso de existir mas de una cadena con la misma longitud la cadena que prevalecerá será la que logre crecer antes que las otras [34]. Por ejemplo, de presentarse un escenario donde todo el sistema menos un elemento tiene una cadena de longitud n y el elemento faltante tiene también una cadena de longitud n pero son cadenas diferentes, si en la cadena que tiene el elemento único agrega un bloque antes que las demás, esta pasara a replicarse en todo el sistema aboliendo las demás cadenas, aunque idealmente solo se estaría repudiando el último bloque de la cadena que no coincide con la nueva cadena más grande.

2.14 Resumen Capitulo 2

Los sistemas IoT cuentan con un sinnúmero de campos de aplicación actualmente, de la misma manera aun existen muchos campos inexplorados para la automatización de procesos. El lograr proponer una arquitectura IoT que adicionalmente a las propiedades autonómicas definidas cuente con las propiedades de autoensamblaje y autorreplicación es posible si tomamos como punto de partida los sistemas naturales que ya cumplen con esta tarea, como la replicación del ADN. Esto pensándolo en una replicación que se presenta de forma secuencial, realizando tareas específicas, como complementar la cadena separada de ADN con su par complementario, esto nos da un indicio de una replicación basada en módulos.

Luego "tomando el enfoque de una arquitectura basada en servicios" podemos pensar en una representación modular de un sistema que con la interacción de los módulos antes mencionados pueda cumplir con las tareas especificadas, además, si contemplamos los dispositivos IoT como agentes como se propone en el computo autonómico, podemos pensar en un entorno de sistemas multi-agente donde cada agente este programado de manera modular, y además de cumplir su funcionalidad original como sería esperado en cualquier sistema IoT, contara con un plano de control separado del plano de datos (funcionalidad base) como una SDN.

Es decir, un agente que realiza alguna tarea en el sistema IoT y que además se encarga de tareas autonómicas, como la autogestión, pudiendo analizar código fuente para realizar intervenciones en él, con un enfoque de metaprogramación. Dicho código fuente deberá ser modular y estar estandarizado para que cada agente pueda analizarlo y realizar intervenciones en el mismo, además de aportar información pre-

cisa al sistema de sus propias necesidades básicas de operación; por ejemplo, qué características debe tener un dispositivo IoT para realizar alguna tarea específica, esto analizando los bloques de código. Tal tarea sería especialmente útil entre sistemas que puedan compartir intersecciones en campos de dominio, dicho de otra forma, si uno o mas sistemas contaran con este tipo de arquitecturas modulares Autorreplicables y Autoensamblables, se podría pensar en la generación automática de una arquitectura para un nuevo sistema IoT, esto con mínima intervención humana.

Capítulo 3

Estado del Arte

Las arquitecturas IoT son un campo en constante expansión debido a los múltiples campos de dominio en los que los sistemas IoT pueden intervenir, abarcando áreas como la domótica, la agricultura urbana, o un uso industrial, al ser los sistemas IoT un subdominio específico de los sistemas ciberfísicos existen algunas arquitecturas de propósito general que buscan estandarizar el desarrollo de los sistemas IoT y buscan dotarlos de algunas propiedades específicas, sin embargo las arquitecturas IoT propuestas para el computo autónomo son pocas, y debido a la naturaleza de la propuesta de dos nuevas propiedades autónomas no existe alguna arquitectura IoT que las contemple, sin embargo, existen múltiples sistemas donde podemos ver reflejadas las propiedades propuestas en esta arquitectura, así como algunos otros conceptos de funcionamiento que nos ayudaran a sustentar la propuesta, por esto se abordará el estado del arte de las arquitecturas IoT con propiedades autónomas, las propuestas de auto arquitectura (self-architecture), sistemas con autorreplicación y autoensamblaje.

3.1 Arquitecturas relacionadas con sistemas IoT

Villela [36] propone una evolución de la arquitectura de nivel cinco de Bahga y Madiseti [7], en la que adiciona la propiedad de autoconfiguración propuesta por Kephart [13], proponiendo ediciones a los nodos de base de conocimiento, análisis y planeación, y nodo observador, servicios REST, y a su vez añadiendo un nodo de autoadaptación de tareas y una base de objetivos, tomando inspiración en el bucle de control MAPE-K [13], figura 3.1.

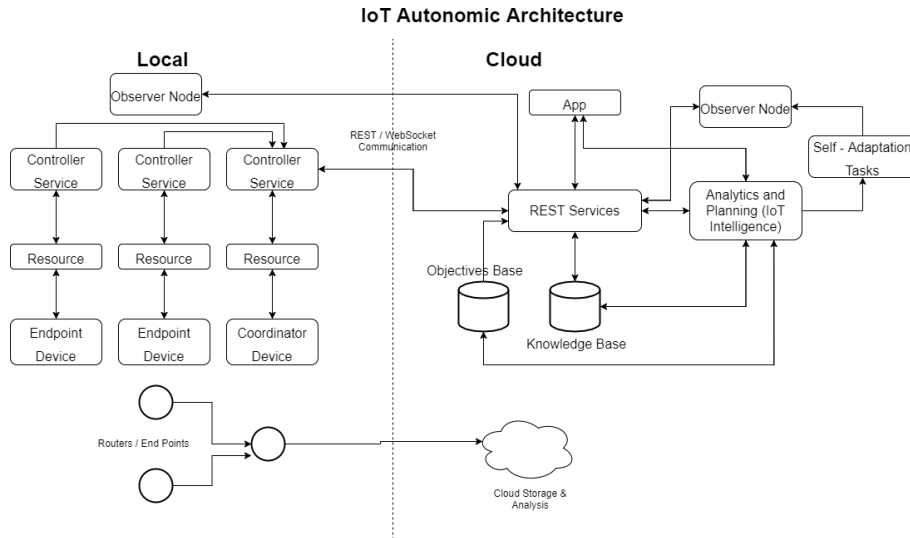


Imagen 3.1: Arquitectura IoT Autnómica [36].

Otra propuesta importante a considerar es la propuesta de Ruiz [37] donde se aborda la posibilidad de tratar los sistemas IoT como un sistema multi-agente, y se realiza una paralelización de las capas de datos y capas de control de infraestructura como si de una red definida por software se tratase, por su naturaleza de arquitectura orientada a servicios esta arquitecturas se especializa en la escalabilidad y heterogeneidad de dispositivos y protocolos, agregando a su vez una capa de aplicación para habilitar la interacción del usuario final, figura 3.2.

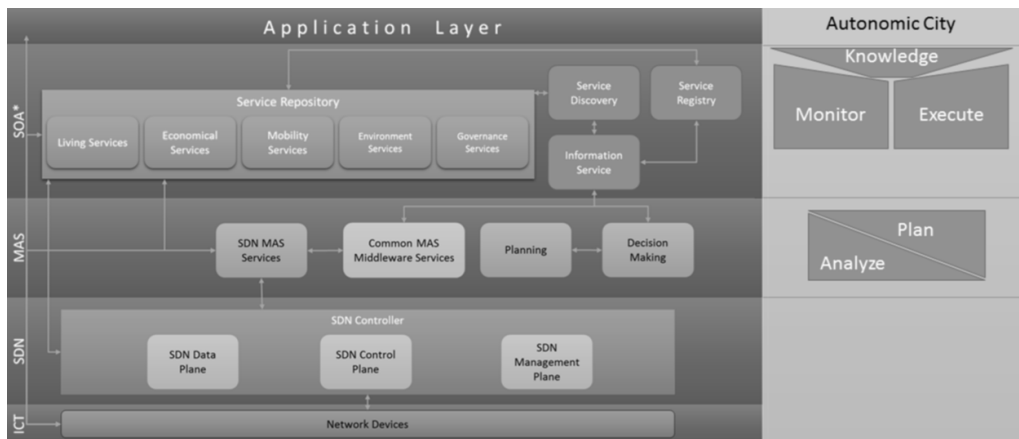


Imagen 3.2: Arquitectura de Referencia para Sistemas IoT en Ciudades Inteligentes (SCRAIoT) [37].

Así mismo Manate [38] propone una arquitectura basada cien por ciento en la teoría de los sistemas multi-agente, dotando a agentes individuales de la capacidad de censado y procesamiento de grandes volúmenes de datos, y manejando sus configuraciones con altas políticas de control.

Movahedi [39] propone una arquitectura que sustentada en su base de conocimientos y base de objetivos realiza la tarea de gestión en una red de sensores (WSN), la

arquitectura esta dividida en 4 grandes bloques, plan de gestión, plan de datos, plan de control y plan de conocimiento, siendo este ultimo el encargado de transmitir los datos del sistema y los objetivos del mismo para la toma de decisiones que habilita algunas características de computo autónomico.

arquitecturas	Villela [36]	Ruiz [37]	Manate [38]	Movahedi [39]
Escalabilidad	X	X	X	X
Heterogeneidad	X	X	X	X
Distribuido		X	X	
Interoperable	X	X	X	X
CAC	X	X		
Basado en AE		X	X	
WSN	X	X		X
Base de objetivos	X			X
Base de conocimiento	X			X
Autoconfiguración	X	X		X
Autosanación	X			
Autooptimización	X			
Autoprotección	X			

Tabla 3.1: Comparativa de arquitecturas IoT con enfoques autónomicos.

En la tabla 3.1 se muestran los parámetros más relevantes para este trabajo en el diseño de arquitecturas IoT, iniciando la capacidad de escalabilidad del sistema, su heterogeneidad y las propiedades autónomicas. Además, tomando en cuenta los parámetros que impactan en sistemas IoT y sistemas multi-agente como la interoperabilidad, el computo distribuido, y la definición de elementos autónomicos como dispositivos IoT.

3.2 Autoarquitectura

La auto ingeniería o ingeniería propia se puede definir como un sistema capaz de registrar y responder a una perdida en la funcionalidad o capacidad de operación y reaccionar a dicho suceso con medidas para recuperar la funcionalidad perdida [40], el mismo autor que propone esta definición propone un método basado en la respuesta biológica de reparación de un hueso fracturado, para esto estipula un medio de monitoreo, un evento disparador y la respuesta ante tal evento, de manera que se pueda subsanar, este trabajo esta orientado a la auto reparación de sistemas robóticos en su capa física.

En el libro “Self Engineering: Learning from Failures” [41] se define la auto ingeniería como aquello que habla de lo que se puede realizar por mí mismo (traducción del inglés “Do it myself”) y se refiere a las habilidades que tiene cada ente por sí mismo, además expone los beneficios de la modularización para los sistemas de ensamblaje

o de producción, esto es especialmente importante ya que podemos contemplar el código fuente como un elemento que es ensamblable.

Hee Lee [42] propone que la auto-ingeniería consiste en ocho módulos: modulo de reglas de rendimiento, módulo de adquisición de datos, desempeño de base de datos móvil, modulo de configuración de parámetros de optimalidad, modulo de motor de inferencia, modulo de interfaz del operador, modulo de diagnostico y modulo de aprendizaje, el modelo propuesto esta basado principalmente en los datos obtenidos a través del censado y el análisis de los mismos a través del modulo de aprendizaje, y expone la auto Ingeniería como autoconfiguración.

3.3 Sistemas Autorreplicables y Autoensamblables

Para este trabajo se toma como inspiración los sistemas biológicos donde podemos observar las propiedades de autorreplicación y autoensamblaje, específicamente el área donde podemos observar este tipo de comportamientos reside en la replicación del ADN, que al desensamblar la doble hélice con burbujas de replicación logra iniciar este proceso con el ensamblaje automático de su hélice complementaria, tomando las bases nitrogenadas del ambiente y complementando la cadena. En el artículo [43] se habla sobre la utilización del ADN como materia prima para autoensamblaje en el área de nanotecnología, basándose enteramente en el autoensamblaje de las bases nitrogenadas [28] y la intervención química para su modificación, y posterior reestructuración de bloques de construcción.

En el libro “Self-Assembly and Nanotechnology” [44] se presenta un enfoque de autoensamblaje donde a escala microscópica solo se puede hablar de autoensamblaje cuando las piezas a ensamblar solo pueden encajar de una única manera y el proceso completo depende de un buen diseño para ese ensamblaje.

Un enfoque más apegado al área de computación es el presentado en el trabajo del autómata celular de Von Newman [30], donde siguiendo reglas específicas un elemento autómata podía replicarse, mantenerse o morir, definiendo estados en los que el sistema puede realizar estas tareas, es decir, momentos o condiciones apropiadas para la autorreplicación.

Mas recientemente el área donde se puede ver un poco de avance en el tema del autoensamblaje y la autorreplicación es el área de la manufatura, en esta área se estudian estos términos en la rama de la nanotecnología, en [45] se hace mención que para el autoensamblaje se necesitan partes que sólo se puedan ensamblar de una manera, y que el ensamblaje de los virus únicamente es la sustitución de partes que también pueden ensamblar en el sistema. En [46] se trata de probar la replicación a nano escala en una simulación, los principios antes vistos son aplicados y se logra la obtención de patrones esperados como resultado de las simulaciones.

Capítulo 4

Propuesta

En este capítulo se detallará la descripción de la propuesta de solución para el problema identificado en el capítulo 1 y la metodología seguida para su realización.

4.1 Metodología de la investigación

La metodología de investigación seguida para la realización de este trabajo se muestra en la figura 4.1. Esta es una adaptación del método científico usado en investigaciones científicas y académicas [47].

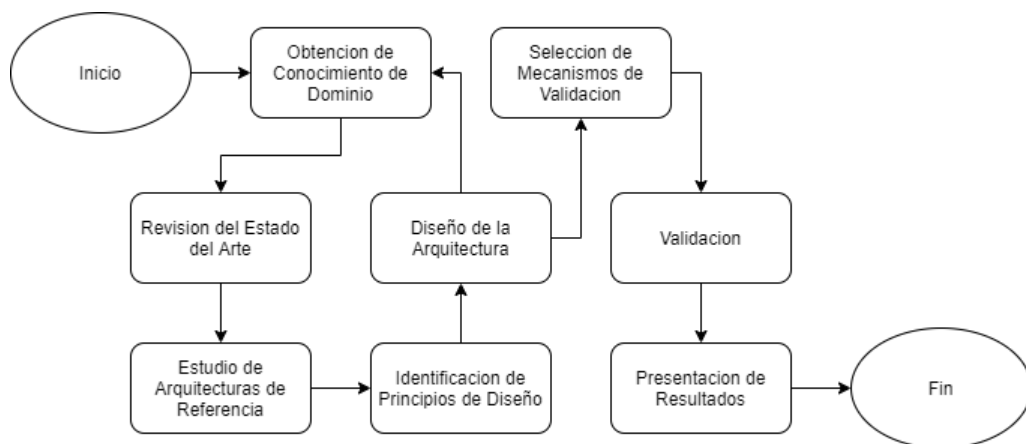


Imagen 4.1: Metodología utilizada para la resolución del problema de investigación.

A continuación, se describen cada uno de los bloques mostrados en la figura 4.1.

4.1.1 Obtención de conocimiento de dominio

Como primer paso es necesario realizar una investigación donde se estudien los temas afines a las arquitecturas IoT y las propiedades de las mismas, Sistemas Multi-agente(MAS) y el modelado de los mismos, y los sistemas Autorreplicables, esto con el fin de tener conocimiento sobre las teorías que fundamentan dichos sistemas.

4.1.2 Revisión del estado del arte

Las propiedades del Autoensamblaje y Autorreplicación han sido estudiadas principalmente en las áreas de la biología, pero debido a su gran complejidad poco abordadas desde sistemas artificiales, donde el enfoque que se sigue es a nivel hardware o más propiamente nivel físico, y dejando de lado totalmente un enfoque de software. Por esto se estudió principalmente las arquitecturas de los sistemas IoT y sus características y computo autónomo, que posteriormente derivó en sistemas multi-agente e información sobre las propiedades de autoensamblaje y autorreplicación, que condujeron al área biológica.

4.1.3 Estudio de arquitecturas de referencia

Se estudió ampliamente las arquitecturas propuestas por Bahga y Madiseti [7] y la arquitectura IoT autónoma propuesta por Villela [36], además de literatura sobre los sistemas Multi-agente.

4.1.4 Identificar principios de diseño

Con base en la investigación realizada durante la revisión del estado del arte se identificaron premisas de diseño necesarias para lograr una evolución arquitectónica con las capacidades de la Autorreplicación y el Autoensamblaje.

4.1.5 Diseño de la arquitectura

El diseño de la arquitectura es un proceso incremental debido a la nueva información obtenida a través de la continua revisión del estado del arte y la meditación durante el avance de la investigación.

4.1.6 Selección del mecanismo de validación

Dodig en [48] y Gamukama en [49], sugieren que en el campo de las ciencias computacionales, el uso de un caso de estudio como mecanismo de validación es conveniente. Por lo tanto el presente trabajo propone el uso de un caso de estudio utilizando una validación estática por medio del análisis de la arquitectura, y una validación dinámica utilizando un entorno de pruebas y una plataforma de simulación de Modelado Basado en Agentes (MBA).

4.1.7 Validación

Se diseñaron varios caso de estudio para realizar una validación por medio de la prueba estática y el entorno de pruebas en el software de simulación GAMA, donde se observan aspectos como la escalabilidad de los sistemas, su ejecución en tiempo real, etc.

4.1.8 Presentación de resultados

Posterior a la realización de los experimentos correspondientes para la validación del caso de estudio, se procedió a reportar todos los hallazgos encontrados en un

documento de tesis.

4.2 Metodología de diseño arquitectónico

4.2.1 Introducción

En esta sección se muestran los aspectos principales utilizados para el diseño de la arquitectura, y sus bases, así como los modelos y métodos empleados para tal diseño, siguiendo la metodología propuesta.

4.2.2 Metodología de diseño arquitectónico

La metodología usada para el diseño de la arquitectura propuesta es un hibridaje entre las metodologías propuestas en el libro “Ingeniería del Software” [22] y “Software Architecture in practice” [15], ambos con la premisa de la obtención de los requerimientos y la resolución de estos. Bass propone un modelo donde predomina la iteración de probar hipótesis y generar siguiente hipótesis, para la resolución de los requerimientos, figura 4.2. Por otra parte, Sommerville propone que las arquitecturas deben ser diseñadas partiendo del tipo de aplicación a desarrollar y derivada de sus requerimientos funcionales y no funcionales, además de ser documentados con distintas vistas. Para esto es que propone la utilización del modelo 4+1 de Kruchten [16], que consiste en la vista lógica, vista de procesos, vista de despliegue, vista física y vista de escenarios.

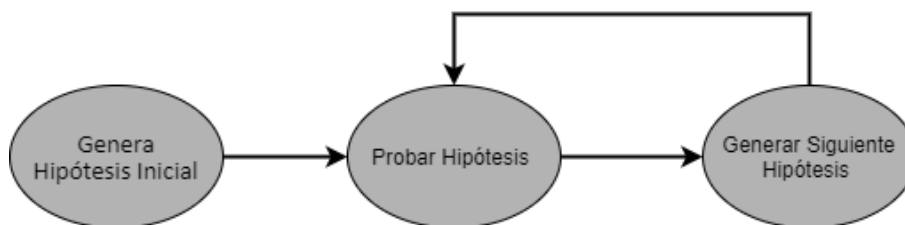


Imagen 4.2: Generación y prueba de diseño arquitectónico [15].

También se muestra en [15], la metodología de desarrollo diseñado por atributos (ADD por sus siglas en inglés) que consta de cinco pasos:

- Seleccionar un elemento del sistema a diseñar.
- Idénticar los Requerimientos Significativos Arquitectónicamente (ASRs por sus siglas en inglés).
- Generar una solución de diseño para el elemento seleccionado.
- Hacer un inventario de los requisitos restantes y seleccionar la entrada para la próxima iteración.
- Repetir los pasos 1-4 hasta que todos los ASRs hayan sido satisfechos.

4.2.3 Primitivas de diseño

Todo sistema debe partir de un diseño arquitectónico que lo defina en todos sus distintos aspectos, para lograr esta definición es necesaria la obtención de las características deseables y necesarias del mismo. Debido a esto, se realizó una investigación para definir el diseño de la arquitectura, identificando las características principales de una arquitectura de un sistema IoT definidas en el libro “Internet of Things: A hands-on approach.” [7] , y debido al enfoque de sistema multi-agente. Las características básicas de dicho tipo de sistemas están identificadas en el artículo [50].

Id	PD1
Nombre	IoT autónómico ->IoT autónómico plus
Descripción	El sistema deberá contar con las propiedades de un sistema autónómico y además ser capaz de Autoensamblarse y Autor-replicarse.
Entrada	Datos
Salida	Arquitectura IoT

Tabla 4.1: Primitiva de diseño 1

Id	PD2
Nombre	Replicación de Código
Descripción	Los AE y el módulo de RestService deberán ser capaces de replicar código de manera selectiva
Entrada	Solicitud de código fuente
Salida	Código fuente

Tabla 4.2: Primitiva de diseño 2

Id	PD3
Nombre	Ensamblaje de Código
Descripción	Los AE deberán ensamblar código funcional a partir de su propio código y nuevo código
Entrada	Código fuente
Salida	Código fuente

Tabla 4.3: Primitiva de diseño 3

4.2.4 Requerimientos funcionales

Los requerimientos funcionales se pueden definir como las tareas esenciales que se desea que pueda realizar la arquitectura sin ahondar en términos de desempeño, por lo cual los requerimientos funcionales tomados en cuenta para la especificación de la arquitectura son los siguientes:

Id	RF1
Nombre	Autoconfiguración
Descripción	El sistema será capaz de cambiar su configuración o la de sus componentes adaptándose a objetivos dinámicos. [36]
Entrada	Contexto.
Salida	Nuevos objetivos.

Tabla 4.4: Requerimiento Funcional 1

Id	RF2
Nombre	Autosanación
Descripción	El sistema y sus componentes serán capaces de reconocer fallas internas, informarlas y tratar de repararlas. [12]
Entrada	Datos de componentes del sistema.
Salida	Datos del estado de los componentes.

Tabla 4.5: Requerimiento Funcional 2

Id	RF3
Nombre	Autooptimización
Descripción	El sistema será capaz de reasignar recursos para la realización de tareas. [12]
Entrada	Contexto
Salida	Asignación de recursos.

Tabla 4.6: Requerimiento Funcional 3

Id	RF4
Nombre	Autoprotección
Descripción	El sistema y sus componentes deberán cubrir todos los aspectos de seguridad en todos los niveles, así como predecir fallos y proteger la integridad del sistema con herramientas como el autorrepudio. [12]
Entrada	Datos de módulos del sistema IoT.
Salida	Datos del estado de los módulos.

Tabla 4.7: Requerimiento Funcional 4

Id	RF5
Nombre	Autonomía
Descripción	El sistema deberá poder operar de manera independiente sin la necesidad de intervención humana. [13]
Entrada	Datos del sistema.
Salida	Tareas

Tabla 4.8: Requerimiento Funcional 5

Id	RF6
Nombre	Proactividad
Descripción	Los elementos autónomos deberán percibir el entorno y actuar en consecuencia. [13]
Entrada	Contexto
Salida	Interacciones.

Tabla 4.9: Requerimiento Funcional 6

Id	RF7
Nombre	Reactividad
Descripción	Los elementos autónomos actuarán en consecuencia cuando sean llamados de manera directa. [13]
Entrada	Solicitud.
Salida	Respuesta.

Tabla 4.10: Requerimiento Funcional 7

Id	RF8
Nombre	Interacción
Descripción	Los elementos autónomos serán capaces de interactuar para generar capacidades emergentes. [50]
Entrada	Comunicación.
Salida	Comunicación.

Tabla 4.11: Requerimiento Funcional 8

Id	RF9
Nombre	Coordinación
Descripción	Los elementos autónomos deberán poder coordinarse para realizar tareas. [50]
Entrada	Comunicación(solicitud).
Salida	Comunicación(coordinación).

Tabla 4.12: Requerimiento Funcional 9

Id	RF10
Nombre	Competición
Descripción	Los elementos autónomos podrán solucionar disputas por recursos o configuraciones. [50]
Entrada	Comunicación(disputa)
Salida	Comunicación (dispositivo ganador)

Tabla 4.13: Requerimiento Funcional 10

Id	RF11
Nombre	Autoensamblaje
Descripción	El sistema y sus elementos autónomos serán capaces de autoensamblar una nueva arquitectura o partes de ella, principalmente en código fuente.[33]
Entrada	Código fuente
Salida	Código fuente

Tabla 4.14: Requerimiento Funcional 11

Id	RF12
Nombre	Autorreplicación
Descripción	El sistema y sus elementos autónomos serán capaces de replicar su propia arquitectura o partes de esta en otro sistema, principalmente en código fuente.[33]
Entrada	Solicitud de replicación.
Salida	Código fuente

Tabla 4.15: Requerimiento Funcional 12

4.2.5 Requerimientos no funcionales

Los requerimientos no funcionales se basan en los aspectos de rendimiento del sistema y otras métricas que debe de seguir, y no definen las tareas principales del sistema o arquitectura sino su rendimiento. Los requerimientos no funcionales tomados en cuenta para la especificación de la arquitectura son los siguientes:

Id	RNF1
Nombre	Seguridad
Descripción	El sistema y sus componentes deberán implementar mecanismos de seguridad. [13]

Tabla 4.16: Requerimiento No Funcional 1

Id	RNF2
Nombre	Confiabilidad
Descripción	Los fallos no deberán implicar que el sistema deje de trabajar y deberán aislarse, así como los tiempos promedio de fallos deberán ser cortos con respecto a la velocidad del sistema. [50]

Tabla 4.17: Requerimiento No Funcional 2

Id	RNF3
Nombre	Modularidad
Descripción	El sistema deberá estar programado de forma modular, de manera que se puedan analizar los distintos bloques de código para su ensamblaje o replicación.

Tabla 4.18: Requerimiento No Funcional 3

Id	RNF4
Nombre	Disponibilidad
Descripción	El sistema deberá contar con una alta disponibilidad, para la entrada o salida de nuevos elementos autónomos. [50]

Tabla 4.19: Requerimiento No Funcional 4

Id	RNF5
Nombre	Estandarización
Descripción	La estructura de programación del sistema deberá ser estandarizada para ayudar a simplificar las tareas de análisis de bloques de código.

Tabla 4.20: Requerimiento No Funcional 5

Id	RNF6
Nombre	Escalabilidad
Descripción	El Sistema deberá ser escalable en dispositivos y funciones [37].

Tabla 4.21: Requerimiento No Funcional 6

4.3 Propuesta de arquitectura

La arquitectura de un sistema IoT documenta su diseño y proyecta los aspectos funcionales y no funcionales del mismo, y aunque existen muchas arquitecturas definidas para los sistemas mencionados, se busca incursionar en un nuevo ámbito: las arquitecturas Auto-Ensamblables y Auto-Replicables, ya que este trabajo está encaminado a dotar de nuevas funcionalidades los sistemas IoT. Resulta importante seleccionar una arquitectura base para aprovechar sus bondades de diseño y extenderlas a un nuevo dominio. Dicha arquitectura base será la propuesta por Villela [36] (figura 4.3), misma que está basada en una evolución de una de las arquitecturas IoT propuestas por Bahga y Madiseti [7] desarrolladas por niveles.

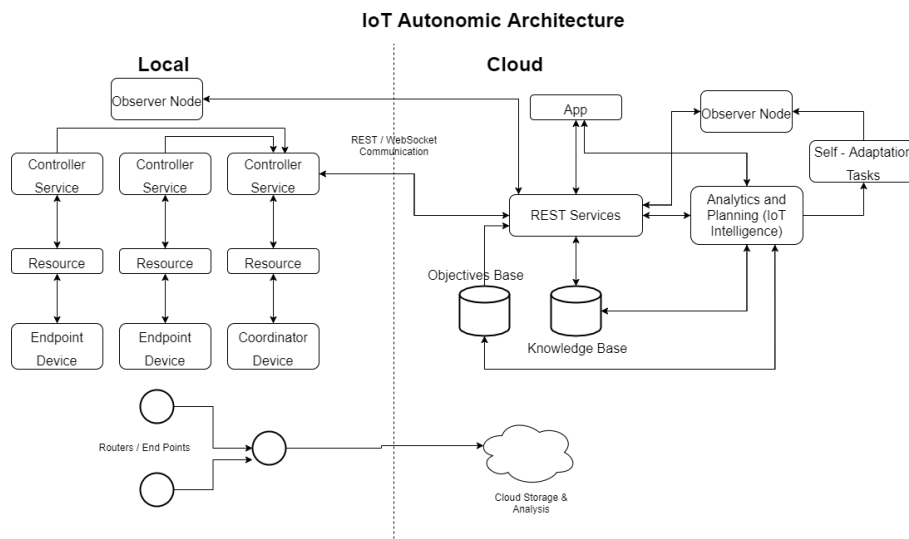


Imagen 4.3: Arquitectura IoT Autónoma propuesta por Villela [36].

La arquitectura base seleccionada cumple con aspectos necesarios para generar la arquitectura propuesta, uno de ellos es su apertura para su modificación, así como

una capa de cómputo en la nube que nos permite realizar un recabado y procesamiento de los datos de forma remota; así mismo lograr una Auto-Configuración y Autogestión de la misma manera, dejando los procesos del sistema y sus configuraciones transparentes al usuario final. Además de estar diseñada para dar pie a la evolución de características autonómicas, definiendo una base de objetivos que coordinara el propio sistema y ayudándose con un nodo de analítica y planeación, que se encarga de recopilar datos de la base de conocimiento, procesarlos y retornar valores ajustados para la configuración del sistema.

Una vez identificada la arquitectura base se procederá a su modificación para lograr incluir las propiedades de Autoensamblaje y Autorreplicación, propiedades con los que la propia arquitectura podrá reconfigurarse con la entrada o salida de dispositivos en el sistema, así como replicar la forma de trabajar del sistema en un sistema similar o adaptar funcionalidades de otro. Dicho objetivo se logrará con la implementación de nuevos módulos y la modificación a otros ya existentes.

Para el rediseño de la arquitectura se tomaran en cuenta el principio de replicación de los programas Quine [32], las nociones de Elemento Autonómico (EA) [14], principios de sistemas distribuidos, las propiedades autonómicas definidas para la arquitectura base [36], el lazo de control MAPE-K [13] y las propiedades autonómicas propuestas por Paul Horn [12].

Los requerimientos de diseño necesarios para lograr una arquitectura IoT autonómica Auto-Ensamblable y Auto-Replicable son muy similares a los necesarios para una arquitectura IoT convencional (identidad única, interoperabilidad, heterogeneidad y seguridad). Adicionalmente es necesario modificar la forma en la que están diseñados los dispositivos IoT y sus algoritmos de control, para a través de la replicación de código lograr un Auto-Ensamblaje y lograr un sistema abierto que permita la Auto-Replicación y reconfiguración del sistema, no solo en la entrada o salida de dispositivos, sino también en la configuración de ajustes del sistema, objetivos o actualizaciones de código ejecutable.

Este estilo de metamorfosis o cambio de código fuente se da en el campo de la metaprogramación [33], que se caracteriza por programas que escriben o manipulan otros programas o a sí mismos(Quine [32]), de manera que al definir programas con estas características y estructurándolos en una arquitectura IoT se busca lograr una composición de código, es decir cuando llegue una actualización de código fuente o un nuevo elemento que tenga nuevo código fuente, los dispositivos que pertenezcan a la misma comunidad tomarán esas nuevas funcionalidades compilándolas en su propio código fuente, aunque dicho comportamiento pudiera acarrear el tener código fuente que no se ejecute en los dispositivos (Código pasivo). Esto puede suceder por falta de datos, poder de procesamiento, cohesión con los demás dispositivos, o inclusive, por comportamientos definidos como, lo podría ser: el no ser designado como el dispositivo coordinador. Aunque dicho código pueda no interferir con el procesamiento normal del sistema, se prevé que pese a ser pasivo pueda activarse de presentarse las condiciones idóneas para ello.

Otro aspecto fundamental en el sistema es el cómo será regido, de manera que

cualquier dispositivo que sea agregado pueda ajustarse (Autoensamblarse) en sistema. Este tipo de comportamiento se aborda en la teoría de sistemas multi agentes con políticas de control, que se definen como conjunto de acciones, actividades, planes, normas, procedimientos, métodos de funcionamiento y configuraciones [8], por las que se rige el actuar de los agentes dentro de un sistema, normalmente impuestas por jerarquías o por comunidades, término que se refiere a un conjunto de agentes con alguna o algunas características en común [8]. Un agente puede pertenecer a una o más comunidades ya que puede compartir algunas características o capacidades de cooperación con algunos y algunas otras con otro conjunto de los mismos.

Tomando como referencia la arquitectura IoT Autónoma de Villela [36], teoría de sistemas multi-Agentes (MAS), y elementos de metaprogramación, se propone la arquitectura IoT autónoma Fig.4.4

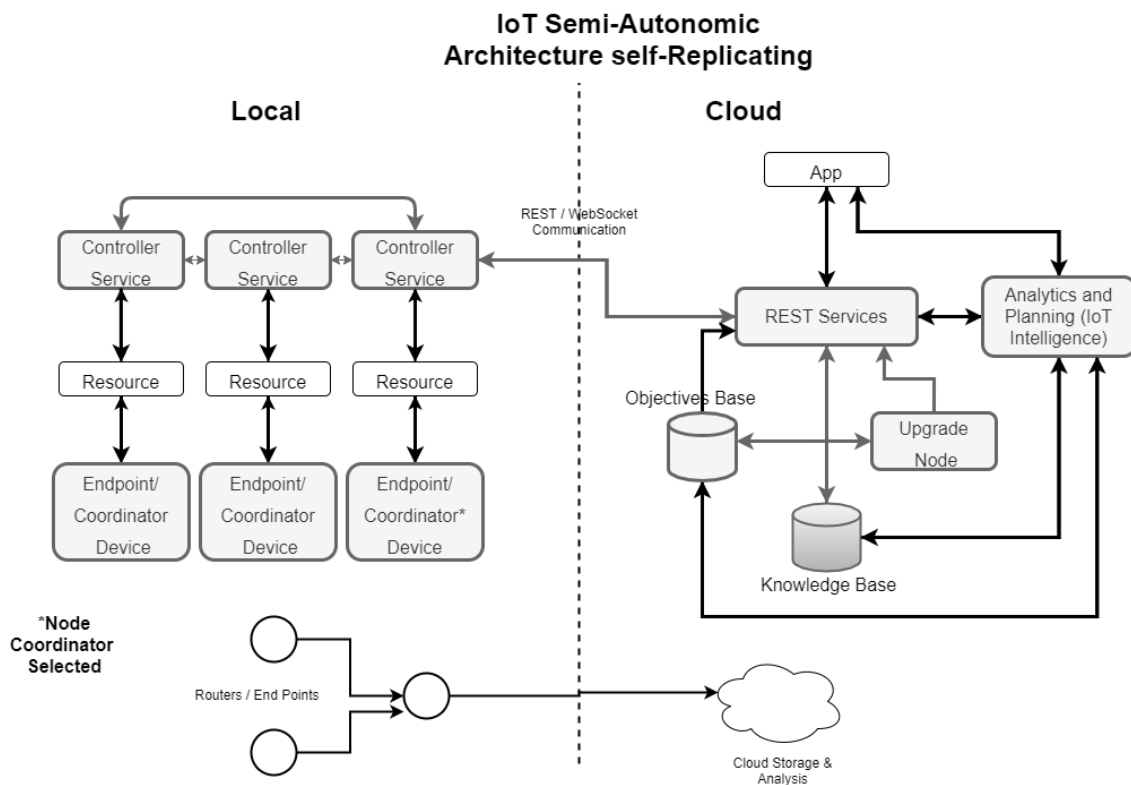


Imagen 4.4: Arquitectura IoT Autónoma, Auto-Ensamblable y Auto-Replicable.

4.3.1 Componentes eliminados/derogados

- Los nodos observadores se eliminan al integrar la capacidad de automonitoreo en el nodo de **Controller Service**, ya que de esta forma cada nodo podrá identificar fallas en su funcionamiento y notificarlas. Posteriormente el nodo designado coordinador podrá identificar tal mal funcionamiento (autosanación [13]) y notificar al administrador, y posteriormente aislar el dispositivo o proceder según las políticas de control definidas para su recuperación.

- Nodo Tareas de Autoadaptacion, es fusionado con Analytics and Planning ya que dicho nodo será el encargado de modificar las políticas de control en caso de identificar una posible mejora en los parametros de funcionamiento de los dispositivos.

4.3.2 Componentes modificados

- End point/coordinador, la tarea de nodo coordinador se habilita para todos los miembros de la comunidad de dispositivos (basado en AE [14]), aunque se elige uno como nodo coordinador con base a un algoritmo de conceso, de tal manera que a la caída del nodo coordinador se pueda elegir otro coordinador, y el sistema pueda seguir operando sin mayor problema, por lo cual el nodo End Point y el nodo Coordinador se ven fusionados.
- Controller Service brindará un canal de comunicación entre los distintos dispositivos IoT, no solo entre los nodos de una misma comunidad, sino también entre nodos de distintas comunidades, habilitando la cohesión entre dispositivos y la generación de nuevos datos, ya que así se evita una gran carga de procesamiento en el nodo de REST Service. Porqué al tener comunidades de sensores muy grandes se necesitaría un gran poder de cómputo para procesar los cruces o interacciones de datos de todos los dispositivos. Así mismo brinda acceso a la configuración y código fuente de los dispositivos en comunidades, habilitando la replicación de código fuente o fracciones del mismo(utilizando métodos de metaprogramación, como por ejemplo los programas Quine) para lograr una composición de un nuevo código fuente que será ejecutada por el propio dispositivo. Este nuevo código puede provenir de un nuevo dispositivo en la comunidad o desde el nodo actualizador, capacitando la actualización de manera local y remota. El funcionamiento de este nodo se rige por la políticas de control definidas en la base de objetivos, además de encargarse de la autosanación en el sistema tratando a cada dispositivo IoT como un elemento autónomo [14], automonitoreándose con la ayuda de políticas de alto nivel (basado en MAS [8]) de la base de objetivos, de manera que si el dispositivo falla sea capaz de notificar dicha falla y disociarse de los dispositivos activos del sistema o tratar de corregir dicho comportamiento.
- Base de Objetivos, es un nodo que trabajará con las políticas de control que rigen el comportamiento de las comunidades de agentes y sus interacciones, por lo que si se desea cambiar el actuar de algunos dispositivos IoT en concreto, el sistema no tendrá que detenerse o para redefinir los objetivos y reconfigurar los dispositivos, sino proporcionar una nueva versión de las políticas de control y distribuirlas entre los dispositivos a través del nodo REST Service y el dispositivo Coordinador en turno para que, posteriormente, realice los cambios.
- Base de Conocimiento, es un nodo redefinido donde se mantiene el almacenaje de valores de configuración obtenidos de forma autónoma; pero dichos valores serán tratados como nuevas políticas de control una vez que sean analizadas por el nodo de Analysis and Planing. Dicho procedimiento se lleva a cabo gracias a que se comunicará directamente con la base de objetivos para

dicha actualización. Así mismo se agrega el versionado de código fuente de los dispositivos IoT en el sistema, de manera que se pueda dar trazabilidad a la evolución del sistema y pueda darse reversa a la misma en caso de ser necesario.

- REST Services se encarga de la comunicación de los dispositivos IoT con la base de objetivos, proporcionando las políticas de control y cada actualización de estas a los dispositivos IoT, a su vez se encargará comunicar el nodo actualizador, que será capaz de transmitir el nuevo código ejecutable, y así como alimentar la base de conocimiento con los datos obtenidos y calculados por el sistema. También será el encargado de monitorear los nodos virtuales y se mantendrá informado del estado de los nodos físicos para notificar cualquier anomalía, además de ser la interface de tránsito de datos.
- Analytics and Planning, se encargará del análisis de los datos de la misma manera que en la arquitectura en la que se basó esta propuesta, además podrá proponer nuevas políticas de control para el mejoramiento del rendimiento en el sistema. Dichas políticas de control serán agregadas a la base de objetivos para su posterior implementación a través del nodo de servicios REST.

4.3.3 Componentes agregados

- Nodo Actualizador, es un nodo que se encarga de modificar las Políticas de Control de la base de objetivos de manera que el sistema no tenga que parar en caso de necesitarse algún cambio en su configuración, además de la actualización remota de código fuente y conexión con otras arquitecturas autonómicas, proporcionando mayor robustez al sistema.

4.3.4 Vista logica

En esta vista se representa la funcionalidad que proporcionará esta arquitectura a los diferentes sistemas IoT.

4.3.4.1 Comunicaciones

En el diagrama 4.5 se muestra la recolección y tratamiento de datos en un sistema IoT bajo esta arquitectura. Los AE recaban datos censados y los proporcionan al nodo coordinador de la comunidad, este nodo es el encargado de comunicarlos al nodo REST que paralelamente los podrá tratar y presentar a los usuarios finales, guardar registro de los mismos en una base de conocimientos y enviarlos al nodo de analítica y planeación. El nodo de analítica y planeación será el encargado de la identificación de patrones con lo que podrá proponer nuevas políticas de control para optimizar el sistema, esto a través de la adición de la nueva política de control en la base de objetivos, que posteriormente se comunicará con el nodo de servicios REST para la distribución de esta nueva política a los dispositivos coordinador, que a su vez la comunicarán a los dispositivos IoT autonómicos.

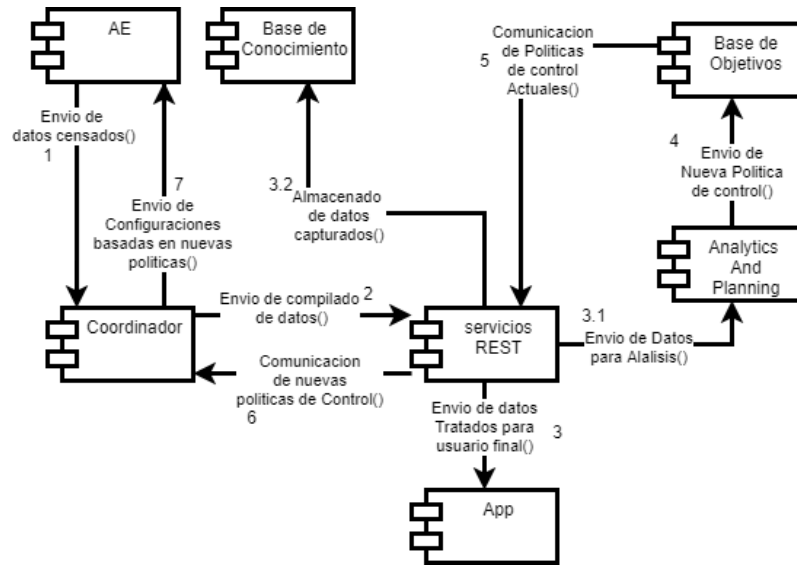


Imagen 4.5: Obtención de datos de un sistema IoT.

Otro análisis importante para realizar recae en la propiedad de autosanación del sistema, donde cada nodo cuenta con capacidades de autoconocimiento, por lo que cada elemento es capaz de identificar una falla en sí mismo, y notificarla a través de los servicios REST al administrador o usuario final, en el diagrama 4.6, se puede observar los pasos a seguir para la notificación de la falla en uno de los elementos autónomos (dispositivo IoT). Una vez identificada la falla se debe avisar al dispositivo coordinador, que enviará esta información a través de los servicios REST al nodo de analítica y planeación, que, de encontrar alguna forma de resolver el problema, propondrá una nueva política de control, o una orden de desacoplamiento del sistema. La nueva política de control estará centrada en modificar el funcionamiento del o los dispositivos con falla para lograr un correcto funcionamiento; en cualquiera de los casos se notifica al usuario final o administrador de la falla y de las acciones tomadas. En caso de desacoplamiento del sistema de un dispositivo IoT, este solo estará habilitado para recibir comunicación del dispositivo coordinador que podría facilitar una nueva compilación de código fuente o políticas de control.

4.3.4.2 Ensamblaje de Dispositivo

El ensamblaje de un dispositivo en el sistema esta representado en el diagrama 4.7, se muestra el procedimiento que debe seguir el dispositivo antes de iniciar la ejecución de sus tareas principales. Al arribo de un dispositivo a una comunidad de agentes, este dispositivo deberá enviar un mensaje de búsqueda de un nodo coordinador esperando un tipo determinado por la respuesta; en caso de recibir respuesta, facilitará la información de su código fuente al coordinador, esto con la intención de identificar si cuenta con la versión actual de software del sistema; si es el caso, el coordinador comunicará las políticas de control de funcionamiento al dispositivo e iniciara su funcionalidad normal; en el caso de no contar con una versión actual de software, el dispositivo coordinador será el encargado de recompilar el código fuente

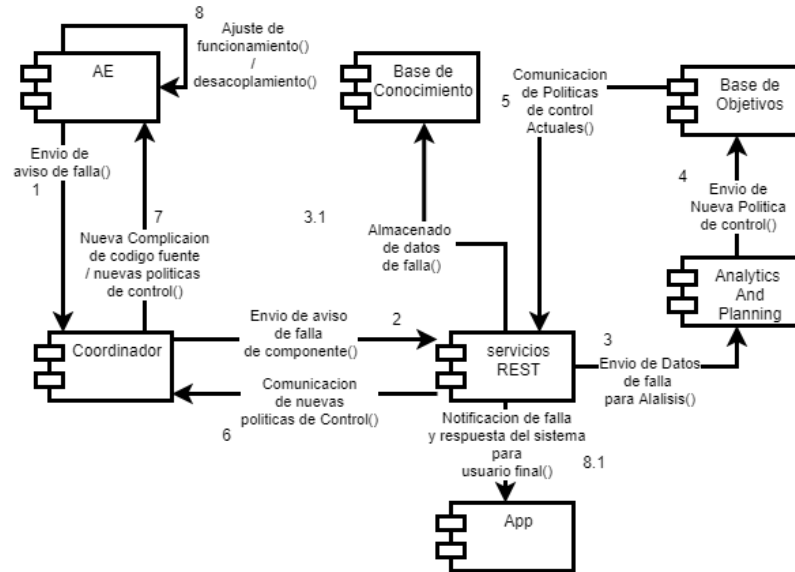


Imagen 4.6: Proceso de autosanación.

o simplemente facilitar una copia del código fuente ya ensamblado con anterioridad en el sistema, según se requiera, para posteriormente reiniciar el dispositivo y ejecutar el nuevo código fuente(CF).

En el caso que no exista un dispositivo coordinador, esperará un tiempo determinado la llegada de un mensaje que comunique un nuevo coordinador o el postulado de alguno, si recibe este mensaje regresará al estado de búsqueda de coordinador, en caso contrario, si el tiempo se agotó y no se recibió ninguna comunicación que informara un nuevo coordinador o postulado a coordinador, el elemento IoT procederá a postularse como coordinador enviando un mensaje de solicitud de votación, que podrá ser respondido con voto positivo si el dispositivo votante no hubiera votado por algún otro elemento o negativo en caso contrario. Posteriormente existirá una ventana de tiempo para el recabado de los votos y posterior conteo, si el número de votos positivos es mayor al número de votos negativos, el dispositivo será electo coordinador y tendrá que comunicarlo a la comunidad de manera inmediata, si no deberá regresar al estado de búsqueda de coordinador.

Una vez que un dispositivo sea nombrado coordinador procederá a recabar la información de la comunidad de los agentes IoT(AE), la información solicitada será su versión de software y hardware, con lo que podrá decidir si todos en la comunidad tienen la versión mas reciente de software y, de no ser así, ensamblar una nueva versión de código fuente para todas las versiones de hardware del sistema, para luego distribuir dichas compilaciones de código fuente, no eximiendo al dispositivo coordinador de dicha actualización. Si el dispositivo coordinador compila una nueva versión de código fuente para sí mismo después de distribuir la nueva compilación para todos los demás dispositivos en la red, procederá a un reinicio y reincorporación a la comunidad.

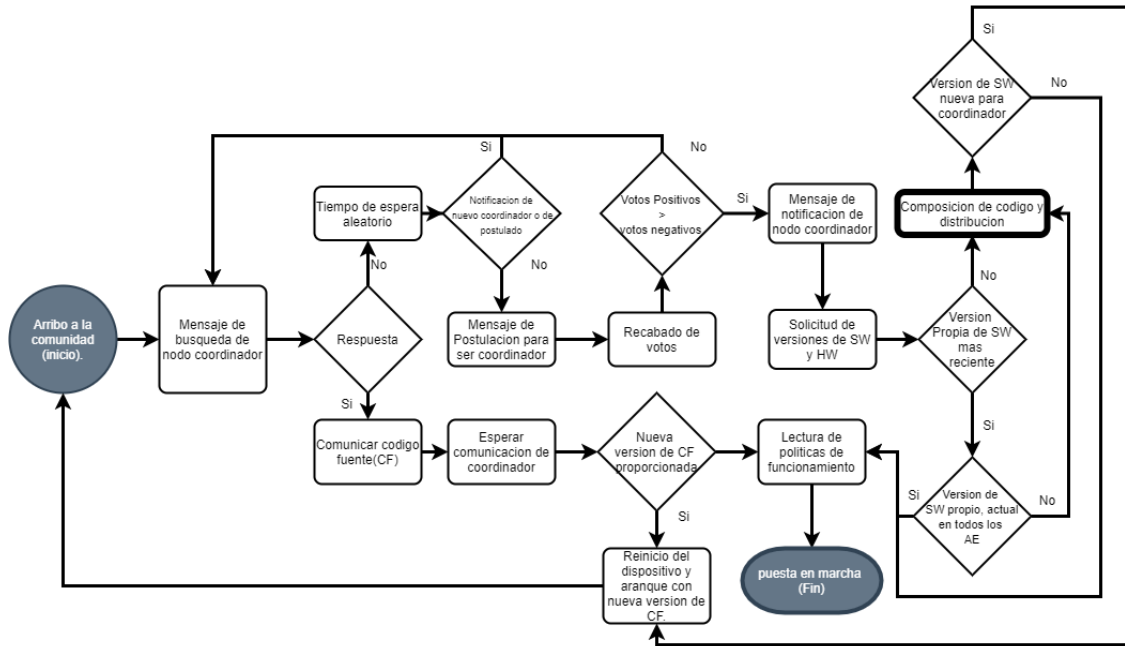


Imagen 4.7: Ensamblaje de dispositivo IoT a una comunidad de AEs.

Los eventos que pueden implicar ensamblaje de código identificados son tres casos particulares: el arribo de una nueva versión de código a través del nodo de servicios REST, que a su vez lo recibe a través del nodo actualizador; el arribo de un dispositivo IoT que se une a la comunidad de agentes, donde si el dispositivo entrante tiene una versión de software desactualizada y/o cuenta con una versión de Hardware no presente en la comunidad, se procederá a un ensamblaje de código y el arribo de un dispositivo IoT con una versión más reciente de código fuente. El segundo escenario es el único que implica un ensamblaje de código para un solo dispositivo y la actualización del mismo, mientras que el resto de eventos desencadenan un ensamblaje de código fuente para todos los dispositivos en la comunidad de AE, diagrama 4.8. Siempre que exista una actualización de código fuente el dispositivo que recibe la nueva versión de código, deberá reiniciar y arrancar su funcionamiento con la nueva versión.

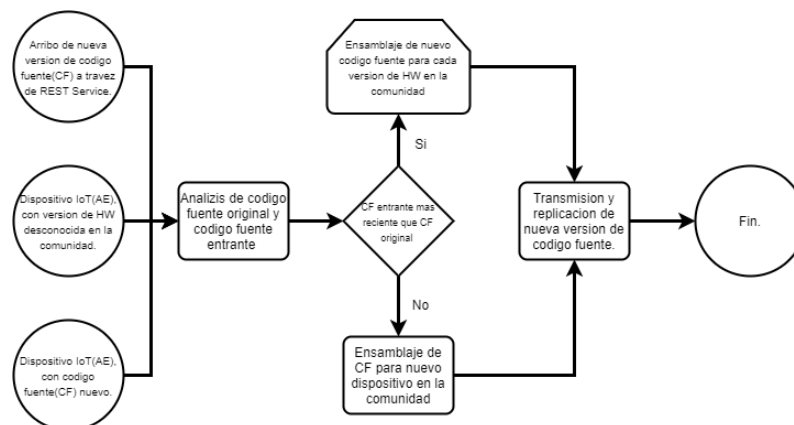


Imagen 4.8: Eventos desencadenadores de autoensamblaje.

4.3.5 Vista de despliegue

En esta vista se muestra la estructura del Software de un sistema IoT, cómo ésta dividido, sus relaciones y sus dependencias.

4.3.5.1 Pila de Protocolos

En la figura 4.9 se puede observar un diagrama con la pila de protocolos que se consideran para sistemas IoT. Considerando la naturaleza heterogénea de los sistemas IoT se agrega el protocolo para red celular 5G a la propuesta original del libro “Internet of Things: A hands-on approach” [7].

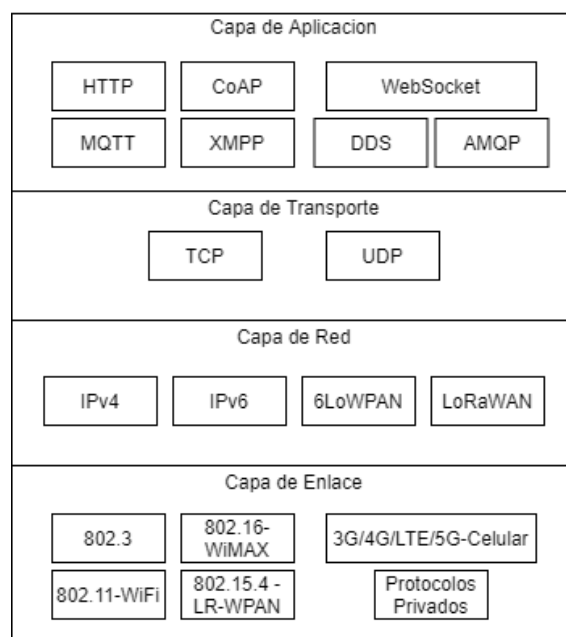


Imagen 4.9: Pila de Protocolos.

4.3.5.2 Estructura de código

La estandarización del código fuente es importante ya que esto facilitará la tarea de lectura e identificación de funciones. Así como sus dependencias para lograr un Autoensamblaje y Autorreplicación de código, es necesario contar con una estructura de código que nos permita el análisis de forma automática, dicha estructura de código está basada en la modularidad de este, por lo que se propone seguir una estructura similar a la mostrada en la figura 4.10.

4.3.5.3 Estructura de Módulos

La estructura del primero módulo “información de dispositivo” se muestra en la figura 4.11. Este módulo de código fuente deberá contener la información de versión de software, versión de hardware y capacidades del dispositivo. Las capacidades

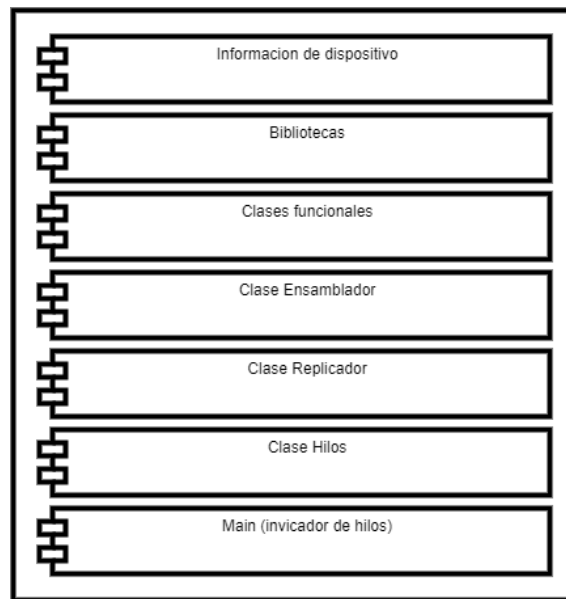


Imagen 4.10: Estructura de código fuente.

estarán relacionadas a los elementos que conforman el dispositivo IoT, tales como actuadores o sensores, especializados en tal o cual tarea; un ejemplo de esto podría ser un sensor DTH que nos provee de la capacidad de lectura de temperatura y humedad, por lo que DTH debería ser listado como capacidad, esta información servirá para posteriores análisis.

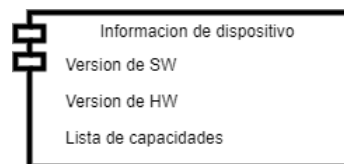


Imagen 4.11: Estructura de módulo de información.

El módulo de bibliotecas deberá contener únicamente las bibliotecas utilizadas en la estructura del código, esto permitirá el análisis y adición de bibliotecas (en caso de ser necesario) para la ejecución de nuevo código fuente.

En el modulo de clases funcionales contaremos con dos submódulos que serán denominados clases activas(CA) y clases Pasivas (CP) como se ilustra en la figura 4.12. Esto es así ya que al contar con código Autoensamblable se podría presentar la situación donde un dispositivo no pudiera realizar alguna tarea especifica por la falta de algún componente físico, el código de ejecución de tales tareas es denominado código pasivo. Es importante conservar este código ya que podría existir alguna actualización de componentes que dote a los AE de dicha capacidad, o podría arribar un AE que cuente con la capacidad para realizar tal tarea, y de esta manera solo sería necesario reensamblar el código para ejecutar estas tareas, por su parte el submódulo de CA contendrá el código para el recabado de datos, procesado o interacción física con el ambiente propio del sistema IoT que se diseña, un ejemplo sería

la clase que obtiene los datos de temperatura y humedad y los reporta al dispositivo coordinador.

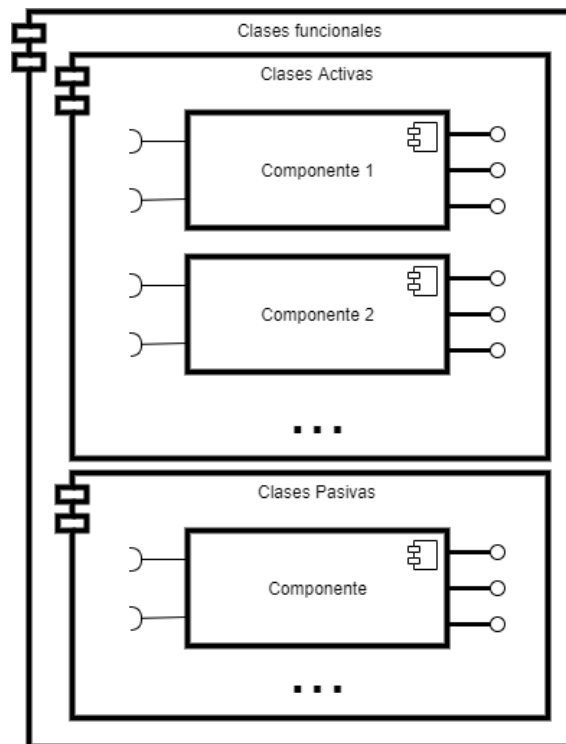


Imagen 4.12: Estructura de módulo de clases funcionales.

Se recomienda que el modelado de este código siga la definición de componente figura 4.13, donde se define los servicios necesarios (es decir las “capacidades” del dispositivo IoT); este modelo es propuesto en [22].



Imagen 4.13: Estructura de diseño por componentes.

Dentro del módulo ensamblador contaremos con la clase y métodos encargados del análisis del código fuente. El procesamiento que debe realizar es la obtención de código fuente original y código fuente nuevo (CF a ensamblar). Este proceso se detallara más adelante.

El módulo replicador contendrá la clase y métodos encargados de facilitar el nuevo código ensamblado a los distintos dispositivos IoT. Tanto este modulo de código como el modulo de ensamblador no se ejecutarán en un dispositivo IoT que no sea el dispositivo coordinador ya que las tareas de ensamblaje de código y replicación son tareas exclusivas del propio coordinador en turno.

En el módulo de la clase hilos, tenemos dos submódulos como se ilustra en la figura 4.14, en el submódulo de hilos core se necesita una definición similar a las clases funcionales, es decir, proporcionar información de las dependencias (capacidades necesaria) para su correcta ejecución, ya que cada hilo core contendrá el código que realiza las tareas para las que fue diseñado el sistema(programa principal), haciendo uso de las clases definidas en el área de clases funcionales. En el submódulo hilo de control, se tendrá el código que permita al AE la comunicación en la comunidad de agentes, y la comunicación con el nodo de servicios REST así como el control en general de su actuar como agente. Aquí a su vez, se encontrará el código que hace uso de la clase ensamblador y replicador y que solo podrá ejecutar el AE designado coordinador.

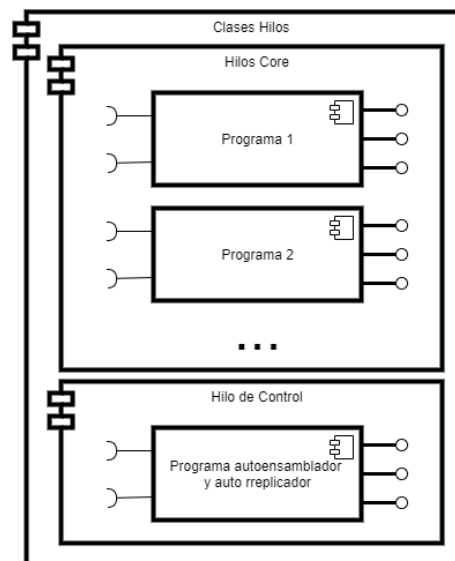


Imagen 4.14: Estructura de módulo de clases hilos.

El módulo Main, solo contendrá las invocaciones de los hilos de procesamiento core y el hilo de procesamiento control, solo se incluirán las invocaciones a los hilos que estén definidos como código activo y no código pasivo, es decir, si la tarea no puede ejecutar por la falta de capacidades no contará con un hilo de ejecución habilitado para dicha tarea.

4.3.5.4 Ensamblaje de código

El proceso que se sigue para la fusión de código en el nivel de clases funcionales es ilustrado en la figura 4.15, el dispositivo coordinador contara con dos versiones de código fuente, la original del dispositivo para el que se está realizando la composición de código, y el código entrante, en ambos documentos se podrá analizar las capacidades necesarias para la ejecución del código contenido. El primer bloque a ensamblar es el bloque de información de dispositivo, bloque que es tomado directamente del código fuente original. Ya que esta información no cambiará, en el bloque de bibliotecas se obtiene la lista de bibliotecas necesarias para ejecutar el código de cualquiera de las dos versiones de código fuente, obteniendo la fusión de las bibliotecas (omitiendo las bibliotecas repetidas).

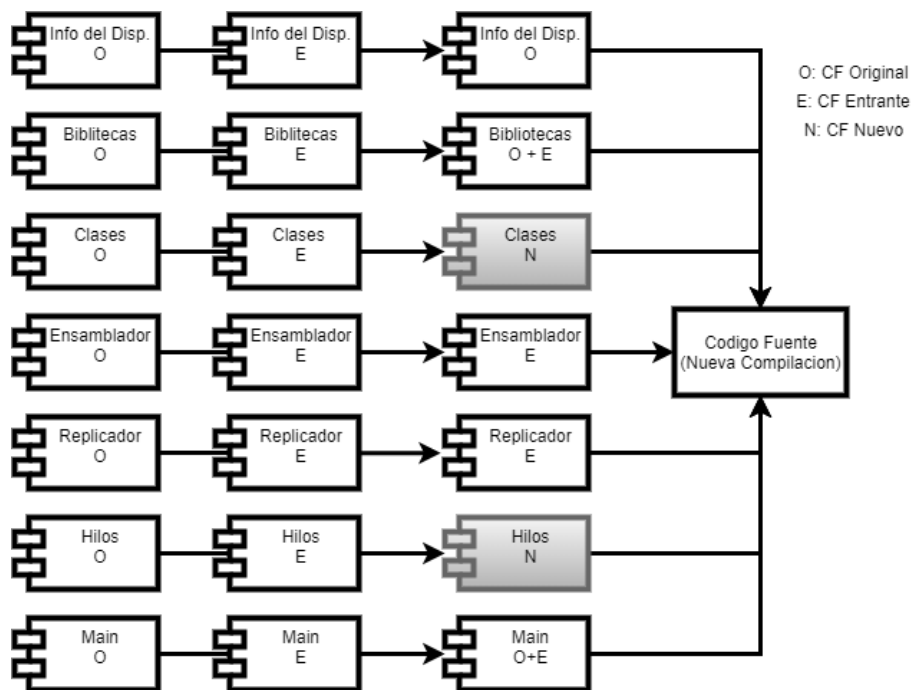


Imagen 4.15: Proceso de ensamblaje de código.

Para el ensamblaje del módulo clases funcionales es necesario otro análisis específico descrito en el diagrama 4.16, en esta composición se pueden algunos casos específicos, estos son:

- Cuando las dos versiones de código tienen una clase con el mismo nombre y el dispositivo origen tiene las capacidades para ejecutar cualquiera de las dos versiones, en este caso prevalece la versión más reciente (código entrante) y es asignada a código activo.
- Cualquiera de las dos versiones de código tiene una clase que el otro no, si el dispositivo tiene las capacidades para ejecutarla, es agregada a código activo, en el caso contrario es catalogada como código pasivo.

- Cuando los archivos de código fuente presenten dos clases con el mismo nombre y solo alguna de las dos versiones pueda ser ejecutada en el sistema, prevalecerá aquella que dependa de las capacidades presentes en el dispositivo para el que se está realizando el ensamblaje y es catalogada como código activo.
- Si ambos archivos presentan una clase para la que no se cuenta con las capacidades para ejecutarla, prevalecerá la versión más reciente (código entrante) y es agregada a código pasivo.

Continuando con el ensamblaje de código fuente, el módulo ensamblador para la nueva versión de código fuente será siempre la versión más reciente, solo prevalecerá la versión contenida en código fuente original del dispositivo para el que se está realizando el ensamblaje cuando no se cuente con este módulo en el código entrante; esta situación también se puede presentar para el módulo replicador. El módulo de hilos tiene una composición muy similar a la del módulo clases funcionales, realizando el mismo tipo de análisis antes mencionado, y en su submódulo hilo de control prevaleciendo la versión más reciente. El ensamblaje del modulo main consiste en la prevalencia de las invocaciones de hilos de las tareas que pueda ejecutar evitando invocaciones a código pasivo.

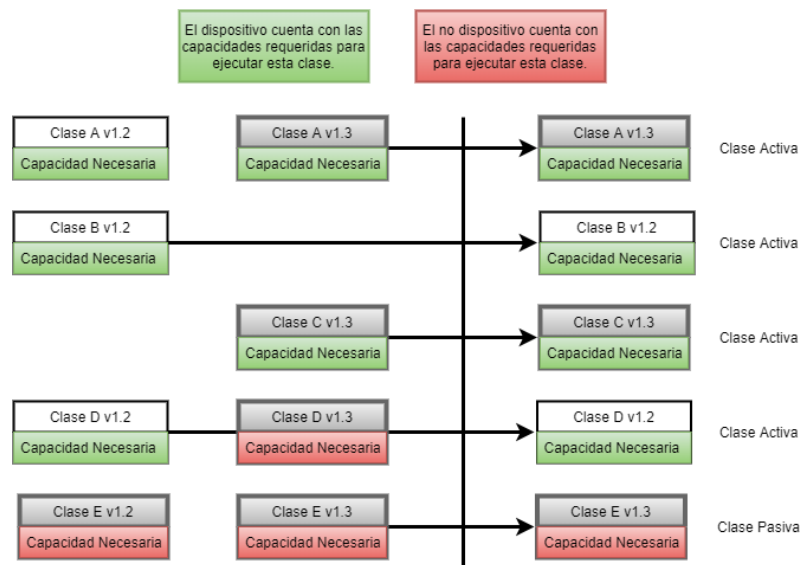


Imagen 4.16: Proceso de ensamblaje de clases activas y pasivas.

4.3.6 Vista de procesos

4.3.6.1 Arribo de código fuente o dispositivos

El arribo de un dispositivo a la comunidad de dispositivos planea distintos escenarios, uno de ellos es descrito en el diagrama 4.17, donde el proceso que sigue es la comunicación de versiones de Sw y Hw; en este caso, el dispositivo coordinador solicitará el código fuente del dispositivo entrante y procederá a realizar un ensamblaje automático para cada versión de Hardware en la comunidad y posteriormente enviarlo a cada dispositivo, y que cada dispositivo lo ponga en marcha.

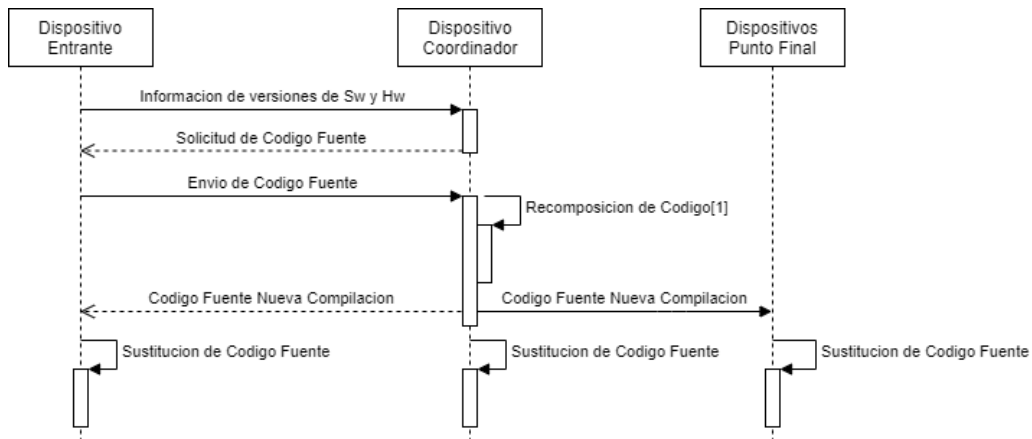


Imagen 4.17: Proceso de ensamblaje de código fuente proveniente de un dispositivo nuevo en la comunidad.

Otro escenario es el arribo de un dispositivo entrante con código fuente obsoleto, pero de una versión de Hardware conocida (existente en la comunidad), diagrama 4.18; en este caso el dispositivo coordinador solo deberá facilitar el código fuente al dispositivo entrante para que remplace su compilación actual y pueda ponerse en funcionamiento.

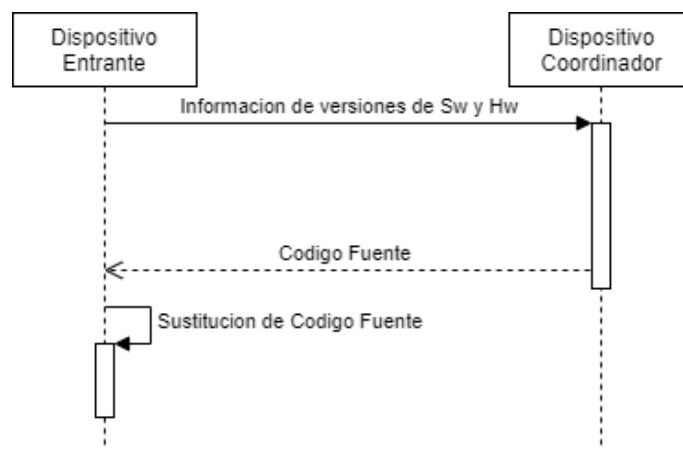


Imagen 4.18: Sustitución de código fuente de dispositivo entrante.

Así mismo se puede presentar el escenario donde el dispositivo entrante directamente solicite código fuente para unirse a la comunidad (diagrama 4.19), el dispositivo coordinador realiza el ensamblaje de código fuente basado en las capacidades del dispositivo solicitante y posteriormente lo envíe.

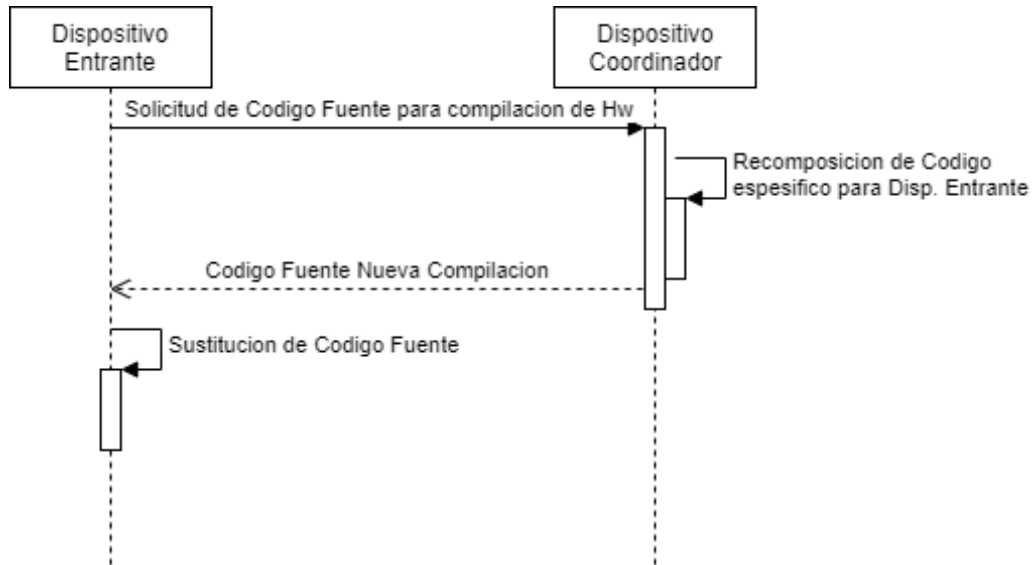


Imagen 4.19: Replicación directa de código.

A través del nodo servicios REST el nodo actualizador puede proveer de una nueva versión de código ensamblable (diagrama 4.20), por lo tanto el código fuente arribará a la comunidad a través del nodo de servicios REST, facilitándolo al dispositivo coordinador que realizará un ensamblaje del código fuente para cada versión de hardware en el sistema. Cada dispositivo en la comunidad procederá a ejecutar la nueva versión de código fuente. Como nota adicional cada nuevo ensamblaje que se realiza es enviado al nodo de servicios REST para su almacenaje en el nodo base de conocimiento.

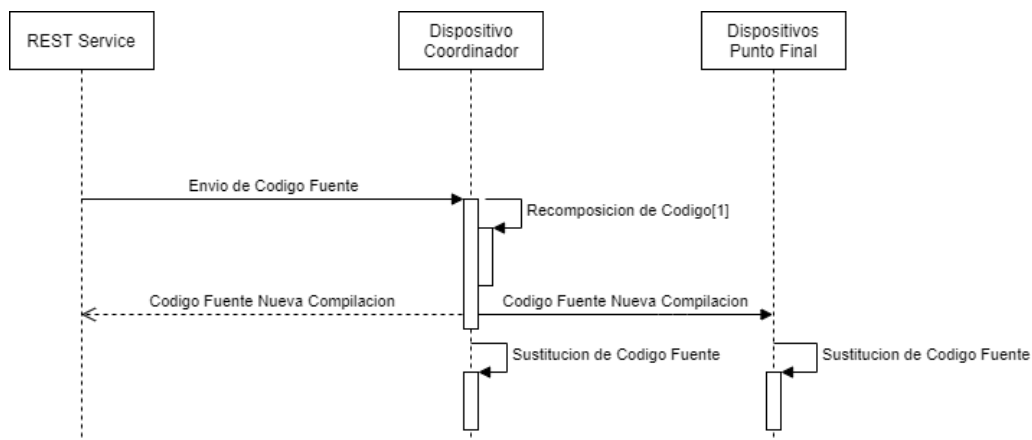


Imagen 4.20: Ensamblaje de CF proveniente de servicios REST.

Un posible escenario para la Autorreplicación se presenta en el diagrama 4.21, cuando un dispositivo IoT cuente con una capacidad o algunas capacidades de las que no esté haciendo uso por la falta de programación de código fuente que las utilice, se procederá a dar aviso al dispositivo coordinador de dicha situación. A su vez el nodo coordinador solicitará código fuente para ensamblar al nodo de servicios REST. El nodo de servicios REST será el encargado de buscar código fuente para explotar las capacidades de los dispositivos IoT cuando no exista código para hacerlo en el propio sistema, tomando como primer paso la búsqueda de código fuente para el uso de las capacidades en todas las comunidades de AE en el propio sistema; luego, revisando de forma escalonada en sistemas con el mismo campo de dominio, con dependencias compatibles y con proveniencia de arquitecturas confiables, es decir si no encuentra el código fuente buscado en un conjunto de arquitecturas identificadas como arquitecturas confiables, procederá a buscar solo que cuente con dependencias compatibles y, en un caso extremo, sólo que el sistema en el que esté buscando este bajo el mismo campo de dominio, esto con el fin de ayudar a la identificación capacidades necesarias para la evolución del sistema, ya que tomando como referencia el último caso podría ensamblarse código pasivo, ya que sería código que usa algunas capacidades del sistema con las que sí cuenta el dispositivo IoT pero también necesita otras con las que no cuenta, pero notificando al administrador o usuario final de esta situación para su posterior revisión.

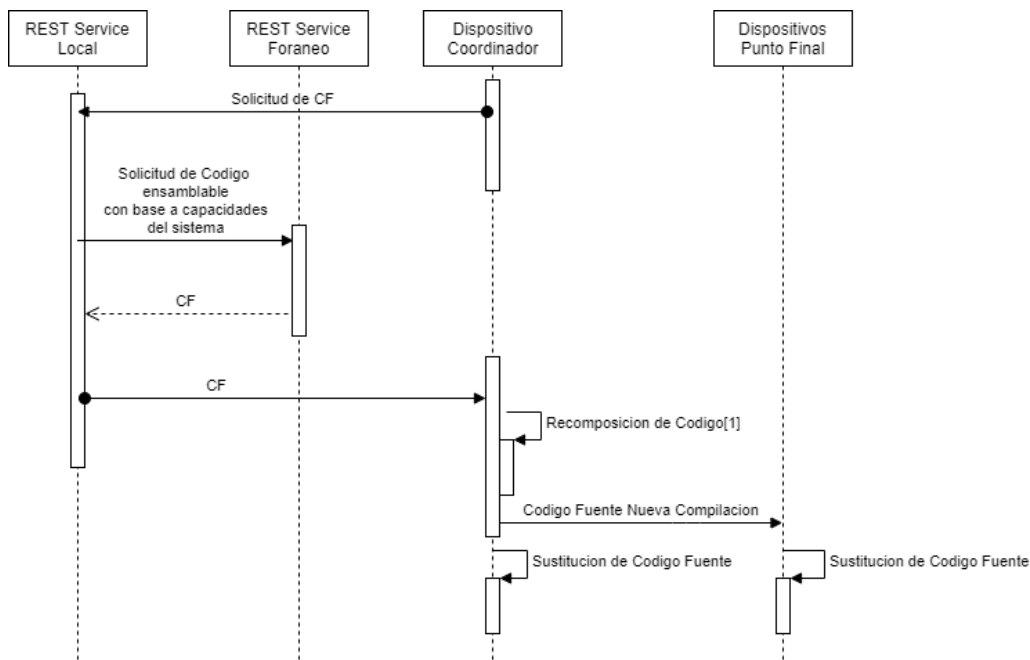


Imagen 4.21: Autoensamblaje y autorreplicación de código fuente solicitado.

Este proceso sigue un principio de potencial de máximo funcionamiento, es decir, si puede realizar alguna tarea realizarla, apegándose a un modelo de capacidad instalada, capacidad habilitada.

4.3.7 Vista física

La vista física está diseñada para mostrar el sistema en el ámbito de sus componentes físicos, sus conexiones y su topología. Por la naturaleza diversa y heterogénea de los sistemas IoT se propone un despliegue de comunidades de agentes como se ilustra en la figura 4.22, donde se muestra la comunicación entre comunidades diferentes de dispositivos a través de los AE que son asignados como coordinador. Además, conectando dichos nodos coordinador al cómputo en la nube, utilizando los servicios REST. Es importante mencionar que estos modelos se puede adaptar a las necesidades específicas de cada sistema IoT de ser necesario solo conservando la presencia de un AE coordinador que se pueda comunicar con los demás elementos en la comunidad o red y pueda conectar con servicios REST.

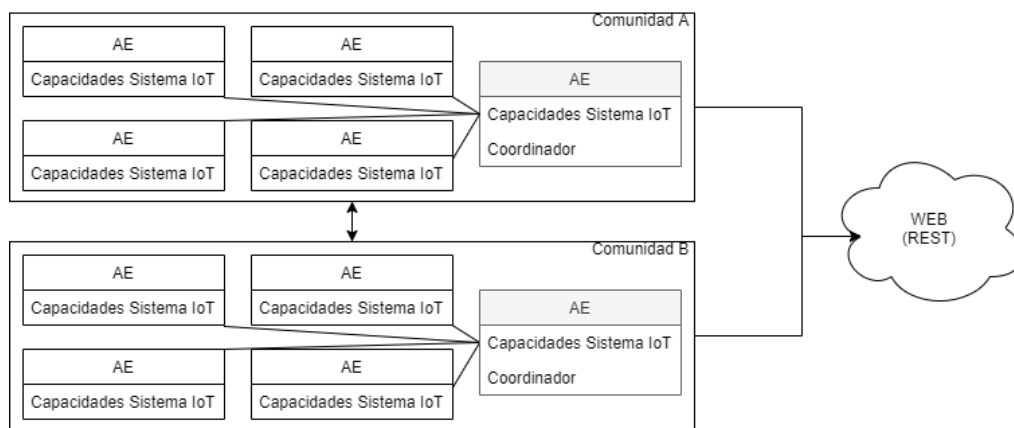


Imagen 4.22: Posible escenario físico de sistema IoT.

4.3.8 Vista de escenarios

Una vez identificadas las cuatro vistas antes mencionadas (vista de despliegue, vista lógica, vista de procesos y vista física), se puede identificar el funcionamiento de la arquitectura y sus métodos. En el diagrama de vista general 4.23, se puede observar la comunidad de AEs (Endpoint) con un nodo coordinador (Coordinator Device) que puede realizar del Autoensamblaje y Autorreplicación de código, su conexión bidireccional con servicios REST (REST Services), el paso de datos al módulo de aplicación (App) para usuario final o administrador, su conexión con base de conocimientos (Knowledge Base) y su interacción con analítica y planeación (Analytics and Planning); donde el nodo de analítica y planeación esta habilitado para generar nuevas políticas, así como el nodo de actualización (Upgrade Node), que no solo puede generar nuevas políticas sino proveer de nuevo código fuente para ser ensamblado.

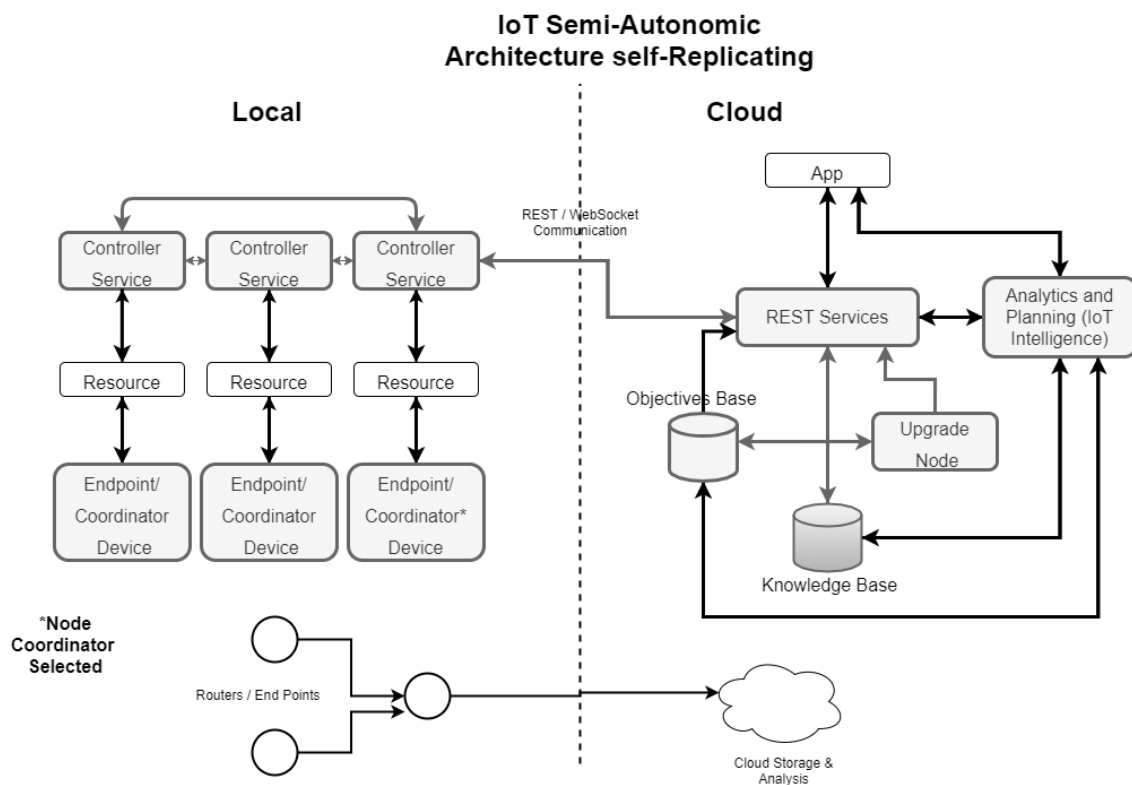


Imagen 4.23: Vista integradora de arquitectura propuesta..

4.3.9 Comparativa

En la tabla 4.22 se muestran trabajos relacionados al campo de estudio encontrados en la literatura donde se abordan los enfoques del cómputo autónomo, arquitecturas basadas en sistemas multi-agente, propiedades importantes de los sistemas IoT, y se realiza una comparativa con la propuesta arquitectónica. Además, se muestran los parámetros más relevantes para este trabajo en el diseño de arquitecturas IoT, iniciando la capacidad de escalabilidad del sistema, su heterogeneidad y las propiedades autónomas. Además, tomando en cuenta los parámetros que impactan en sistemas IoT y sistemas multi-agente como la interoperabilidad, el cómputo distribuido, la definición de elementos autónomos como dispositivos IoT y las nuevas capacidades propuestas (Autoensamblaje y Autorreplicación) para esta arquitectura.

Arquitecturas	Villela [36]	Ruiz [37]	Manate [38]	Movahedi [39]	propuesta
Escalabilidad	X	X	X	X	X
Heterogeneidad	X	X	X	X	X
Distribuido		X	X		X
Interoperable	X	X	X	X	X
CAC	X	X			X
Basado en AE		X	X		X
WSN	X	X		X	X
Base de objetivos	X			X	X
Base de conocimiento	X			X	X
Autoconfiguración	X	X		X	X
Autosanación	X				X
Autooptimización	X				X
Autoprotección	X				X
Autoensamblaje					X
Autorreplicación					X

Tabla 4.22: Comparativa de arquitecturas IoT y Propuesta.

Capítulo 5

Experimentación y Análisis de Resultados

En este capítulo se exponen los experimentos desarrollados para la validación de la propuesta. Para realizar esta tarea se seleccionó el caso de estudio de camas de cultivo urbano hidropónicas. Se muestran las pruebas estáticas realizadas con el objetivo de seguir los flujos de la aplicación y los experimentos dinámicos a los que se sometieron los mecanismos mostrados en la propuesta. Se realizó una investigación concerniente a los simuladores de sistemas multi-agente, seleccionando el software de GAMA Platform. En las siguientes secciones se muestra a detalle el procedimiento seguido y los resultados obtenidos.

Además de los experimentos mostrados en este capítulo, se desarrollaron pruebas en Python con programas Quines, esto con el fin de realizar una implementación de un sistema Autoensamblable y Autorreplicable en las camas de cultivo hidropónico con las que se cuenta en el laboratorio de computación en CINVESTAV Guadalajara.

5.1 Descripción del Caso de Estudio

De la gran diversidad de sistemas IoT existentes, que podrían verse beneficiados se seleccionó el caso de estudio de camas de cultivo urbano hidropónicas, esto debido a su simplicidad y las posibles variaciones con las que puede contar. Es importante mencionar que la selección de caso de estudio se realizó basados en la sencillez y la experiencia previa con estos sistemas, aunque la arquitectura puede implementarse en otros tipos de sistemas IoT siempre y cuando se encuentren bajo el mismo campo de dominio.

El caso de estudio está conformado por dos sistemas IoT que se encuentran bajo el mismo campo de dominio, la agronomía urbana. Los sistemas serán considerados como camas de cultivo con una funcionalidad similar y algunas capacidades diferentes, estos sistemas se encuentran situados en diferentes puntos geográficos y están conectados a internet, los sistemas confían el uno del otro y ambos están desarrollados bajo la arquitectura propuesta. En la figura 5.1 se muestra un diagrama con la representación de la distribución física de los sistemas IoT de camas de cultivo hidropónico y sus capacidades, en la imagen 5.2 se muestra la fotografía de una de las camas de cultivo hidropónico con las que se cuenta en el laboratorio de com-

putación en CINVESTAV Guadalajara .

Los escenarios principales a analizar son los siguientes:

- Escenario 1: el sistema 1 cuenta con dispositivos IoT que tienen capacidades para realizar una tarea, pero no cuentan con la programación para hacerlo, y el sistema 2 cuenta con dispositivos que tienen la misma capacidad, pero sí cuentan con la programación para explotar dicha capacidad.
- Escenario 2: el arribo de un dispositivo IoT (cama de cultivo hidropónico) proveniente del sistema 2, a una nueva comunidad de dispositivos IoT perteneciente al sistema 1.

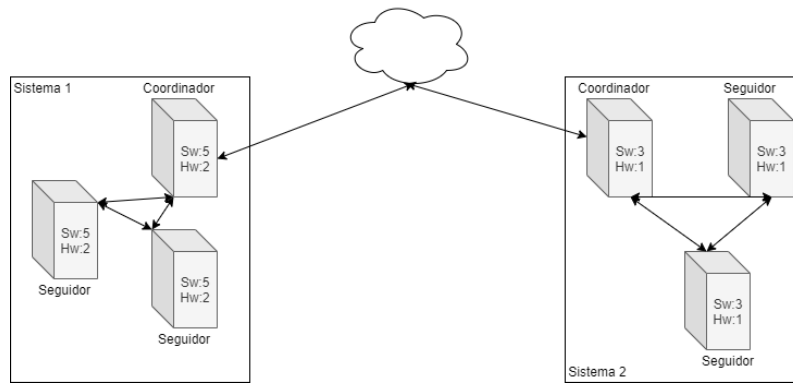


Imagen 5.1: Distribución física de los sistema IoT.

El código fuente de los dispositivos en cada sistema (comunidad) se muestra en el diagrama 5.3, las secciones sombreadas serán las implicadas en el proceso detallado de autoensamblaje de código.

5.2 Experimento Estático

5.2.1 Primer escenario

Ya que contamos con dos sistemas IoT de camas de cultivo hidropónicas, donde cada sistema cuenta con algunas capacidades similares (activación de riego), y con otras que los diferencian entre sí: obtención de temperatura por parte del sistema 1 y cambio de color de luz y obtención de PH por parte del sistema 2. Además de contar con diferencias en la sección de capacidades, y en el caso del sistema 1, contar con una capacidad para la que no cuenta con código que la utilice. El proceso seguido para la solicitud de código fuente a una arquitectura bajo el mismo campo de dominio y denominada de confianza es el mostrado en el diagrama 5.4.

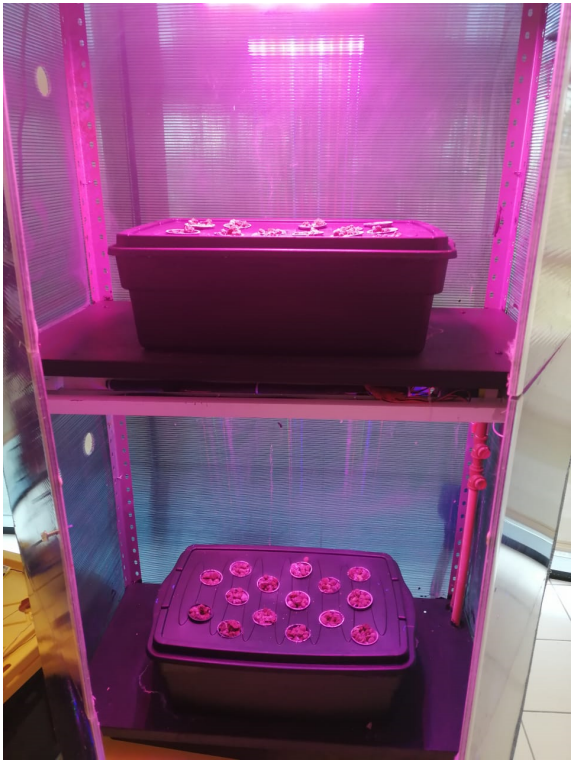


Imagen 5.2: Cama de cultivo hidropónico de CINVESTAV Guadalajara.

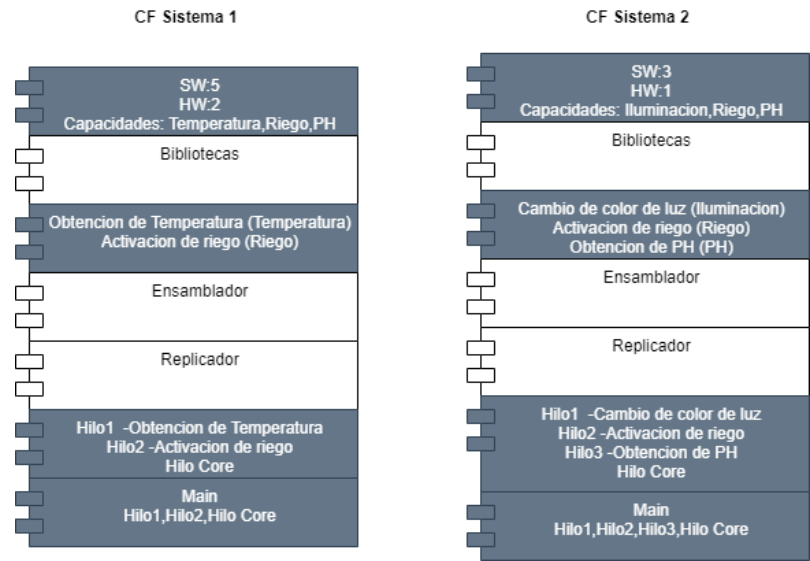


Imagen 5.3: Estructura de código fuente.

En el diagrama 5.5 se ilustra el proceso de ensamblaje de código fuente siguiendo el mecanismo propuesto en el diagrama 4.15, y detallado en el diagrama 4.16. Se puede observar cómo en este caso particular solo se ensambla el código fuente que habilita el censado de PH en las camas de cultivo (elementos IoT).

IoT por cuestiones de capacidades será catalogado como código pasivo. La secuencia de ensamblaje a nivel dispositivo se detalla en el diagrama 5.6.

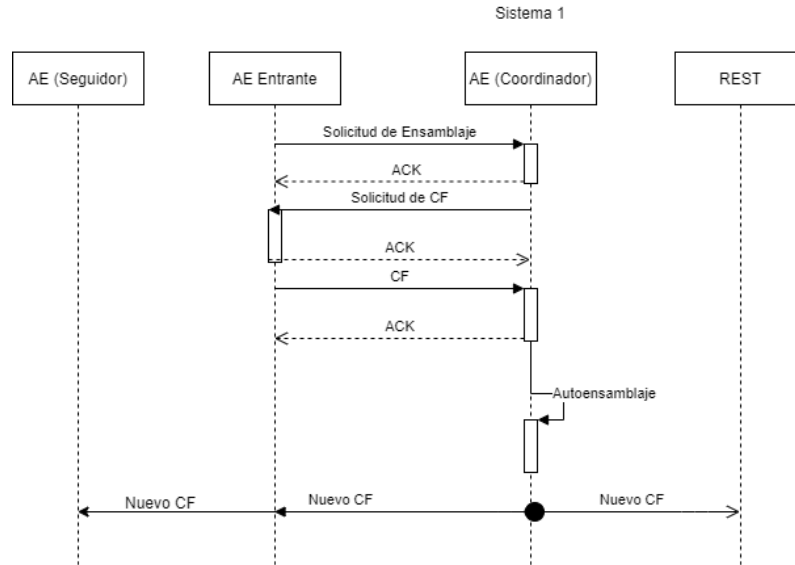


Imagen 5.6: Estructura de código fuente.

El ensamblaje a nivel código se lleva a cabo como se muestra en el diagrama 5.7, esta vez el ensamblaje nos otorga dos ensamblajes de código, la primera es para los dispositivos con los que ya contábamos en nuestra comunidad y el segundo para el nuevo dispositivo.

5.3 Experimento Dinamico

5.3.1 GAMA Platform

GAMA es un entorno de desarrollo de modelado, para la generación de simulaciones especialmente explícitas basadas en agentes. El software de GAMA es de propósito general y puede ser utilizado en una gran cantidad de aplicaciones en diferentes capos de dominio, GAMA es una plataforma de diseño desarrollada por el Instituto de Investigación para el Desarrollo (IRD) de Francia, que fue diseñada para el modelado de sistemas Multi-agente, la descripción de su página oficial es la siguiente, “*GAMA es una plataforma de simulación con un completo entorno de desarrollo integrado para el modelado y simulación para crear modelos de agentes espacialmente explícitos*” [51]. GAML es el lenguaje utilizado en GAMA, codificado en Java, es un lenguaje basado en agentes, que brinda la posibilidad de construir modelos con varios paradigmas de modelado. Una vez que su modelo se diseñó, algunas características permiten explorarlo y calibrarlo, utilizando los parámetros que se definieron como entrada de la simulación.

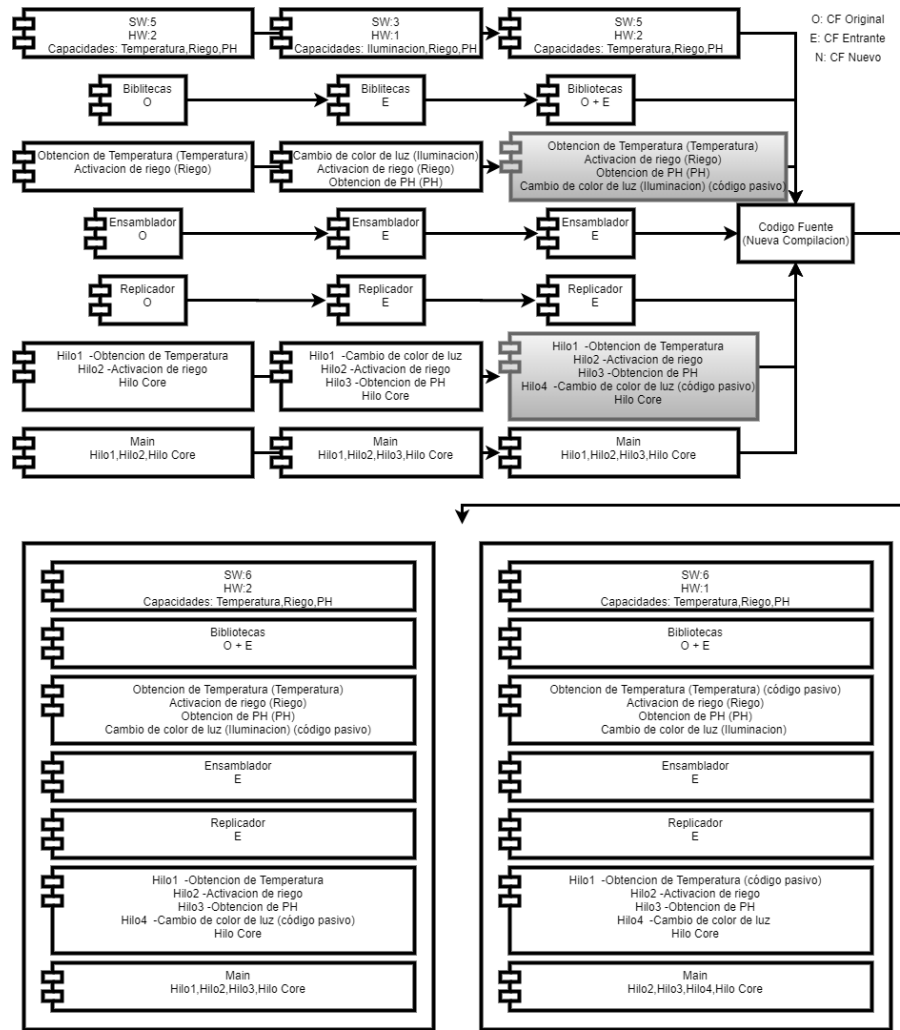


Imagen 5.7: Fusión de clases de código de dispositivo entrante.

5.3.2 Experimentación

Se realizaron múltiples experimentos para mostrar el Autoensamblaje y la Autorreplicación en la GAMA Platform, donde se modelaron los agentes con los bloques definidos en la arquitectura. La definición de las capacidades es mostrada en la imagen 5.8, estas capacidades se trataron como estáticas, y aplican para todos los agentes de la comunidad. Las clases activas originales de muestran en la imagen 5.9, y las clases pasivas en la imagen 5.10.

Las clases se muestran en una tupla conformada del nombre de la clase y su dependencia (capacidad necesaria para ejecutarla).

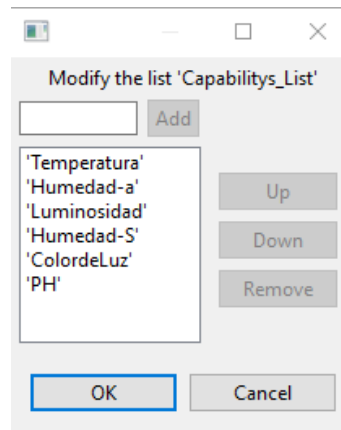


Imagen 5.8: Lista de capacidades.

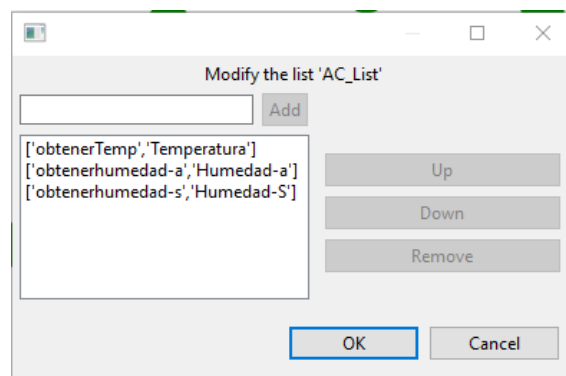


Imagen 5.9: Lista de clases activas.

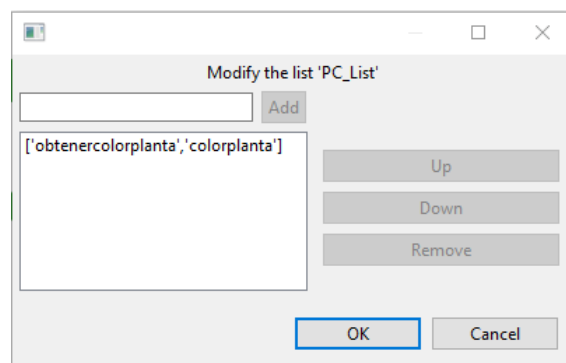


Imagen 5.10: Lista de clases pasivas.

5.3.2.1 Primer bloque de experimentos

Se analiza la replicación de código proveniente de un elemento de la comunidad. La comunidad esta compuesta por veinticinco elementos, donde veinticuatro de ellos cuentan con una versión 1.0 de Software y uno cuenta con la versión 2.0, que agrega dos clases funcionales, con el proceso de replicación y análisis de código una de las clases deberá verse ensamblada con las clases activas mientras que la otra deberá ensamblarse con la lista de clases pasivas.

La grafica 5.11, muestra el número de agentes con cada versión de software, veinticuatro agentes versión 1.0, un agente versión 2.0., el periodo de ciclos para la asignación de dispositivo coordinador y la posterior actualización de código fuente.

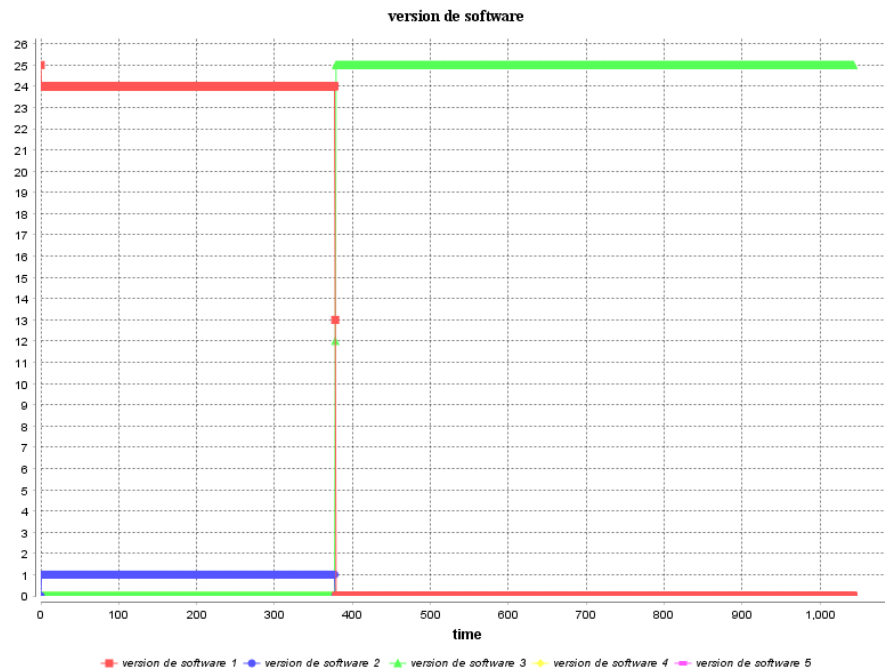


Imagen 5.11: Número de agentes con cada versión de software, 24 agentes versión 1.0, 1 agente versión 2.0.

Como parte de la misma simulación se obtuvo la información sobre las replications hechas, veinticinco replications de código tomando en cuenta que una vez realizado el ensamblaje se actualizan todos los nodos del sistema incluyendo a el nodo que contenía la versión que se ensamblo. Y el conteo de clases ensambladas resultante del ensamblado de las clases activas (cuatro clases) y las nuevas dos clases que portaba el dispositivo actualizado.

Las clases activas resultantes se muestran en la imagen 5.13. Se puede ver la presencia de una nueva clase que los dispositivos pueden ejecutar debido a su lista de capacidades.

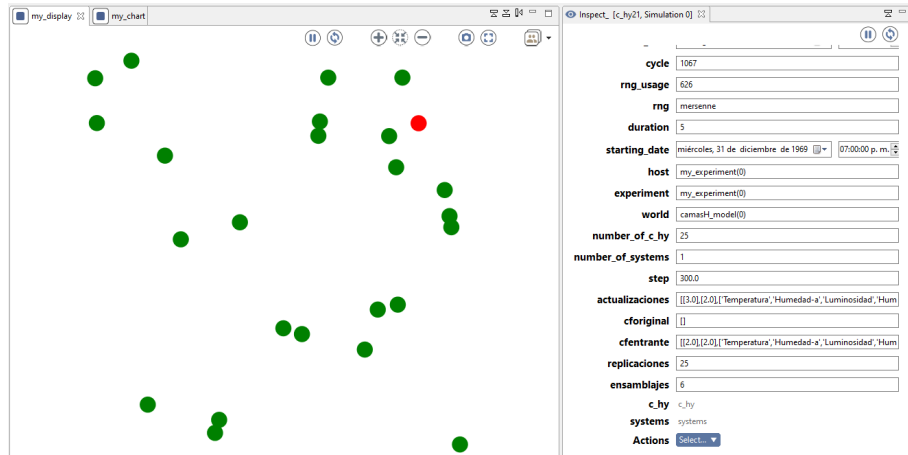


Imagen 5.12: Comunidad compuesta por veinticuatro agentes con dos clases a replicar.

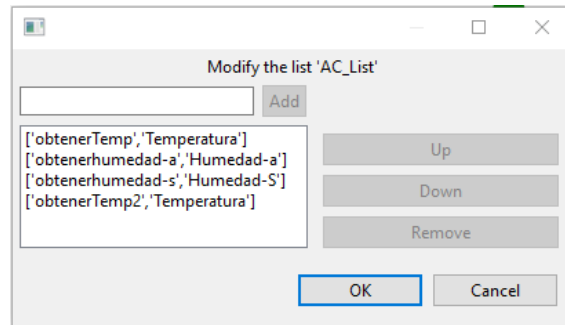


Imagen 5.13: Clases activas ensambladas, primer experimento.

Las clases pasivas obtenidas luego del proceso de ensamblaje se muestran en la imagen 5.14. Se puede ver la presencia de una nueva clase que los dispositivos no pueden ejecutar, por esto su asignación al bloque de clases pasivas.

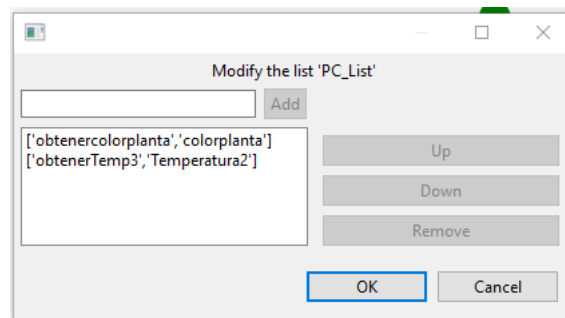


Imagen 5.14: Clases pasivas ensambladas, primer experimento.

Se realizo un segundo experimento con la presencia de siete clases a ensamblar, obteniendo resultados similares a los vistos en el primer experimento.

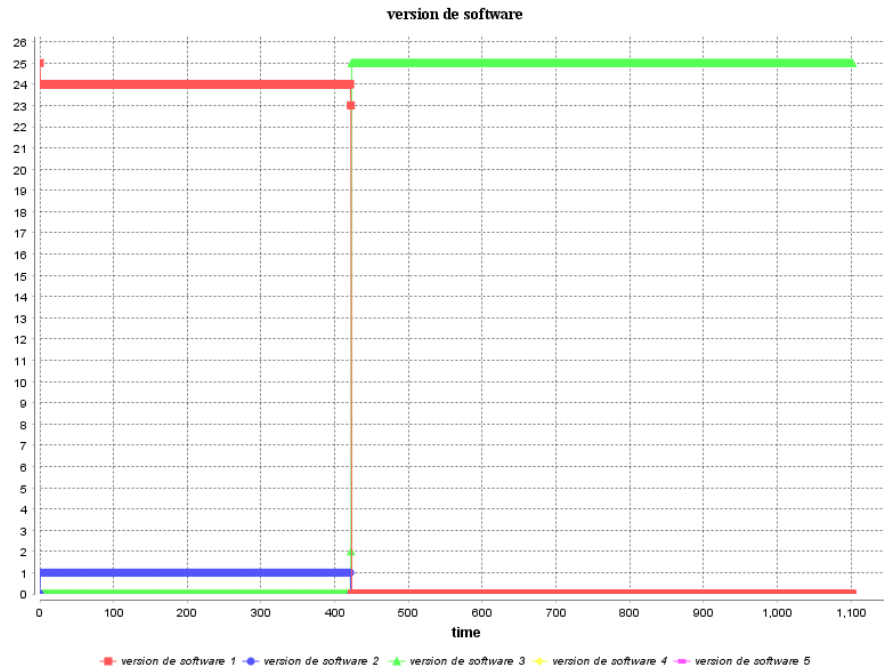


Imagen 5.15: Número de agentes con cada versión de software, veinticuatro agentes versión 1.0, un agente versión 2.0.

La información obtenida de esta simulación se muestra en la imagen 5.16, donde se obtuvieron once ensamblajes, siete de las nuevas clases y cuatro de las clases ya contenidas anteriormente, y una replicación a todos los dispositivos de la comunidad.

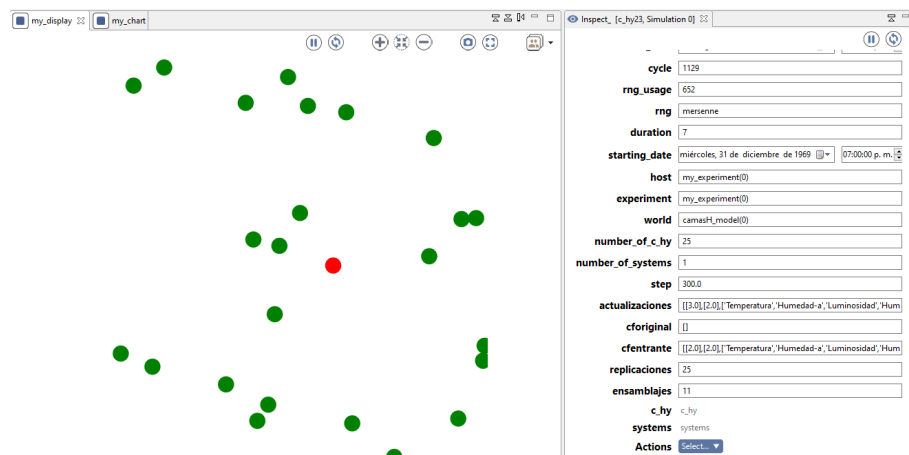


Imagen 5.16: Comunidad compuesta por veinticinco agentes con siete clases a replicar.

En las imágenes, 5.17 y 5.18 se puede observar las clases ensambladas en cada clases activas y clases pasivas respectivamente, esto por el análisis de ensamblaje con respecto a las capacidades de los dispositivos.

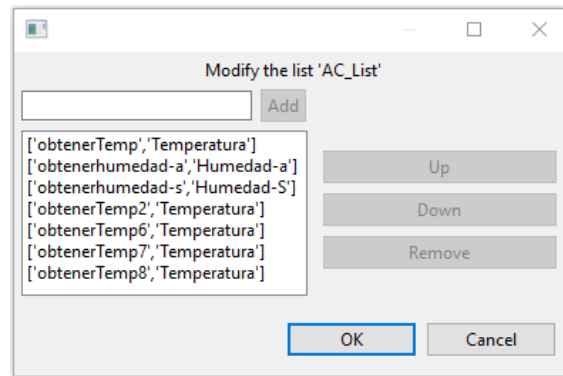


Imagen 5.17: Clases activas ensambladas, segundo experimento.

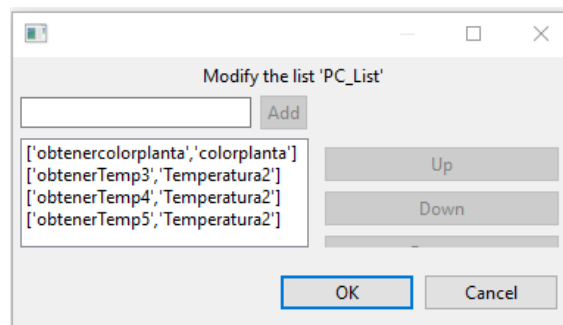


Imagen 5.18: Clases pasivas ensambladas, segundo experimento.

El siguiente experimento se llevo a cabo para una comunidad de veinticinco agentes con el objetivo de ver la replicación de catorce clases nuevas contenidas en un agente que fungirá como actualizador. Se obtienen resultados similares a los mostrados en los dos experimentos anteriores con una replicación exitosa de código fuente (imagen 5.19) y de sus clases activas (imagen 5.21) y pasivas (imagen 5.22), además de obtener el número de ensamblajes y replicaciones esperados, imagen 5.20.

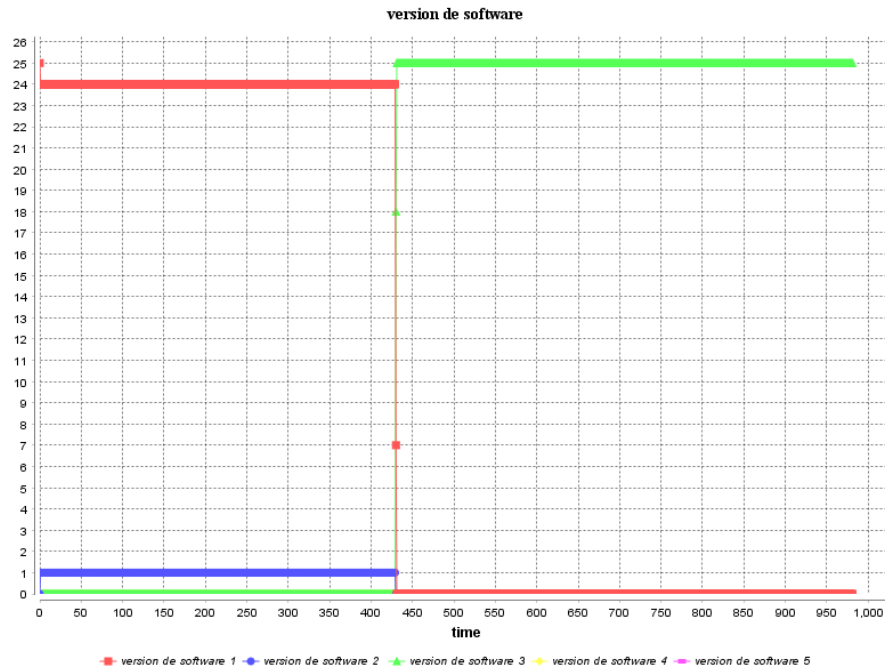


Imagen 5.19: Número de agentes con cada versión de software, veinticuatro agentes versión 1.0, un agente versión 2.0.

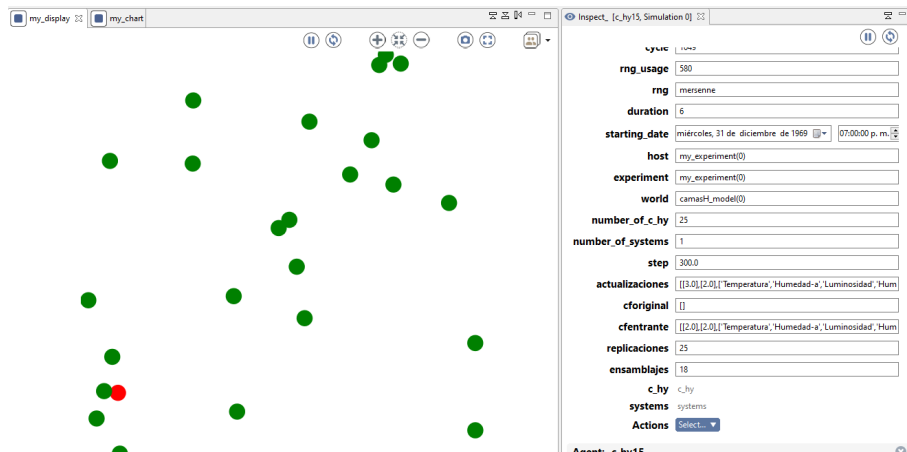


Imagen 5.20: Comunidad compuesta por veinticinco agentes con catorce clases a replicar.

5.3.2.2 Segundo bloque de experimentos

El segundo bloque de experimentos se realizó con comunidades de agentes con cincuenta elementos, esto para probar la escalabilidad del sistema y su correcto funcionamiento en replicación y ensamblaje de clases activas y pasivas.

La grafica 5.23, muestra el número de agentes con cada versión de software, cuarentainueve49 agentes versión 1.0, un agente versión 2.0, el periodo de ciclos para la asignación de dispositivo coordinador y la posterior actualización de código fuente. Se debe recalcar que la diferencia de ciclos para la replicación de código con respecto a las anteriores simulaciones mostradas, es originada por la selección de un líder, ya que

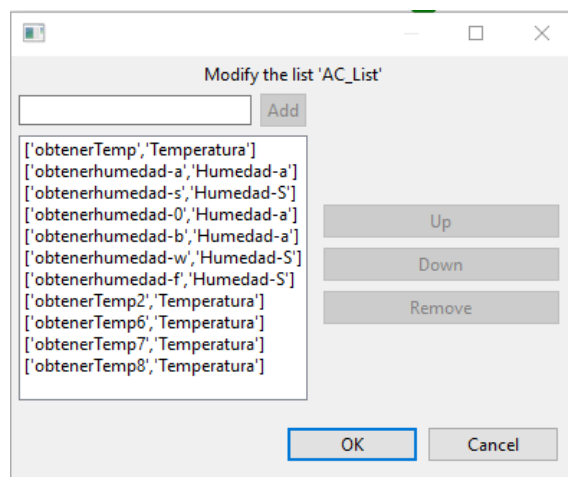


Imagen 5.21: Clases activas ensambladas, tercer experimento.

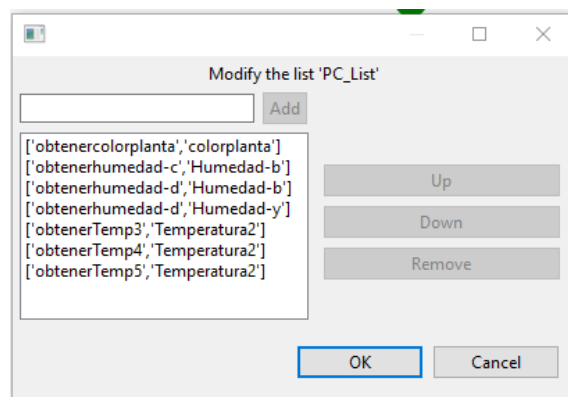


Imagen 5.22: Clases pasivas ensambladas, tercer experimento.

a mas volumen de dispositivos es mas tardado llegar a un consenso.

Se obtuvo la información sobre las replicaciones hechas, cuarentainueve replicaciones de código, de la misma manera que con las pruebas anteriormente realizadas, el conteo de clases ensambladas resultante del ensamblado de las clases activas (cuatro clases) y las nuevas dos clases que portaba el dispositivo actualizado, imagen 5.24.

En las imágenes, imagen 5.25 e imagen 5.26 se puede observar las clases ensambladas en cada clases activas y clases pasivas respectivamente, esto por el análisis de ensamblaje con respecto a las capacidades de los dispositivos.

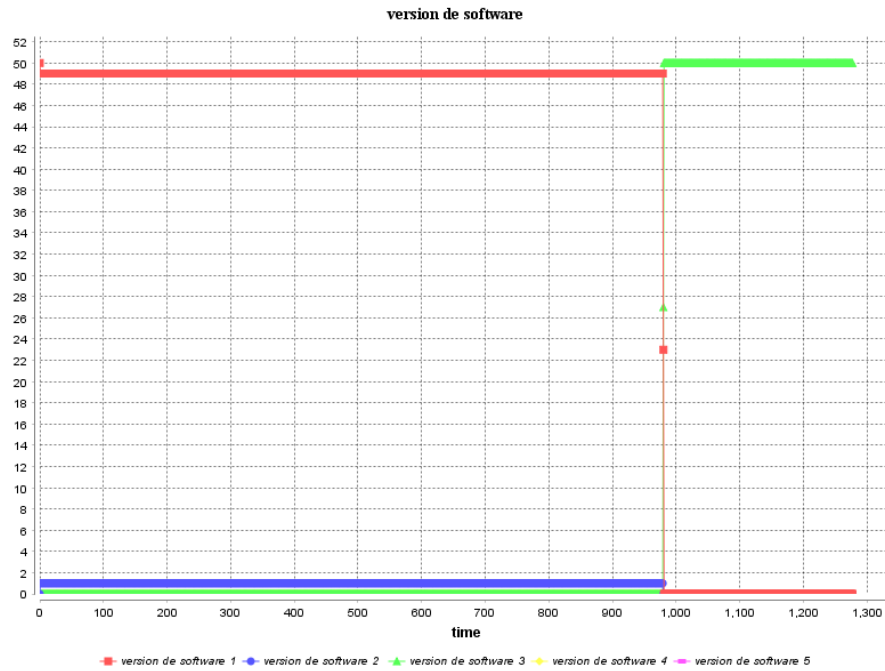


Imagen 5.23: Número de agentes con cada versión de software, cuarentainueve agentes versión 1.0, un agente versión 2.0.

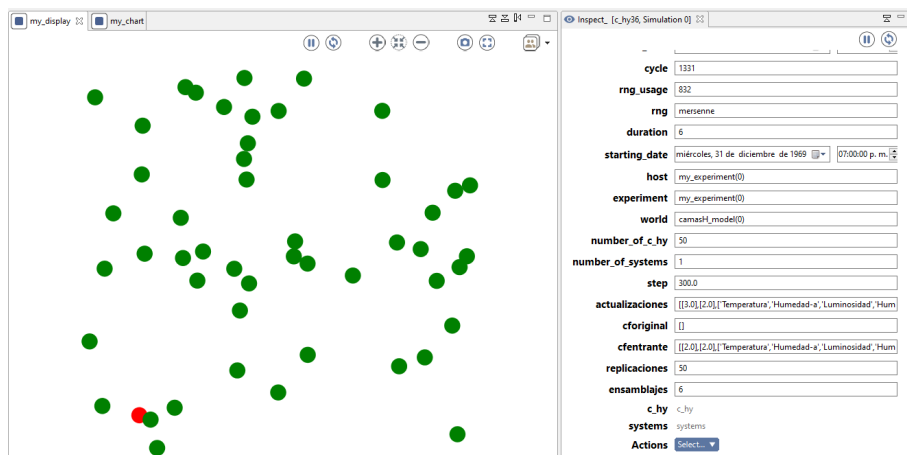


Imagen 5.24: Comunidad compuesta por cuarentainueve agentes con dos clases a replicar.

El quinto experimento se llevo a cabo para una comunidad de cincuenta agentes con el objetivo de ver la replicación de catorce clases nuevas contenidas en un agente que fungirá como actualizador. Se obtienen resultados similares a los mostrados en los dos experimentos anteriores con una replicación exitosa de código fuente (imagen 5.27) y de sus clases activas (imagen 5.29) y pasivas(imagen 5.30), además de obtener el número de ensamblajes y replicaciones esperados, imagen 5.28.

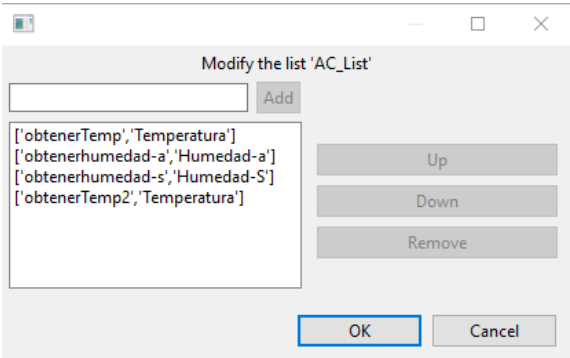


Imagen 5.25: Clases activas ensambladas, cuarto experimento.

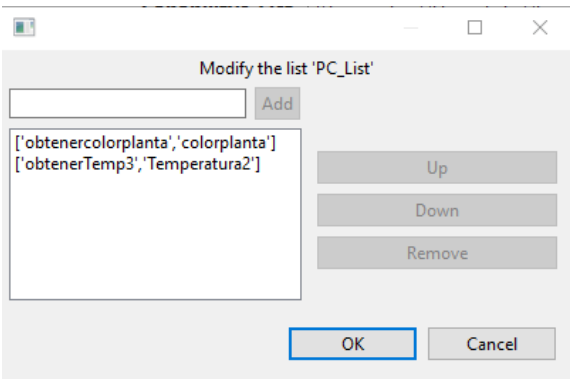


Imagen 5.26: Clases pasivas ensambladas, cuarto experimento.

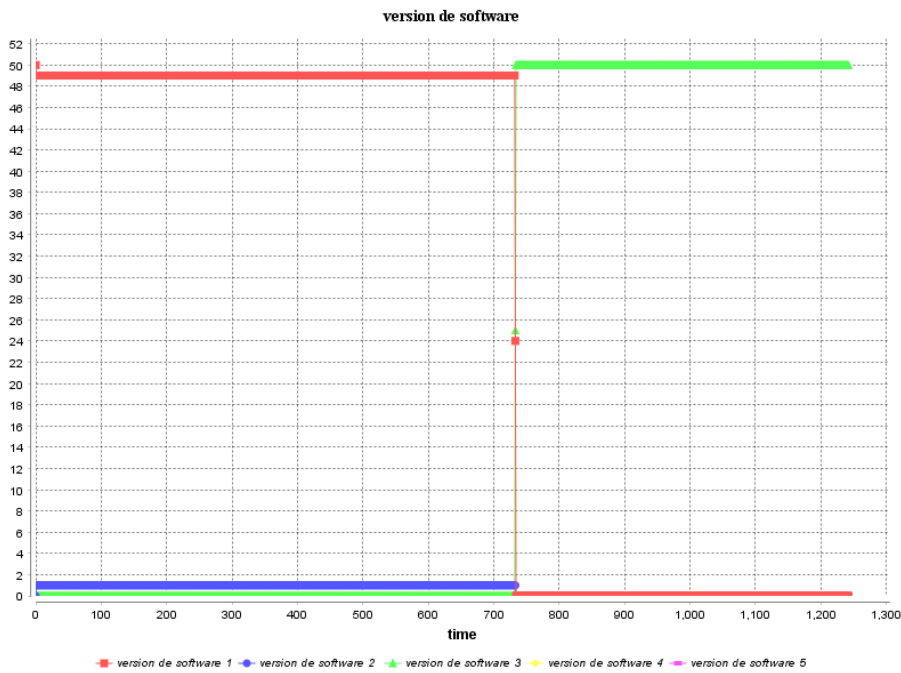


Imagen 5.27: Número de agentes con cada versión de software, cuarentainueve agentes versión 1.0, 1 agente versión 2.0.

El último experimento a analizar se compone de una comunidad de cincuenta agentes, se desea verificar la replicación de catorce nuevas clases. Estas clases se pensaron para que algunas fueran categorizadas como clases pasivas ya que la de-

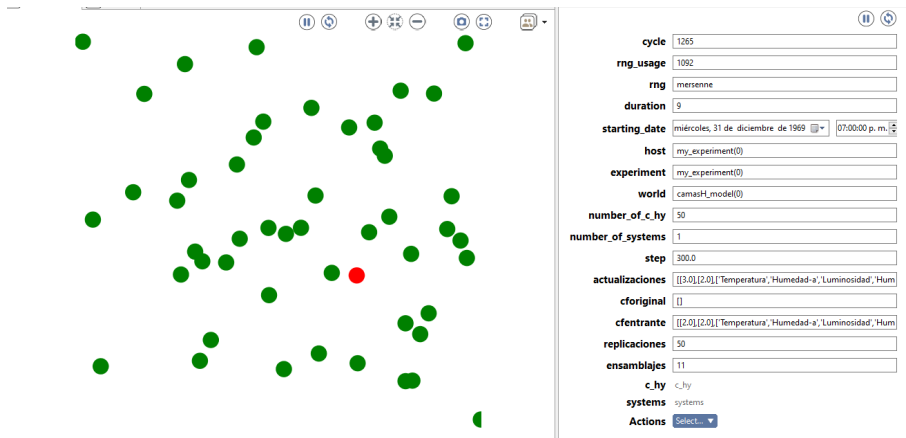


Imagen 5.28: Comunidad compuesta por cincuenta agentes con siete clases a replicar.

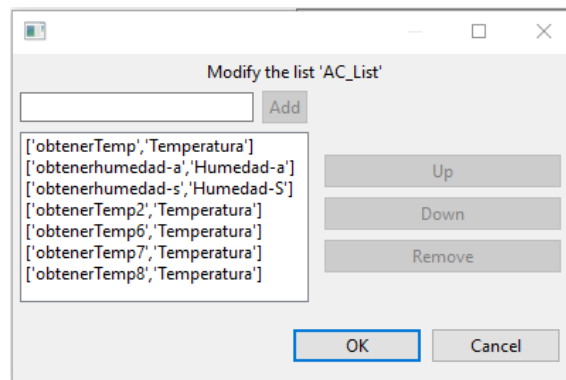


Imagen 5.29: Clases activas ensambladas, quinto experimento.

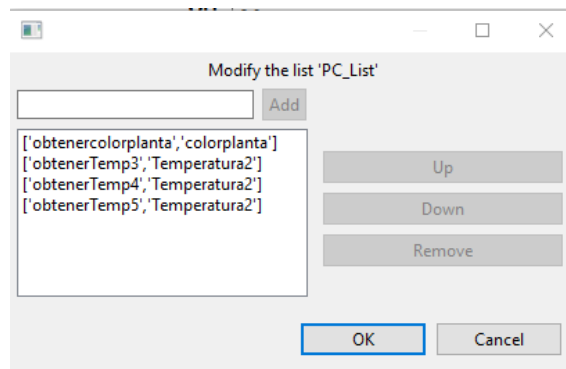


Imagen 5.30: Clases pasivas ensambladas, quinto experimento.

scripción de capacidad necesaria para ejecutarlas no se ajusta las capacidades de los agentes para los que se realizará la replicación.

En la imagen 5.31 podemos observar la actualización del código fuente alcanzando la versión 3.0 aproximadamente en el ciclo seiscientos diez de la simulación, y adquiriendo una población de cincuenta agentes con dicha versión de software.

El despliegue de la comunidad de agentes y la información concerniente a los Autoensamblajes y Autorreplicaciones se muestra en la imagen 5.32, obteniendo

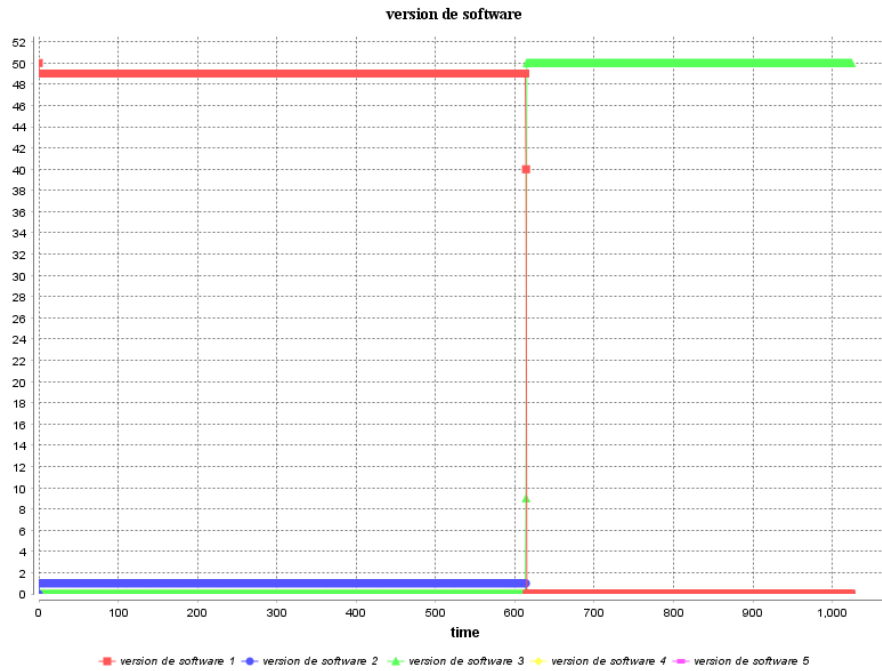


Imagen 5.31: Número de agentes con cada versión de software, 49 agentes versión 1.0, 1 agente versión 2.0.

cincuenta replicasiones del nuevo código fuente versión 3.0 y obteniendo un conteo de dieciocho ensamblajes de código, catorce por las nuevas clases funcionales y cuatro por las clases contenidas originalmente.

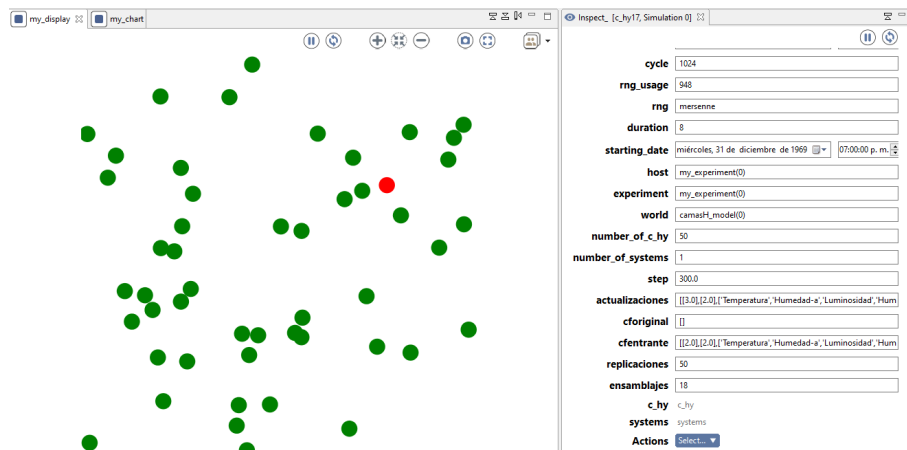


Imagen 5.32: Comunidad compuesta por cincuenta agentes con catorce clases a replicar.

En las imágenes, imagen 5.33 e imagen 5.34 se puede observar las clases ensambladas en cada clases activas y clases pasivas respectivamente.

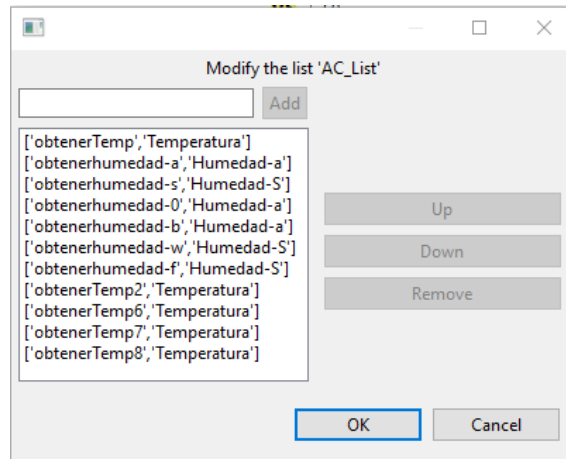


Imagen 5.33: Clases activas ensambladas, sexto experimento.

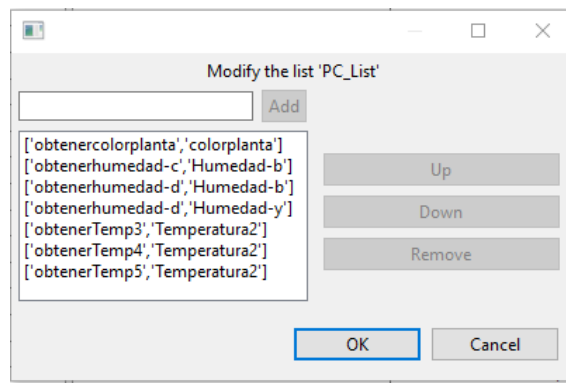


Imagen 5.34: Clases pasivas ensambladas, sexto experimento.

5.3.2.3 Análisis de resultados de simulación

En los resultados se observa que el número de ciclos requeridos de simulación aumenta razonablemente en función del tamaño de la comunidad de agentes y número de clases funcionales involucradas en el Autoensamblaje y la Autorreplicación. Es decir, aunque el número de clases aumentó en siete y el número de agentes se duplica, los ciclos resultantes son menores al doble de los observados en los escenarios con valores mínimos. En términos de implementación este incremento no resulta significativo y estaría acotado por las capacidades en hardware de los dispositivos, ya que, la tarea de ensamblaje de código depende de la capacidad del dispositivo que la realiza. Finalmente, se muestra que es posible que un sistema IoT pueda replicar parte de su funcionamiento y arquitectura de aplicación en otro sistema que así lo requiera.

Capítulo 6

Conclusiones y Trabajo Futuro

En este capítulo se mencionarán las contribuciones, los objetivos alcanzados y los objetivos a alcanzar en trabajo futuro. Si bien en este trabajo se responden algunas preguntas, abre muchas mas interrogantes a ser exploradas y resueltas.

6.1 Discusión

6.1.1 Contribución

Se propuso una arquitectura IoT que cuenta con las propiedades del Autoensamblaje y Autorreplicación que permite el ensamblaje de módulos de arquitecturas bajo el mismo campo de dominio y el ensamblaje de nuevos dispositivos en un sistema IoT, Permitiendo un acoplamiento de dispositivo IoT al sistema de manera automática o con intervención humana mínima, apoyándose totalmente en las propiedades autonómicas, puntualmente en la autoconfiguración a nivel dispositivo y en el Autoensamblaje a nivel código fuente.

Esta arquitectura propuesta fue documentada siguiendo el modelo de 4+1 de Kruchten, mostrando cual debe ser la estructura del código fuente de los dispositivos IoT para habilitar el análisis de este, la forma de análisis al que se someterá dicho código y el ensamblaje de este. También se muestra la secuencia de interacción o comunicación entre los agentes del sistema y los demás nodos en la arquitectura. Otros aspectos vistos son: el procedimiento de ensamblaje de un nuevo dispositivo en la comunidad de agentes y una descripción detallada de los componentes modificados de la arquitectura en la que se basó este trabajo.

En los resultados de simulación se observa que el número de ciclos requeridos aumenta razonablemente en función del tamaño de la comunidad de agentes y número de clases funcionales involucradas en el Autoensamblaje y la Autorreplicación. Es decir, aunque el número de clases aumentó en siete y el número de agentes se duplica, los ciclos resultantes son menores al doble de los observados en los escenarios con valores mínimos. En términos de implementación este incremento no resulta significativo y estaría acotado por las capacidades en hardware de los dispositivos. Finalmente, se muestra que un sistema IoT pueda replicar parte de su funcionamiento y arquitectura de aplicación en otro sistema que así lo requiera.

6.1.2 Objetivos logrados

Dentro de los objetivos que se lograron se encuentran los siguientes:

- Se realizó una integración de las propiedades de Autoensamblaje y Autorreplicación.
- Se establecieron los pasos para el Autoensamblaje de dispositivos y el Autoensamblaje y Autorreplicación de código fuente.
- Se documentó la arquitectura propuesta con múltiples vistas arquitectónicas.
- Se validó la propuesta.

6.2 Trabajo futuro

El área del trabajo futuro concerniente a este tipo de arquitecturas y propiedades es enorme, ya que nos permite pensar en un futuro donde los sistemas IoT pudieran generarse de manera espontánea y administrarse con una mínima intervención humana, esto, claro, en un futuro tal vez no tan lejano.

Una de las principales áreas a tomar en cuenta en trabajos futuros debe ser el mecanismo de análisis de código fuente, ya que, si bien, el mecanismo propuesto nos ayuda a probar la hipótesis, es un mecanismo que puede ser ampliado y mejorado, ayudando a la identificación de capos de dominio y mejor ensamblaje de código.

Otra área que debe ser atendida de manera inmediata deberá ser un mecanismo para la identificación de arquitecturas IoT autonómicas Autorreplicables y Autoensamblables que muestre la confiabilidad de estas y algunas métricas de calidad, además de un mecanismo de identificación de código malicioso que pudiera verse contenido en alguna versión de algún dispositivo y replicado por la naturaleza de la arquitectura.

También se deberá atender la búsqueda de mecanismos para el análisis del funcionamiento de los sistemas IoT que proponga configuraciones o modificaciones para hacer más eficiente el sistema.

Bibliografía

- [1] Simon Kemp. *DIGITAL 2020: 3.8 BILLION PEOPLE USE SOCIAL MEDIA*. 2020. URL: <https://wearesocial.com/blog/2020/01/digital-2020-3-8-billion-people-use-social-media> (visited on 01/30/2020).
- [2] Kevin Ashton et al. “That ‘internet of things’ thing”. In: *RFID journal* 22.7 (2009), pp. 97–114.
- [3] Industrial Internet Consortium. *INDUSTRIAL INTERNET SECURITY FRAMEWORK TECHNICAL REPORT*. 2019. URL: <https://www.iiconsortium.org/IISF.htm> (visited on 09/05/2019).
- [4] Steve Heath. *Embedded systems design*. Elsevier, 2002.
- [5] Rajeev Alur. *Principles of cyber-physical systems*. MIT Press, 2015.
- [6] ISO/IEC JTC1. “Internet of Things (IoT)”. In: (2015).
- [7] Arshdeep Bahga and Vijay Madisetti. *Internet of Things: A hands-on approach*. Vpt, 2014.
- [8] Michael Wooldridge. *An introduction to multiagent systems*. John Wiley & Sons, 2009.
- [9] Yoav Shoham, Kevin Leyton-Brown, et al. “Multiagent systems”. In: *Algorithmic, Game-Theoretic, and Logical Foundations* (2009).
- [10] Peter Naur and Brian Randell. “Report on a conference sponsored by the NATO Science Committee”. In: 8 (1968).
- [11] Toni A Bishop and Ramesh K Karne. “A Survey of Middleware.” In: (2003), pp. 254–258.
- [12] Paul Horn. “Autonomic computing: IBM’s perspective on the state of information technology”. In: (2001).
- [13] Jeffrey O Kephart and David M Chess. “The vision of autonomic computing”. In: *Computer* 36.1 (2003), pp. 41–50.
- [14] Mohammad Reza Nami and Koen Bertels. “A survey of autonomic computing systems”. In: *Third International Conference on Autonomic and Autonomous Systems (ICAS’07)*. IEEE. 2007, pp. 26–26.
- [15] Len Bass, Paul Clements, and Rick Kazman. *Software architecture in practice*. Addison-Wesley Professional, 2003.
- [16] Philippe Kruchten. “Architectural Blueprints—The 4+ 1 View Model of Software Architecture, Software, Vol. 12, Number 6”. In: (1995).
- [17] Klaus Kronlöf. *Method integration: concepts and case studies*. John Wiley & Sons, Inc., 1993.

- [18] CM Eckert and MK Stacey. “What is a process model? Reflections on the epistemology of design process models”. In: *Modelling and Management of Engineering Processes*. Springer, 2010, pp. 3–14.
- [19] P Michael Politano, Robert O Walton, and Donna L Roberts. *Introduction to the Process of Research: Methodology Considerations*. Lulu. com, 2017.
- [20] M Endrei et al. “Ibm: Patterns: service-oriented architecture and web services”. In: *Biztek. edu. pk* (2004).
- [21] Norbert Bieberstein et al. *Service-oriented architecture compass: business value, planning, and enterprise roadmap*. FT Press, 2006.
- [22] Ian Sommerville. *Software engineering 9th Edition*. 2011, p. 18.
- [23] The Open Group. *Service-Oriented Architecture – What Is SOA?* 2020. URL: <http://www.opengroup.org/soa/source-book/soa/p1.htm> (visited on 04/12/2020).
- [24] Microsoft. *Estilo de arquitectura de microservicios*. 2019. URL: <https://docs.microsoft.com/es-es/azure/architecture/guide/architecture-styles/microservices> (visited on 10/30/2019).
- [25] Sam Newman. *Building microservices: designing fine-grained systems.*” O’Reilly Media, Inc.”, 2015.
- [26] John Apostolopoulos. *Why is intent-based networking good news for software-defined networking*. 2018. URL: <https://blogs.cisco.com/analytics-automation/why-is-intent-based-networking-good-news-for-software-defined-networking>.
- [27] Sakir Sezer et al. “Are we ready for SDN? Implementation challenges for software-defined networks”. In: *IEEE Communications Magazine* 51.7 (2013), pp. 36–43.
- [28] Pablo Javier Patiño and María Teresa Rugeles López. “Replicación del ADN.” In: *Fondo Editorial Biogénesis* (2006), pp. 119–132.
- [29] William Aspray. *John von Neumann and the origins of modern computing*. Vol. 191. Mit Press Cambridge, MA, 1990.
- [30] János Neumann, Arthur W Burks, et al. *Theory of self-reproducing automata*. Vol. 1102024. University of Illinois press Urbana, 1966.
- [31] David Alejandro Reyes Gómez. *Descripción y Aplicaciones de los Autómatas Celulares*. 2011.
- [32] V Huber-Dyson. “DR Hofstadter Gödel, Escher, Bach: an Eternal Golden Braid (New York: Basic Books Inc. 1979).” In: *Canadian Journal of Philosophy* 11.4 (1981), pp. 775–792.
- [33] Patricia Hill and John Gallagher. “Meta-programming in logic programming”. In: *Handbook of Logic in Artificial Intelligence and Logic Programming* 5 (1998), pp. 421–497.
- [34] Tiago Fernández-Caramés and Paula Fraga-Lamas. “A Review on the Use of Blockchain for the Internet of Things”. In: *IEEE Access* 6 (May 2018), pp. 32979–33001. DOI: 10.1109/ACCESS.2018.2842685.

- [35] Mayra Samaniego, Uurtsaikh Jamsrandorj, and Ralph Deters. “Blockchain as a Service for IoT”. In: *2016 IEEE international conference on internet of things (iThings) and IEEE green computing and communications (GreenCom) and IEEE cyber, physical and social computing (CPSCoM) and IEEE smart data (SmartData)*. IEEE. 2016, pp. 433–436.
- [36] Luis Eduardo Villela Zavala. “Arquitectura y Algoritmo de Gestión para Redes IoT Autónomas”. MA thesis. CINVESTAV Guadalajara, Aug. 2018.
- [37] Adan Octavio Ruiz Martinez. “Arquitectura de Referencia para Sistemas IoT en Ciudades Inteligentes (SCRAIoT)”. PhD thesis. CINVESTAV Guadalajara, Dec. 2019.
- [38] B. Manate, V. I. Munteanu, and T. Fortis. “Towards a Scalable Multi-agent Architecture for Managing IoT Data”. In: *2013 Eighth International Conference on P2P, Parallel, Grid, Cloud and Internet Computing*. 2013, pp. 270–275.
- [39] Z. Movahedi et al. “A Survey of Autonomic Network Architectures and Evaluation Criteria”. In: *IEEE Communications Surveys Tutorials* 14.2 (2012), pp. 464–490.
- [40] Rajkumar Roy and Sam Brooks. “Self-engineering – Technological Challenges”. In: May 2020, pp. 16–30. ISBN: 978-3-030-46816-3. DOI: 10.1007/978-3-030-46817-0_2.
- [41] Shūichi Fukuda. *Self Engineering: Learning from Failures*. Springer, 2019.
- [42] Hee Lee et al. “Wireless networks self engineering engine”. In: Feb. 1999, pp. 258–263. ISBN: 0-7695-0122-2. DOI: 10.1109/ASSET.1999.756777.
- [43] Hanying Li, Joshua Carter, and Thomas Labean. “Nanofabrication by DNA Self-Assembly”. In: *Materials Today - MATER TODAY* 12 (May 2009), pp. 24–32. DOI: 10.1016/S1369-7021(09)70157-9.
- [44] Yoon Seob Lee. *Self-Assembly and Nanotechnology: A Force Balance Approach*. Jan. 2008.
- [45] J Storrs Hall. “Architectural considerations for self-replicating manufacturing systems”. In: *Nanotechnology* 10.3 (Aug. 1999), pp. 323–330. DOI: 10.1088/0957-4484/10/3/316. URL: <https://doi.org/10.1088/0957-4484/10/3/316>.
- [46] Robert Ewaschuk and Peter Turney. “Self-Replication and Self-Assembly for Manufacturing”. In: *Artificial life* 12 (Feb. 2006), pp. 411–33. DOI: 10.1162/artl.2006.12.3.411.
- [47] Roberto Hernández Sampieri, Carlos FERNÁNDEZ-COLLADO, and Pilar BAPTISTA-LUCIO. “Metodología de la investigación Mc Graw Hill”. In: *México DF: Interamericana Editores* (2014).
- [48] Gordana Dodig-Crnkovic. “Scientific methods in computer science”. In: *Proceedings of the Conference for the Promotion of Research in IT at New Universities and at University Colleges in Sweden, Skövde, Suecia*. 2002, pp. 126–130.

- [49] Elly Amani Gamukama and Oliver Popov. “The Level of Scientific Methods Use in Computing Research Programs”. In: *31st International Convention on Information and Communication Technology*. 2008, pp. 176–183.
- [50] Carla Silva, Jaelson Castro, and Patricia Tedesco. “Requirements for Multi-Agent Systems.” In: (Jan. 2003), pp. 198–212.
- [51] GAMA-Platform. *GAMA*. 2020. URL: <http://gama-platform.org>.

Capítulo 7

Anexo

7.1 Imágenes

En esta sección se muestran algunas imágenes escaladas que pudieran no apreciarse totalmente en el documento principal.

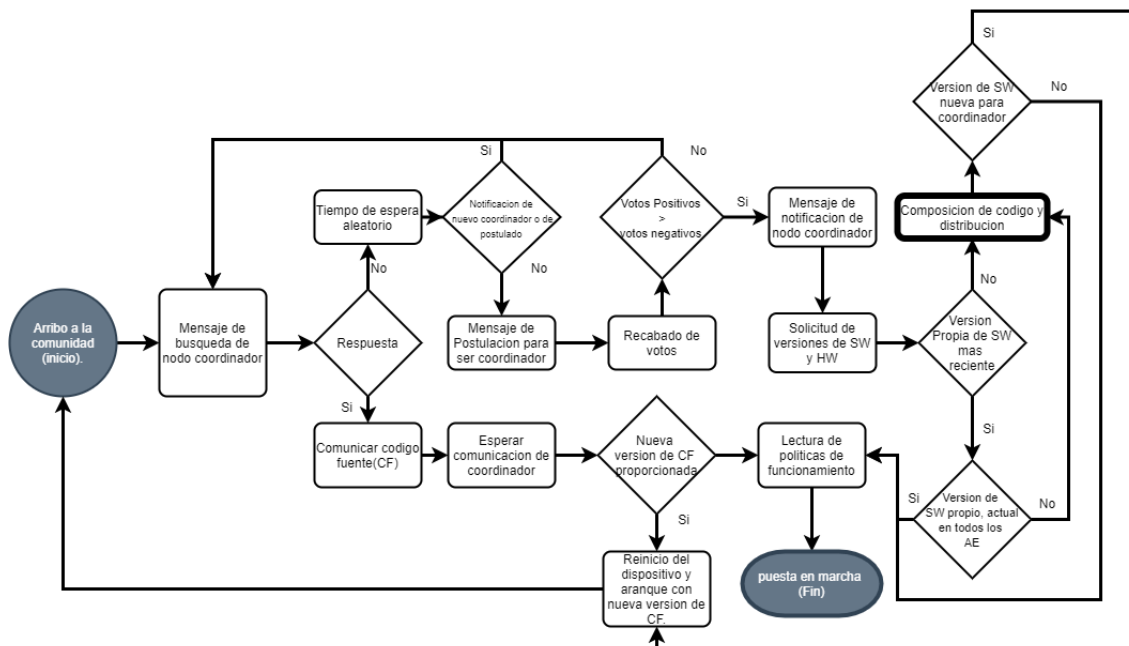


Imagen 4.7: Ensamblaje de dispositivo IoT a una comunidad de AEs.

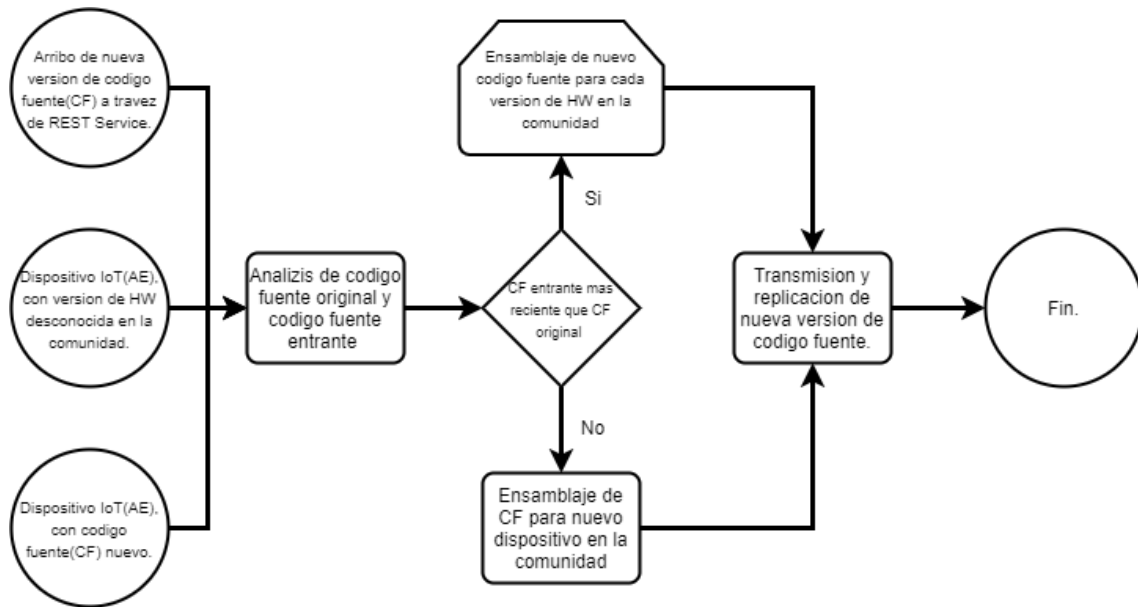


Imagen 4.8: Eventos desencadenadores de autoensamblaje.

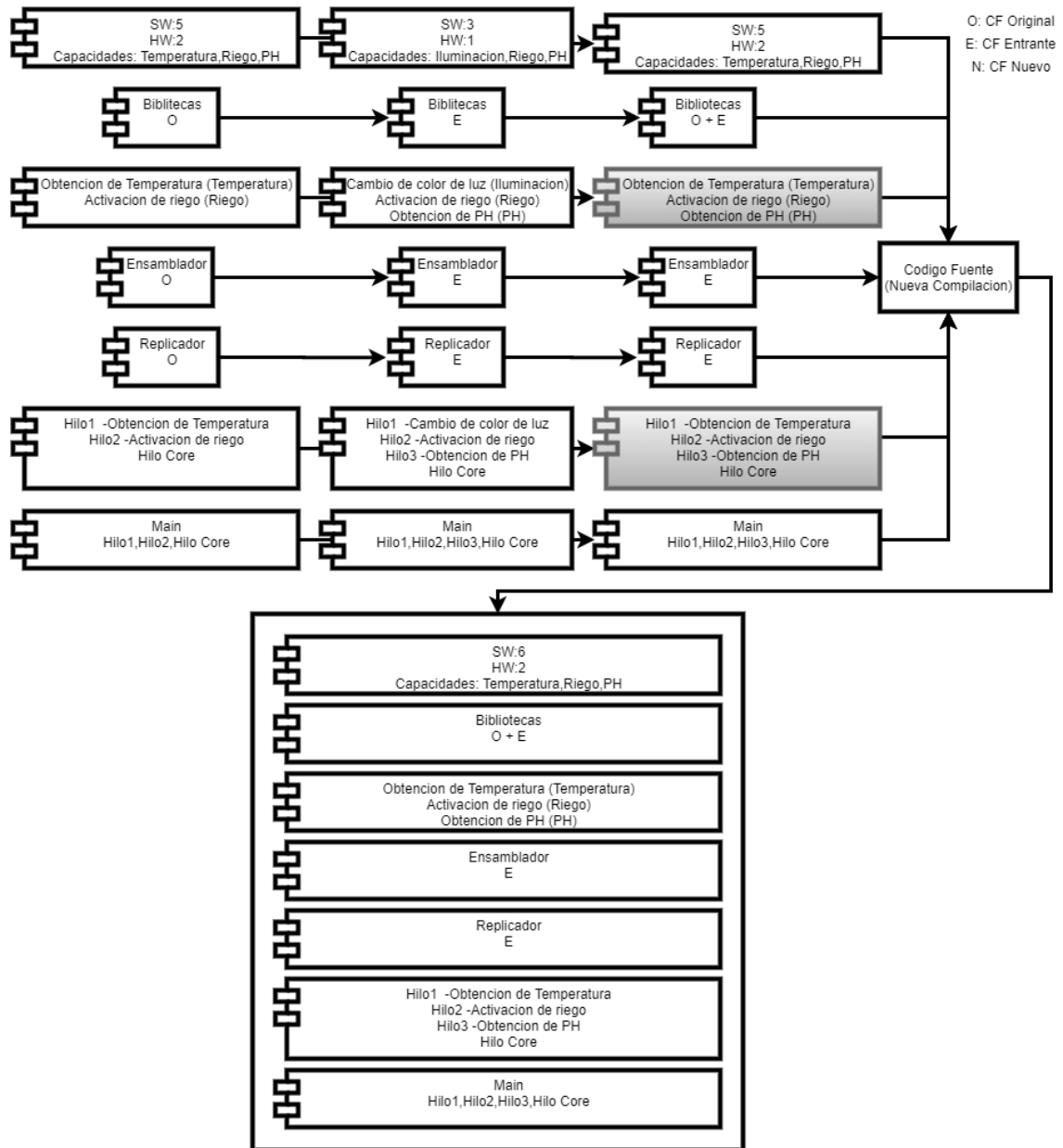


Imagen 5.5: Fusión de clases de código foráneo.

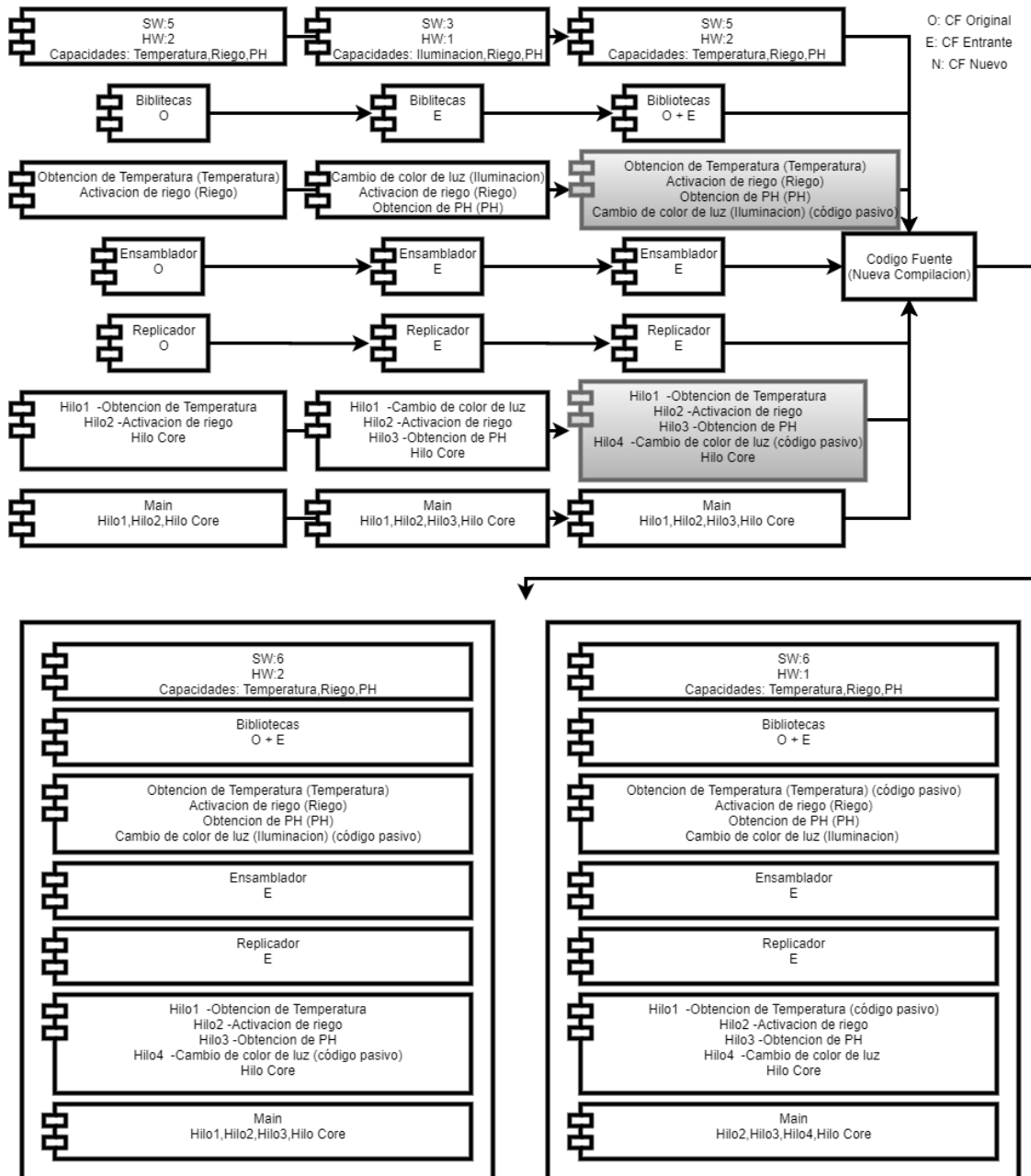


Imagen 5.7: Fusión de clases de código de dispositivo entrante.

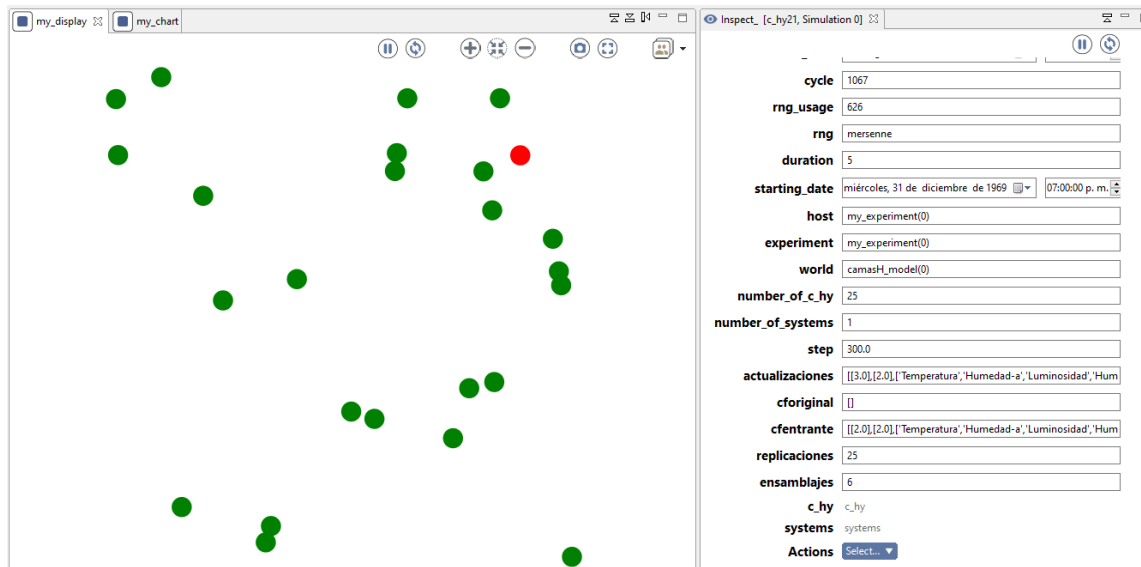


Imagen 5.12: Comunidad compuesta por 25 agentes con 2 clases a replicar.

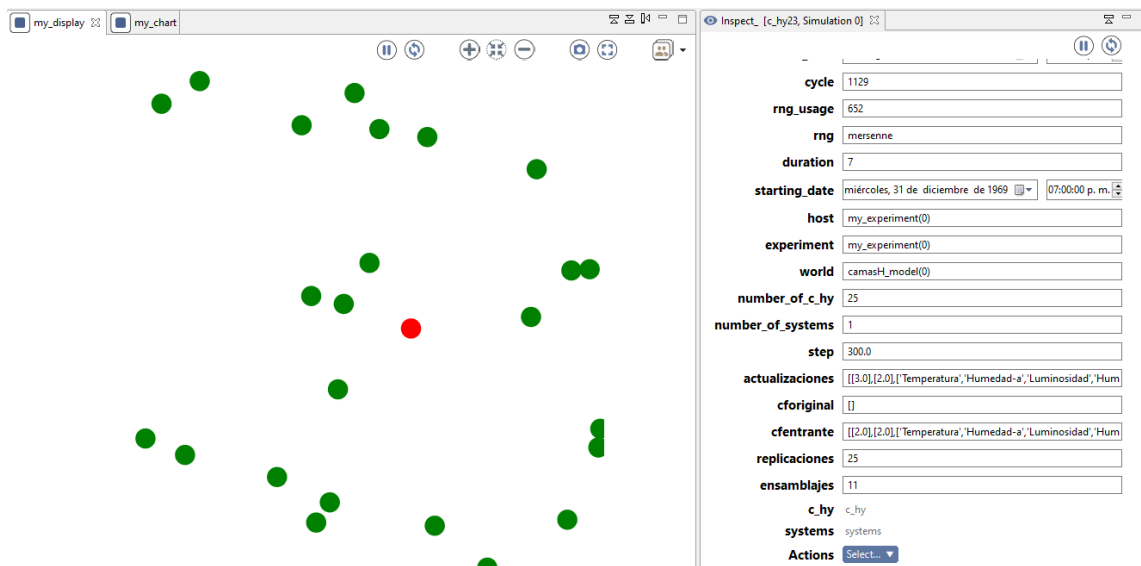


Imagen 5.16: Comunidad compuesta por 25 agentes con 7 clases a replicar.

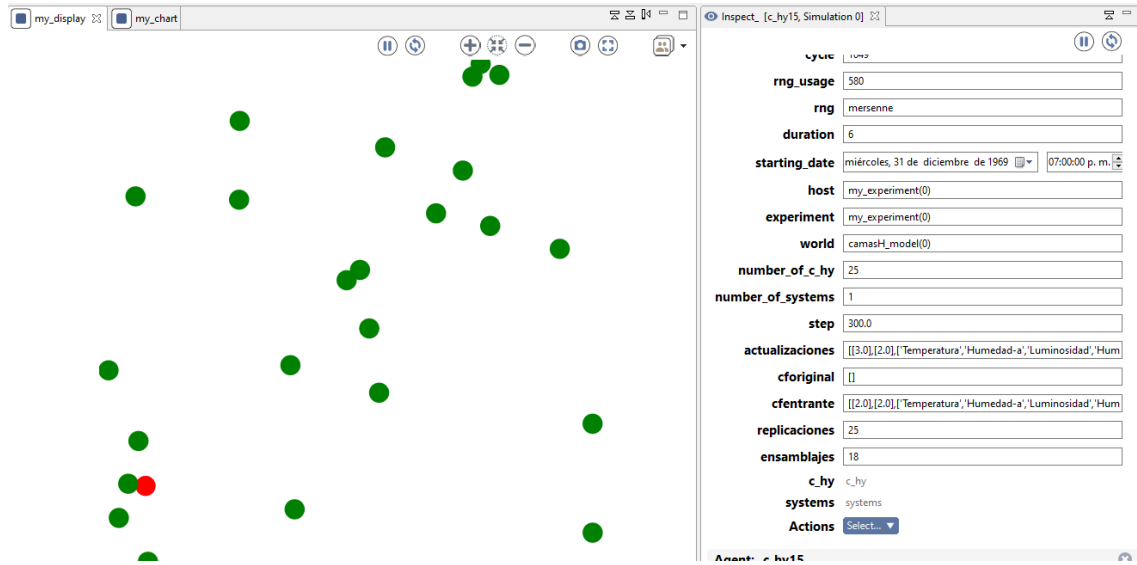


Imagen 5.20: Comunidad compuesta por 25 agentes con 14 clases a replicar.

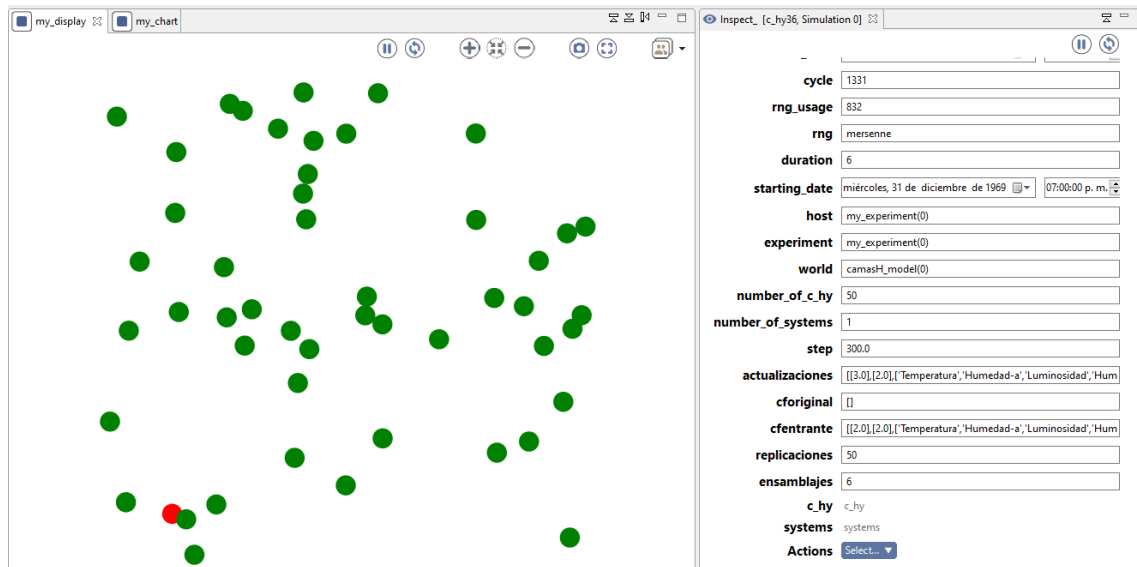


Imagen 5.24: Comunidad compuesta por 49 agentes con 2 clases a replicar.

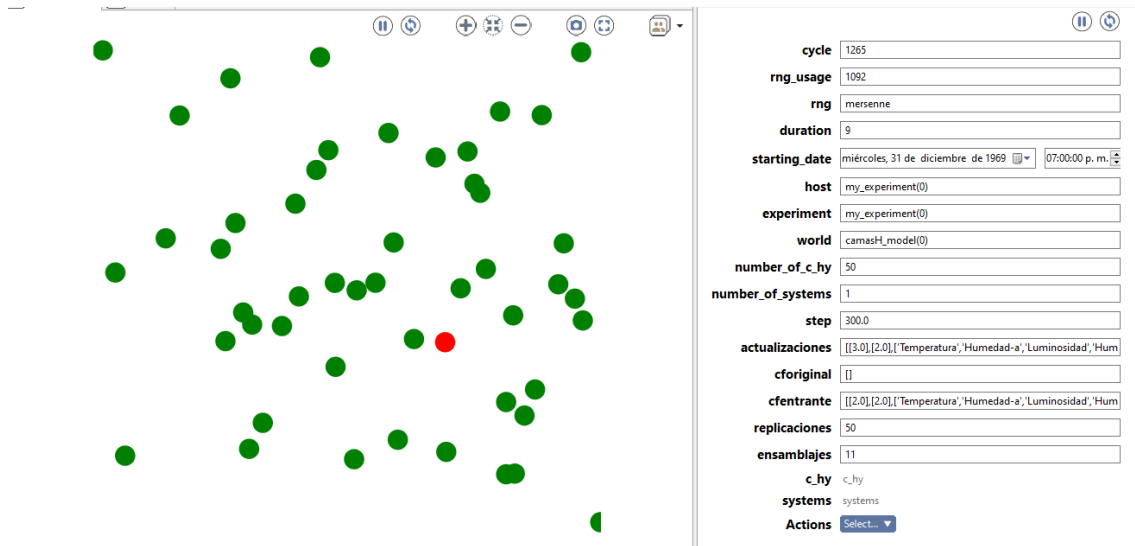


Imagen 5.28: Comunidad compuesta por 50 agentes con 7 clases a replicar.

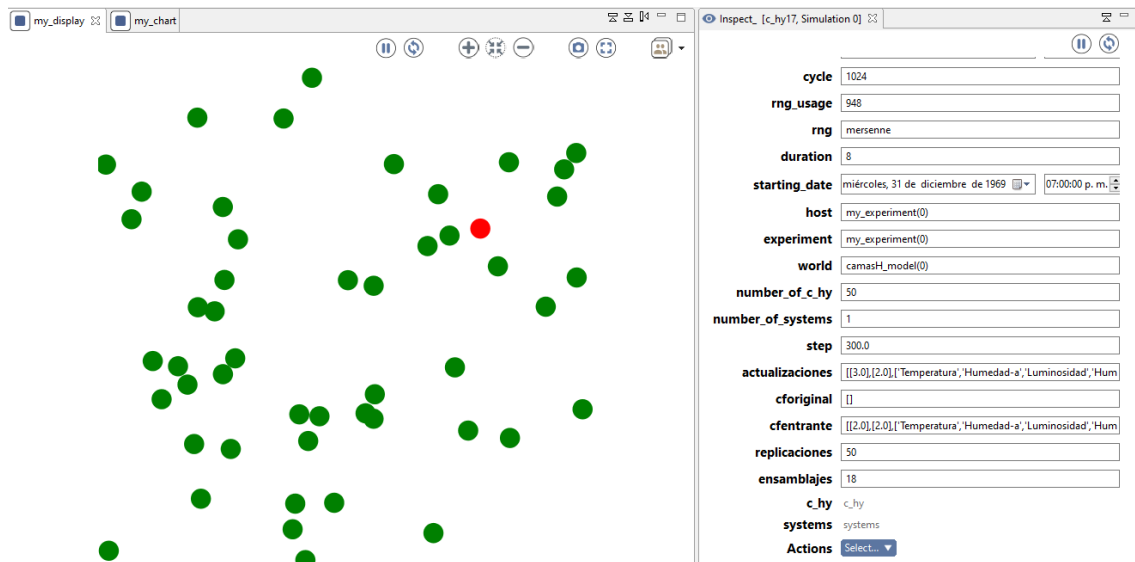


Imagen 5.32: Comunidad compuesta por 50 agentes con 14 clases a replicar.

7.2 Código de Simulación

model camash

```
global {
  int number_of_c_hy <- 50;
  int number_of_systems<-1;
  float step <- 5 #mn;
  init{
    create c_hy number:number_of_c_hy;
  }
  list<list> actualizaciones<-[];
  list<list> cforiginal<-[];
  list<list> cfentrante<-[];
  int replications<-0;
  int ensamblajes<-0;
}

species c_hy skills: [fipa] {
  bool is_coordin <- false;
  bool exist_coordinat <-false;
  float VS<- 1.0;
  float VH<- 2.0;
  list<string> Capabilities_List <- ["Temperatura", "Humedad-
a", "Luminosidad", "Humedad-S", "ColordeLuz", "PH"];
  list<string> libery_List <- ["DTH", "Random", "OS", "Math"];
  list<list> AC_List <- [ ["obtenerTemp", "Temperatura"], ["obtenerhumedad-
a", "Humedad-a"], ["obtenerhumedad-s", "Humedad-S"]];
  list<list> PC_List <- [ ["obtenercolorplanta", "colorplanta"]];
  string ensamblador<- "ensambladorv2";
  string replicador<- "replicadorv1";
  list<list> hiloscore<- [ ["obtener-datos-temperatura", "Temperatura"], ["obtener-
datos-humedad", "Humedad-a"]];
  list<string> hiloreplicador<- ["hiloversion1", ""];
  list<list> main<- [ ["obtener-datos-temperatura", "Temperatura"], ["obtener-datos-
humedad", "Humedad-a"], ["hiloversion1", ""]];
  list<list> versioneshw<-[];
  bool listversion<-false;
  bool comp_sw_reciente <-false;
  list<list> compilado<-[];

  int cycle_comp<-0;
  int cycle_compfin<-0;
  bool compr <-false;

  bool voto <- false;
  int cycle_vote <- 0;
  int votospositivos<-0;
  int votosNegativos<-0;
  bool postulate <-false;
  int timerrecolecciondeversiones<-0;

  reflex comilatuverson when: compilado=[] {
    add [VS] to:compilado;
    add [VH] to:compilado;
```

```

    add Capabilitys_List to:compilado;
    add libery_List to:compilado;
    add AC_List to:compilado;
    add PC_List to:compilado;
    add [ensamblador] to:compilado;
    add [replicador] to:compilado;
    add hiloscore to:compilado;
    add hiloreplicador to:compilado;
    add main to:compilado;
}

    reflex Ncoor when: !exist_coordinat and !is_coordin and cycle_vote=0 and
!postulate{
    do start_conversation to: list(c_hy) protocol: 'fipa-contract-net'
performative: 'cfp' contents: ['ruhere'];
    cycle_compfin<-cycle_comp+10;
}
    reflex myScoor when: is_coordin and !listversion{
    listversion<-true;
    do start_conversation to: list(c_hy) protocol: 'fipa-contract-net'
performative: 'cfp' contents: ['dametuversiondeswhw'];
}

    reflex capturacompletadeversiones when: timerrecolecciondeversiones!=0 and
timerrecolecciondeversiones=cycle and is_coordin{
    timerrecolecciondeversiones<-0;
    list<list> nueva <-[];
    loop i over: versioneshw{
        if(!(nueva contains i)){
            add i to:nueva;
        }
    }
    versioneshw<-nueva;
    comp_sw_reciente<-true;
    float versionvs<-VS;
    bool myversionres<-true;
    loop i over: versioneshw{
        float k <-float(i[1][0]);
        if(k > versionvs){
            versionvs<-k;
        }
    }
    if(versionvs>VS){
        do start_conversation to: list(c_hy) protocol: 'fipa-contract-
net' performative: 'cfp' contents: [versionvs];
    }
    else{
        if(length(versioneshw)!=1){
            do start_conversation to: list(c_hy) protocol: 'fipa-
contract-net' performative: 'cfp' contents: [versionvs];
        }
        else{
            write "todos actualizados";
        }
    }
}

```



```

    }
}

reflex postulate when: cycle_vote!=0{
  if cycle_vote=cycle
  {
    postulate<-true;
    do start_conversation to: list(c_hy) protocol: 'fipa-contract-
net' performative: 'cfp' contents: ['vote'];
  }
  else if cycle=cycle_vote+number_of_c_hy{
    postulate<-false;
    if(votospositivos>votosNegativos){
      is_coordin<-true;
      exist_coordinat<-true;
      do start_conversation to: list(c_hy) protocol: 'fipa-
contract-net' performative: 'cfp' contents: ['imcoordin'];
    }
    else{
      do start_conversation to: list(c_hy) protocol: 'fipa-
contract-net' performative: 'cfp' contents: ['imNcoordin'];
    }
    votospositivos<-0;
    votosNegativos<-0;
  }
}

reflex comp when: cycle_comp!=0 and cycle_comp=cycle and !is_coordin{
  do start_conversation to: list(c_hy) protocol: 'fipa-contract-
net' performative: 'cfp' contents: ['ruhere'];
  cycle_compfin<-cycle_comp+10;
}

reflex finconteo when: cycle_compfin=cycle and cycle!=0{
  cycle_compfin<-0;
  if compr{
    cycle_comp<-cycle_comp+rnd(50,100);
    compr<-false;
    exist_coordinat<-true;
  }
  else{
    cycle_vote<- cycle +rnd (1, (number_of_c_hy * 100));
    voto <- false;
    votospositivos<-0;
    votosNegativos<-0;
    postulate <-true;
    exist_coordinat <-false;
    cycle_comp<-0;
  }
}

reflex receive_inform_messages when: !empty(informs) {
  loop i over: informs {

```

```

switch i.contents[0][0]{
  match 'info'{
    add i.contents to:versioneshw;
  }
  default{
  }
}
}
timerrecolecciondeversiones<-cycle+10;
do end_conversation message: i contents: ['ok'];
}

reflex receive_cfp when: !empty(cfps) {
  message proposalFromInitiator <- cfps[0];
  list respuesta <-proposalFromInitiator.contents;
  switch respuesta[0]{
    match 'vote'{
      if(voto){
        do refuse message: proposalFromInitiator contents:
['vn'];
      }
      else{
        voto<-true;
        do refuse message: proposalFromInitiator contents:
['vp'];
      }
    }
    match 'imcoordin'{
      exist_coordinat<-true;
      postulate<-false;
      cycle_vote<-0;
      cycle_comp<-cycle+rnd(50,100);
      votospositivos<-0;
      votosNegativos<-0;
      do refuse message: proposalFromInitiator contents: ['ok'];
    }
    match 'imNcoordin'{
      voto<-false;
      do refuse message: proposalFromInitiator contents: ['ok'];
    }
    match 'ruhere'{
      if(is_coordin){
        do propose message: proposalFromInitiator contents:
['imhcoordin'];

        cycle_comp<-0;
        compr<-true;
      }
      else{
        do refuse message: proposalFromInitiator contents:
['ok'];
      }
    }
  }
  match 'dametuversiondeswhw'{

```

```

do inform message: proposalFromInitiator contents:
[['info'],[VS],[VH]];
}
match VS{

cfentrante<-compilado;
do refuse message: proposalFromInitiator contents: ['cfe'];
}
match 'nuevaversion'{
replicaciones<-replicaciones+1;
VS<-float(actualizaciones[0][0]);
AC_List <- actualizaciones[4];
PC_List <- actualizaciones[5];
ensamblador<-actualizaciones[6];
replicador<-actualizaciones[7];
hiloscore<-actualizaciones[8];
hiloreplicador<-actualizaciones[9];
main<-actualizaciones[10];
compilado<-[];
do refuse message: proposalFromInitiator contents: ['ok'];
}
default{

}

}

}

```

```

reflex receive_refuse_messages when: !empty(refuses) {
loop r over: refuses {
try{

switch r.contents[0]{
match 'vp'{
votospositivos<-votospositivos+1;
}
match 'vn'{
votosNegativos<-votosNegativos+1;
}
match 'imhcoordin'{
cycle_comp<-cycle+rnd(60,100);
compr<-true;
}
match 'ok'{

}
match 'cfe'{
list<list> entrante <- cfentrante;
list<list> compilado2<-[];
list<list> activas<-[];
list<list> pasivas<-entrante[5];
loop i over: entrante[4]{
if(Capabilitys_List contains i[1]){
add i to:activas;
}
}
}
}
}

```

```

        ensamblajes<-ensamblajes+1;
    }
    else
    {
        add i to:pasivas;
        ensamblajes<-ensamblajes+1;
    }
}
loop i over: AC_List{
    if(!(activas contains i)){
        add i to:activas;
    }
}
float auxx <-float(entrante[0][0]) +1;
add [auxx] to:compilado2;
add [VH] to:compilado2;
add Capabilities_List to:compilado2;
add entrante[3] to:compilado2;
add activas to:compilado2;
add pasivas to:compilado2;
add entrante[6] to:compilado2;
add entrante[7] to:compilado2;
add entrante[8] to:compilado2;
add entrante[9] to:compilado2;
add entrante[10] to:compilado2;
actualizaciones<-compilado2;
entrante<-[];
ensamblajes<-ensamblajes+1;
do start_conversation to: list(c_hy) protocol: 'fipa-
contract-net' performative: 'cfp' contents: ['nuevaversion'];
}
default{
}
}
}
}
}
}

reflex receive_propose_messages when: !empty(proposes) {
    loop p over: proposes {
        switch p.contents[0]{
            match 'imhcoordin'{
                cycle_comp<-cycle+rnd(60,100);
                compr<-true;
                do accept_proposal message: p contents:
[['info'],[VS],[VH]] ;
            }
        }
    }
}

```

```

    reflex receive_accept_proposals when: !empty(accept_proposals) {
      message a <- accept_proposals[0];
      list respuesta <-a.contents;
      switch respuesta[0][0]{
        match 'info'{
          }
        }
      }
    }

    aspect base {
      draw circle(2) color: (is_coordin) ? #red : #green;
    }
  }

  species systems{

  }

  experiment my_experiment type:gui{
    parameter "numero de dispositivos:" var: number_of_c_hy;
    output{
      display my_display{
        species c_hy aspect:base;
      }
      display my_chart{
        chart "version de software"{
          data "version de software 1" value: length (c_hy where
(each.VS=1));
          data "version de software 2 " value: length (c_hy where
(each.VS=2));
          data "version de software 3" value: length (c_hy where
(each.VS=3));
          data "version de software 4" value: length (c_hy where
(each.VS=4));
          data "version de software 5" value: length (c_hy where
(each.VS=4));
        }
      }
    }
  }
}

```